

**IMPLEMENTASI YOLOV8 DAN RAY CASTING UNTUK DETEKSI
SERTA PENGELOMPOKAN MAKANAN BERDASARKAN
HUBUNGAN SPASIAL DENGAN WADAH (STUDI KASUS SISTEM
SELF-SERVICE PRASMANAN HAJI SUMI)**

TUGAS AKHIR



Disusun Oleh :
GREGORIUS RAFAEL SANTOSA
123210102

**PROGRAM STUDI INFORMATIKA
JURUSAN INFORMATIKA
FAKULTAS TEKNIK INDUSTRI
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN"
YOGYAKARTA
2026**

**IMPLEMENTASI YOLOV8 DAN RAY CASTING UNTUK DETEKSI
SERTA PENGELOMPOKAN MAKANAN BERDASARKAN
HUBUNGAN SPASIAL DENGAN WADAH (STUDI KASUS SISTEM
SELF-SERVICE PRASMANAN HAJI SUMI)**

TUGAS AKHIR

Tugas Akhir ini sebagai salah satu syarat untuk memperoleh gelar sarjana Informatika
Universitas Pembangunan Nasional “Veteran” Yogyakarta



Disusun Oleh :

GREGORIUS RAFAEL SANTOSA
123210102

**PROGRAM STUDI INFORMATIKA
JURUSAN INFORMATIKA
FAKULTAS TEKNIK INDUSTRI
UNIVERSITAS PEMBANGUNAN NASIONAL “VETERAN”
YOGYAKARTA
2026**

HALAMAN PENGESAHAN PEMBIMBING

IMPLEMENTASI YOLOV8 DAN RAY CASTING UNTUK DETEKSI SERTA PENGELOMPOKAN MAKANAN BERDASARKAN HUBUNGAN SPASIAL DENGAN WADAH (STUDI KASUS SISTEM SELF-SERVICE PRASMANAN HAJI SUMI)

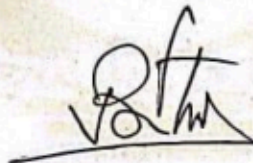
Disusun Oleh:

Gregorius Rafael Santosa

123210102

Telah diuji dan dinyatakan lulus oleh pembimbing
pada tanggal: ...4 Juni...2026

Menyetujui,
Pembimbing



Budi Santosa, S.Si., M.T.

NIDN. 0510097001

Mengetahui,
Koordinator Program Studi



Dessyanto Boedi Prasetyo, S.T., M.T.

NIDN. 0505127501

HALAMAN PENGESAHAN PENGUJI

IMPLEMENTASI YOLOV8 DAN RAY CASTING UNTUK DETEKSI SERTA PENGELOMPOKAN MAKANAN BERDASARKAN HUBUNGAN SPASIAL DENGAN WADAH (STUDI KASUS SISTEM SELF-SERVICE PRASMANAN HAJI SUMI)

Disusun Oleh:
Gregorius Rafael Santosa
123210102

Telah diuji dan dinyatakan lulus oleh penguji
pada tanggal: 4 Juni 2026

Menyetujui,

Penguji 1



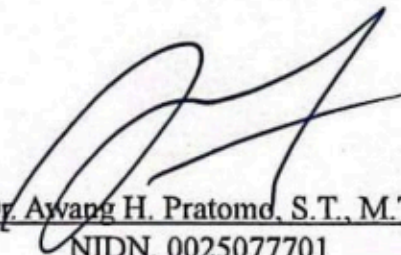
Budi Santosa, S.Si., M.T.
NIDN. 0510097001

Penguji 2



Bambang Yuwono, S.T., M.T.
NIDN. 0512027401

Penguji 3



Dr. Awang H. Pratomo, S.T., M.T.
NIDN. 0025077701

Penguji 4



Andrey Ferriyan, S.T., M.Cs., Ph.D.
NUPTK. 1255761662130193

SURAT PERNYATAAN KARYA ASLI TUGAS AKHIR

Sebagai mahasiswa Program Studi Informatikai Fakultas Teknik Industri Universitas Pembangunan Nasional “Veteran” Yogyakarta, yang bertanda tangan di bawah ini, saya:

Nama : Gregorius Rafael Santosa
NIM : 123210102

Menyatakan bahwa karya ilmiah saya yang berjudul:
Implementasi YOLOv8 Dan Ray Casting Untuk Deteksi Serta Pengelompokan Makanan Berdasarkan Hubungan Spasial Dengan Wadah (Studi Kasus Sistem *Self-Service* Prasmanan Haji Sumi)

Merupakan karya asli saya dan belum pernah dipublikasikan dimanapun. Apabila di kemudian hari, karya saya disinyalir bukan merupakan karya asli saya, maka saya bersedia menerima konsekuensi apa pun yang diberikan Program Studi Informatika Fakultas Teknik Industri Universitas Pembangunan Nasional “Veteran” Yogyakarta kepada saya.

Demikian surat pernyataan ini saya buat dengan sebenarnya.

Dibuat di : Yogyakarta
Pada Tanggal : 4 Juni 2026

Yang Menyatakan



Gregorius Rafael Santosa
NIM. 123210102

PERNYATAAN BEBAS PLAGIASI

Saya yang bertanda tangan di bawah ini:

Nama : Gregorius Rafael Santosa
NIM : 123210102
Prodi : Informatika

Dengan ini saya menyatakan bahwa judul Tugas Akhir
Implementasi YOLOv8 Dan Ray Casting Untuk Deteksi Serta Pengelompokan Makanan Berdasarkan Hubungan Spasial Dengan Wadah (Studi Kasus Sistem *Self-Service* Prasmanan Haji Sumi)

adalah hasil kerja saya dan benar bebas dari plagiasi kecuali cuplikan serta ringkasan yang terdapat di dalamnya telah saya jelaskan sumbernya (Sitasi) dengan jelas. Apabila pernyataan ini terbukti tidak benar maka saya bersedia menerima sanksi sesuai peraturan Mendiknas RI No 17 Tahun 2010 dan Peraturan Perundang-undangan yang berlaku.

Demikian surat pernyataan ini saya buat dengan penuh tanggung jawab.

Dibuat di : Yogyakarta
Pada Tanggal : 4 Juni 2026

Yang Menyatakan



Gregorius Rafael Santosa
NIM. 123210102

ABSTRAK

Peningkatan kebutuhan pelanggan terhadap efektivitas dan efisiensi layanan mendorong berkembangnya teknologi otomatisasi, salah satunya Self-Service Technology (SST). Dalam konteks rumah makan prasmanan, proses perhitungan makanan sering dilakukan secara manual sehingga dapat menyebabkan kesalahan perhitungan, antrean panjang, serta kurang efisiennya proses pelayanan. Oleh sebab itu, dibutuhkan pendekatan yang sanggup mendeteksi dan mengelompokkan makanan secara otomatis agar proses perhitungan menjadi lebih cepat dan akurat. Penelitian ini bermaksud untuk menguji kemampuan deteksi dan pengelompokan makanan menggunakan kombinasi model YOLOv8-seg sebagai metode deteksi objek serta algoritma Ray-Casting untuk menentukan hubungan spasial makanan terhadap wadahnya. Studi kasus penelitian dilakukan pada rumah makan prasmanan Haji Sumi.

Metode penelitian diawali dengan pengumpulan data primer dan sekunder. Tahapan preprocessing meliputi pembersihan data, anotasi, pembagian data, resize, serta augmentasi data. Selanjutnya dilakukan pelatihan model YOLOv8-seg untuk mendeteksi objek makanan dan wadah, kemudian diterapkan algoritma Ray-Casting untuk menentukan apakah objek makanan berada di dalam suatu wadah. Data sekunder sebanyak 4068 gambar digunakan sebagai pre-trained dataset, sedangkan data primer sebanyak 3264 gambar digunakan untuk proses fine-tuning. Untuk pengujian sistem, digunakan 360 gambar data primer yang mewakili berbagai variasi kondisi jumlah wadah dan tingkat *overlapping*. Kinerja model dievaluasi menggunakan metrik recall, precision, dan mAP.

Hasil penelitian menunjukkan bahwa model YOLOv8-seg mampu mendeteksi objek makanan dan wadah dengan baik pada sistem self-service prasmanan. Pengujian model menggunakan parameter epoch 50, learning rate 0.001, batch size 8, serta IoU 0.7 menghasilkan nilai precision sebesar 83.7%, recall sebesar 79.6%, mAP@50 sebesar 82.1%, dan mAP@50-95 sebesar 58.4%, dengan peningkatan sebesar 39.7% pada mAP@50-95 dibandingkan model baseline. Hasil confusion matrix menunjukkan bahwa sebagian besar objek makanan berhasil dideteksi dengan benar, terutama pada kondisi non-overlapping, meskipun pada kondisi overlapping masih ditemukan beberapa kesalahan klasifikasi dan objek yang tidak terdeteksi akibat objek saling bertumpang tindih. Selain itu, algoritma Ray Casting mampu mengelompokkan objek makanan yang berhasil dideteksi ke dalam wadah yang sesuai dengan tingkat akurasi F1-Score sebesar 86%. Implementasi sistem secara real-time menggunakan kamera sebagai input utama juga berhasil dilakukan, di mana sistem mampu menampilkan hasil deteksi, segmentasi, dan pengelompokan objek makanan secara langsung pada antarmuka aplikasi. Dengan demikian, kombinasi YOLOv8-seg dan algoritma Ray Casting dinilai efektif untuk mendukung otomatisasi identifikasi dan pengelompokan makanan pada layanan self-service prasmanan secara real-time.

Kata Kunci: Deteksi Makanan, Pengelompokan Makanan, YOLOv8, Algoritma Ray-Casting, Makanan Prasmanan

ABSTRACT

The increasing demand for service effectiveness and efficiency has driven the development of automation technologies, one of which is Self-Service Technology (SST). In the context of buffet restaurants, the process of calculating food items is often performed manually, which may lead to calculation errors, long queues, and inefficient service processes. **Therefore, a system that can automatically detect and group food items is needed to make the calculation process faster and more accurate.** This study is intended to assess the performance of a food detection and grouping technique using a combination of the YOLOv8-seg model as an object detection method and the Ray-Casting algorithm to determine the spatial relationship between food items and their containers. The case study of this research was conducted at the Haji Sumi buffet restaurant.

The research process started with the gathering of both primary and secondary data. The preprocessing stage included data cleaning, annotation, dataset splitting, resizing, and data augmentation. Subsequently, the YOLOv8-seg model was trained to detect food objects and containers, followed by the implementation of the Ray-Casting algorithm to identify whether a food object is located within a specific container. A total of 4068 secondary images were used as the pre-trained dataset, while 3264 primary images were utilized for the fine-tuning process. For system evaluation, 24 primary images representing various scenarios of container quantities and overlapping conditions were tested using the developed system. The models performance was assessed using recall, precision, and mAP.

The research indicate that the YOLOv8-seg model is capable of detecting food objects and containers effectively in a self-service buffet system. Model testing using parameters of 50 epochs, a learning rate of 0.001, a batch size of 8, and an IoU threshold of 0.7 achieved a precision of 83.7%, recall of 79.6%, mAP@50 of 82.1%, and mAP@50–95 of 58.4%, representing an improvement of 39.7% in mAP@50–95 compared to the baseline model. The confusion matrix results show that most food objects were successfully detected correctly, particularly under non-overlapping conditions, although several misclassifications and undetected objects were still found under overlapping conditions due to object occlusion. In addition, the Ray Casting algorithm was able to group detected food objects into the appropriate containers with F1-Score an accuracy rate of 86%. The real-time system implementation using a camera as the primary input was also successfully conducted, where the system was able to display detection, segmentation, and food object grouping results directly through the application interface. Therefore, the combination of YOLOv8-seg and the Ray Casting algorithm is considered effective in supporting automated food identification and grouping in a real-time self-service buffet system.

Keywords: *Food Detection, Food Grouping, YOLOv8, Ray-Casting Algorithm, Prasmanan Food*

KATA PENGANTAR

Penulis menyartakan puji syukur kepada Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga penulisan skripsi ini dapat diselesaikan. Judul yang dipilih oleh penulis dalam skripsi ini ialah "Implementasi YoloV8 dan Ray Casting Untuk Deteksi Serta Pengelompokan Makanan Berdasarkan Hubungan Spasial Dengan Wadah (Studi Kasus Sistem Self-Service Prasmanan Haji Sumi)"

Dalam penyelesaian skripsi ini tentu tidak lepas dari dukungan, bimbingan dan pengarahan dari berbagai pihak. Oleh karena itu saya ingin mengucapkan rasa terima kasih yang sebesar-besarnya kepada semua pihak yang telah turut serta mendukung dan membantu saya dalam menyelesaikan kegiatan ini. Pada kesempatan kali ini saya ingin mengucapkan terima kasih kepada pihak-pihak tersebut diantaranya:

1. Tuhan Yang Maha Esa atas rahmat, petunjuk, dan kesempatan yang diberikan.
2. Orang tua serta keluarga yang selalu memberikan saya nasihat dan motivasi saat berlangsungnya penelitian ini.
3. Bapak Prof. Dr. Mohamad Irhas Effendi, M.Si. selaku Rektor Universitas Pembangunan Nasional "Veteran" Yogyakarta.
4. Bapak Dr. Awang Hendrianto Pratomo, S.T., M.T. selaku Dekan Fakultas Teknik Industri Universitas Pembangunan Nasional "Veteran" Yogyakarta
5. Bapak Budi Santosa S.Si., M.T. selaku Dosen Pembimbing dalam penelitian ini.
6. Bapak Bambang Yuwono, S.T., M.T., Dr. Awang H. Pratomo, S.T., M.T., dan Andrey Ferriyan, S.T., M.Cs., Ph.D. selaku dosen penguji dalam penelitian ini.
7. Bapak dan Ibu Dosen/Staf pengajar Program Studi Informatika Fakultas Teknik Industri Universitas Pembangunan Nasional "Veteran" Yogyakarta
8. Ibu dr Yulyani Werdiningsih, Sp. PD., FINASIM selaku tenaga medis yang memberikan penanganan kesehatan selama penyusunan skripsi.

Adanya skripsi ini saya harapkan tidak hanya berguna untuk pencapaian pribadi saja tetapi juga menjadi pedoman yang bermanfaat bagi konteks yang relevan khususnya di bidang Machine Learning.

Yogyakarta, 4 Juni 2026



Penulis

DAFTAR ISI

HALAMAN PENGESAHAN PEMBIMBING	iii
HALAMAN PENGESAHAN PENGUJI.....	iv
SURAT PERNYATAAN KARYA ASLI TUGAS AKHIR.....	v
PERNYATAAN BEBAS PLAGIASI.....	vi
ABSTRAK	vii
ABSTRACT	viii
KATA PENGANTAR	ix
DAFTAR ISI	x
DAFTAR TABEL.....	xii
DAFTAR GAMBAR.....	xiii
DAFTAR PSEUDOCODE	xv
BAB I PENDAHULUAN	1
1.1 Latar Belakang Masalah	1
1.2 Rumusan Masalah.....	2
1.3 Batasan Masalah	2
1.4 Tujuan Penelitian	3
1.5 Manfaat Penelitian	3
1.6 Tahapan Penelitian.....	3
1.7 Sistematika Penulisan	6
BAB II TINJAUAN LITERATUR.....	7
2.1 Self-Service Technology	7
2.2 Computer Vision	7
2.3 Instance Segmentation	8
2.4 Convolutional Neural Network (CNN).....	9
2.5 Deep Learning.....	11
2.6 YOLOv8-seg.....	12
2.6.1 Arsitektur YOLOv8-seg.....	14
2.6.2 Backbone.....	15
2.7 Waterfall.....	18
2.8 Analisis Spatial	19
2.9 Matriks Evaluasi	20
2.10 Tinjauan Literatur.....	21
BAB III METODOLOGI PENELITIAN.....	23
3.1 Pengumpulan Data.....	23
3.2 Pre-Processing	25
3.3 Implementasi Model	27
3.4 Proses Lokalisasi Makanan.....	34

3.5	Proses Pengelompokkan Makanan	35
3.6	Evaluasi Model	37
3.7	Metodologi Pengembangan Sistem	37
3.7.1	Analisis Kebutuhan	37
3.7.2	Desain Sistem.....	39
3.7.3	Implementasi	40
3.7.4	Pengujian.....	41
BAB IV HASIL PENGUJIAN DAN PEMBAHASAN		42
4.1	Hasil.....	42
4.1.1	Pengumpulan Data	42
4.1.2	Data Preprocessing.....	43
4.1.3	Implementasi dan Evaluasi Model	45
4.1.4	Proses Lokalisasi Makanan & Metode Ray Casting	55
4.1.5	Pengembangan Sistem.....	58
4.1.6	Pengujian Sistem.....	64
4.2	Pembahasan	86
BAB V PENUTUP		88
5.1	Kesimpulan	88
5.2	Saran	88
DAFTAR PUSTAKA		89

DAFTAR TABEL

Table 2.1 Performa Konfigurasi YOLOv8-seg.....	13
Table 2.2 State of Chart	21
Table 3.1 Jumlah Objek Setiap Jenis Makanan	25
Table 3.2 Perhitungan Proses Konvolusi.....	30
Table 3.3 Confussion Matrix	37
Table 3.4 Analisis Perangkat Lunak	38
Table 3.5 Analisis Perangkat Keras	38
Table 3.6 Pengujian Black Box Testing.....	41
Table 4.1 Parameter Eksperimen Tuning	47
Table 4.2 Hasil Akurasi Pelatihan dan Validation	48
Table 4.3 Hasil Pengujian Pada Data Testing	49
Table 4.4 Hasil Pengujian Data Testing.....	55
Table 4.5 Pengujian Sistem	65
Table 4.6 Hasil Evaluasi Pengujian Overlapping ke-1	66
Table 4.7 Hasil Evaluasi Pengujian Overlapping ke-2	67
Table 4.8 Hasil Evaluasi Pengujian Overlapping ke-3	68
Table 4.9 Hasil Evaluasi Pengujian Overlapping ke-4	68
Table 4.10 Hasil Evaluasi Pengujian Overlapping ke-4 (lanjutan)	69
Table 4.11 Hasil Evaluasi Pengujian Overlapping ke-5	70
Table 4.12 Hasil Evaluasi Pengujian Overlapping ke-6.....	71
Table 4.13 Hasil Evaluasi Pengujian Overlapping ke-7	72
Table 4.14 Hasil Evaluasi Pengujian Overlapping ke-8.....	73
Table 4.15 Hasil Evaluasi Pengujian Overlapping ke-9.....	74
Table 4.16 Hasil Evaluasi Pengujian Overlapping ke-10.....	75
Table 4.17 Hasil Evaluasi Pengujian No Overlapping ke-1	75
Table 4.18 Hasil Evaluasi Pengujian No Overlapping ke-1 (Lanjutan)	76
Table 4.19 Hasil Evaluasi Pengujian No Overlapping ke-2	77
Table 4.20 Hasil Evaluasi Pengujian No Overlapping ke-3	78
Table 4.21 Hasil Evaluasi Pengujian No Overlapping ke-4	79
Table 4.22 Hasil Evaluasi Pengujian No Overlapping ke-5	80
Table 4.23 Hasil Evaluasi Pengujian No Overlapping ke-6	81
Table 4.24 Hasil Evaluasi Pengujian No Overlapping ke-7	81
Table 4.25 Hasil Evaluasi Pengujian No Overlapping ke-7 (Lanjutan)	82
Table 4.26 Hasil Evaluasi Pengujian No Overlapping ke-8	82
Table 4.27 Hasil Evaluasi Pengujian No Overlapping ke-8 (Lanjutan)	83
Table 4.28 Hasil Evaluasi Pengujian No Overlapping ke-9	84
Table 4.29 Hasil Evaluasi Pengujian No Overlapping ke-10	84
Table 4.30 Hasil Evaluasi Pengujian No Overlapping ke-10 (Lanjutan)	85
Table 4.31 Evaluasi Pengujian Overlapping dan No Overlapping	85

DAFTAR GAMBAR

Gambar 2.1 Tugas Computer Vision	8
Gambar 2.2 Arsitektur CNN.....	9
Gambar 2.3 Convolutional Layer	10
Gambar 2.4 Pooling Layer	10
Gambar 2.5 Fully Connected Layer	11
Gambar 2.6 Deep Learning	11
Gambar 2.7 Performa YOLOv5, YOLOv6, YOLOv7, dan YOLOv8.....	12
Gambar 2.8 Cara Kerja YOLO	13
Gambar 2.9 YOLOv8-seg arsitektur.....	14
Gambar 2.10 Proses Metode Waterfall	18
Gambar 2.11 Ray Casting.....	19
Gambar 3.1 Alur Metodologi Penelitian	23
Gambar 3.2 Data Sekunder.....	24
Gambar 3.3 Data Validation	25
Gambar 3.4 Alur Metodologi Penelitian	26
Gambar 3.5 Contoh hasil labelling.....	26
Gambar 3.6 Splitting Dataset.....	27
Gambar 3.7 Contoh Resize dan Augmentation	27
Gambar 3.8 Alur Implementasi Model YOLOv8	28
Gambar 3.9 Arsitektur Model YOLOv8n	28
Gambar 3.10 Pembagian Grid Citra	29
Gambar 3.11 Ilustrasi Proses Lapisan Konvolusi.....	30
Gambar 3.12 Ilustrasi Proses MaxPool2D	33
Gambar 3.13 Ilustrasi Proses Upsample.....	33
Gambar 3.14 Proses Lokalisasi Makanan.....	34
Gambar 3.15 Proses Pengelompokan Makanan	35
Gambar 3.16 Hasil Segmentasi	35
Gambar 3.17 Arsitektur Sistem	39
Gambar 3.18 Rancangan Antarmuka Pengguna.....	40
Gambar 4.1 Pengumpulan data model pre trained	42
Gambar 4.2 Pengumpulan data model fine tuning	42
Gambar 4.3 Cleaning Data	43
Gambar 4.4 Dataset Analitics.....	44
Gambar 4.5 Pembagian Dataset.....	44
Gambar 4.6 Augmentasi Data.....	44
Gambar 4.7 Grafik Training Box dan Seg Loss	50
Gambar 4.8 Grafik Training Class dan Dfl Loss	50
Gambar 4.9 Grafik Validation Box dan Seg Loss	51
Gambar 4.10 Grafik Validation Class dan Dfl Loss	51
Gambar 4.11 Grafik Precision	52
Gambar 4.12 Grafik Recall.....	52

Gambar 4.13 Grafik mAP50.....	53
Gambar 4.14 Grafik mAP50-95	53
Gambar 4.15 Confussion Matrix Data Testing.....	54
Gambar 4.16 Antarmuka Awal Pengguna.....	63
Gambar 4.17 Antarmuka Pengguna Input Gambar	63
Gambar 4.18 Antarmuka Pengguna Input Kamera.....	64
Gambar 4.19 Pengujian Overlapping ke-1	65
Gambar 4.20 Pengujian Overlapping ke-2	66
Gambar 4.21 Pengujian Overlapping ke-3	67
Gambar 4.22 Pengujian Overlapping ke-4	68
Gambar 4.23 Pengujian Overlapping ke-5	69
Gambar 4.24 Pengujian Overlapping ke-6.....	70
Gambar 4.25 Pengujian Overlapping ke-7	71
Gambar 4.26 Pengujian Overlapping ke-8.....	73
Gambar 4.27 Pengujian Overlapping ke-9	73
Gambar 4.28 Pengujian Overlapping ke-10	74
Gambar 4.29 Pengujian No Overlapping ke-1	75
Gambar 4.30 Pengujian No Overlapping ke-2	76
Gambar 4.31 Pengujian No Overlapping ke-3	77
Gambar 4.32 Pengujian No Overlapping ke-4	78
Gambar 4.33 Pengujian No Overlapping ke-5	79
Gambar 4.34 Pengujian No Overlapping ke-6	80
Gambar 4.35 Pengujian No Overlapping ke-7	81
Gambar 4.36 Pengujian No Overlapping ke-8	82
Gambar 4.37 Pengujian No Overlapping ke-9	83
Gambar 4.38 Pengujian No Overlapping ke-10	84

DAFTAR PSEUDOCODE

Pseudocode 1 : Import Library	45
Pseudocode 2 : Import Dataset1	45
Pseudocode 3 : Import Dataset1 (Lanjutan).....	46
Pseudocode 4 : Inisialisasi Dataset.....	46
Pseudocode 5 : Training Model.....	46
Pseudocode 6 : Training Model (Lanjutan).....	47
Pseudocode 7 : Proses Lokalisasi Makanan dan Metode Ray Casting.....	55
Pseudocode 8 : Proses Lokalisasi Makanan dan Metode Ray Casting (Lanjutan)...	56
Pseudocode 9 : Proses Lokalisasi Makanan dan Metode Ray Casting (Lanjutan)...	57
Pseudocode 10 : Struktur Component.....	58
Pseudocode 11 : SumaryPrice Component.....	59
Pseudocode 12 : UploadImage Component.....	60
Pseudocode 13 : UploadImage Component (Lanjutan)	60
Pseudocode 14 : DetectionResult Component.....	61
Pseudocode 15 : DetectionResult Component (lanjutan).....	62

BAB I PENDAHULUAN

1.1 Latar Belakang Masalah

Dalam beberapa dekade terakhir kemajuan dalam teknologi pengolahan citra dan kecerdasan buatan telah membuka peluang bagi solusi inovatif untuk memantau sistem. Teknik seperti pemrosesan citra digital, *machine learning*, dan *deep learning* dapat mendeteksi objek dengan efisiensi dan efektivitas yang lebih baik. Salah satu masalah yang kerap ditemui dalam *machine learning* khususnya deteksi objek yakni sulitnya melakukan deteksi terhadap kompleksitas gambar yang cukup kompleks (Purnomo et al., 2025). Dalam penelitian tersebut, kompleksitas gambar mengacu pada jumlah wadah, jumlah objek serta kondisi objek yang saling tumpang tindih (*overlapping*) pada gambar makanan yang diperoleh dari lingkungan rumah makan. Pada penelitian ini peneliti menerapkan algoritma YOLOv8-seg yang telah terbukti ampuh dalam menangani objek yang *overlapping* (Sapkota et al., 2024) serta mengujinya dengan berbagai parameter pelatihan model untuk menangani masalah tersebut.

Untuk mendeteksi makanan telah dilakukan beberapa penelitian, salah satunya oleh Fahturaman et al., (2023) penelitian ini menggunakan dataset jajanan tradisional Indonesia yang berjumlah 1400 dengan 5 kelas. Penelitian ini membandingkan metode mobilenetV2, ResNet50 dan VGG16. Didapatkan hasil terbaik yakni mobilenetV2 dengan akurasi 98%, diikuti metode ResNet50 dan VGG16 dengan akurasi masing masing 97.33% dan 93.33%. Penelitian ini hanya berfokus pada *image classification* yang hanya mengklasifikasikan isi gambar secara keseluruhan tanpa mendeteksi lokasi atau keberadaan objek di dalamnya.

Penelitian lainnya yakni dengan perbandingan metode YOLOv5, YOLOv7, dan YOLOv8 yang dilakukan oleh Agarwal et al., (2023) dengan dataset Indian Food Items menunjukkan mAP model terbaik pada YOLOv8 mencapai 81%. Data yang digunakan berjumlah 5500 gambar yang terdiri dari 30 kelas. Dalam penelitian menyarankan untuk penambahan dataset agar lebih akurat dan beragam. Selain itu peneliti juga menyarankan untuk menjelajahi lebih dalam petunjuk visual (labelling) yang bisa digunakan agar deteksi makanan makin akurat.

Penelitian yang dilakukan oleh Purnomo et al., (2025) dengan judul “Implementasi You Only Look Once v8 Dalam Deteksi Makanan Warung Tegal Untuk Sistem Perhitungan Harga Otomatis”. Dataset yang digunakan yakni terdiri dari 40 kelas makanan warung tegal dengan total gambar mencapai 400 gambar. Model ini mampu mendeteksi secara real-time dengan *precision* 0.602, *recall* 0.176, F1-score 0.32, dan mAP@0.5 0.229. Kekompleksitasan latar belakang dan sedikitnya dataset yang digunakan menyebabkan akurasi tergolong rendah.

Pengelompokan (*clustering*) data merupakan salah satu teknik fundamental dalam analisis data spasial. Penelitian yang dijalankan oleh beberapa (Jain et al., 2025; Nurulhikmah & Abdi, 2024; Putri et al., 2021) menunjukkan algoritma seperti DBSCAN,

K-Means, dan Agglomerative Clustering dapat digunakan secara efektif untuk mengelompokkan data nutrisi makanan, status desa, maupun data spasial geolokasi. Namun, dalam konteks penelitian pengelompokan makanan berdasarkan wadah, proses pengelompokan tidak semata-mata ditentukan oleh kedekatan spasial antar objek makanan, melainkan juga oleh hubungan spasial antara objek makanan dengan batas geometris wadah. Oleh karena itu, diperlukan pendekatan tambahan seperti *ray casting/event odd algorithm* (Galetzka & Glauner, 2017) untuk mengevaluasi suatu objek makanan berada di dalam batas area suatu wadah, sehingga hasil pengelompokannya menjadi lebih akurat dan sesuai konteks spasial yang diharapkan.

Instance segmentation memiliki keunggulan dibandingkan *object detection*. *Object detection* menggunakan bounding box untuk menandai objek yang membuatnya tidak secara akurat memberikan batasan pada tiap object. Sebaliknya, *instance segmentation* memberikan masker piksel untuk setiap objek individu, memungkinkan identifikasi dan pemisahan objek yang lebih terperinci (Feng et al., 2019). Hal ini membantu dalam memisahkan dan menghitung jumlah objek secara detail setiap pixel. Image segmentation telah diuji dengan YOLOv8 dibandingkan dengan Mask R-CNN dibuktikan bahwa YOLOv8 lebih baik dalam hal kecepatan maupun akurasi dibandingkan Mask R-CN dalam lingkungan kebun buah yang kompleks (Sapkota et al., 2024).

Dari permasalahan yang didapat dari penelitian-penelitian sebelumnya, permasalahan utama dalam sistem *self-service* prasmanan otomatis terletak pada keterbatasan dalam menangani objek yang saling tumpang tindih (*overlapping*) serta makanan yang belum dapat dikelompokkan dalam satu wadah. Oleh karena itu peneliti mengusulkan metode deteksi YOLOv8-seg yang memiliki kemampuan segmentasi sehingga lebih efektif dalam menangani permasalahan *overlapping* objek, serta dievaluasi menggunakan berbagai skenario parameter. Model YOLOv8-seg tersebut dipadukan dengan metode ray casting untuk melakukan pengelompokan objek makanan yang berada dalam satu wadah. Pendekatan ini memungkinkan sistem mengidentifikasi makanan dalam konteks wadah masing-masing, sehingga mendukung perhitungan harga otomatis secara lebih baik.

1.2 Rumusan Masalah

Berdasarkan uraian pada latar belakang, maka rumusan masalah yang akan dibahas yakni sebagai berikut :

1. Bagaimana kinerja algoritma YOLOv8-seg dan ray casting dalam mendeteksi serta mengelompokkan objek makanan pada sistem self-service prasmanan yang memiliki kompleksitas gambar yang kompleks?
2. Bagaimana implementasi algoritma YOLOv8-seg dan ray casting secara *real-time* pada lingkungan rumah makan prasmanan

1.3 Batasan Masalah

Penelitian ini memiliki batasan masalah diantaranya sebagai berikut:

1. Makanan yang dideteksi khusus untuk kelas-kelas makanan yang tersedia di lokasi studi kasus, yaitu: nasi putih, lele goreng, ikan mujair, ayam goreng, tahu, tempe, telur bacem, bakwan, mendoan, sayur lodeh dan oseng kangkung.

2. Algoritma yang digunakan untuk deteksi dan segmentasi objek adalah YOLOv8n-seg.
3. Dataset yang diterapkan dalam penelitian ini mencakup data sekunder yang dari internet, serta data primer yang dikumpulkan secara langsung melalui pengambilan gambar menggunakan objek makanan dan wadah yang ditentukan.
4. Format citra yang digunakan terbatas pada ekstensi .jpg dan .jpeg.
5. Penggunaan kamera statis yang diposisikan secara vertikal dengan jarak tetap sejauh 50 cm dari permukaan wadah.
6. Sistem hanya mendeteksi jenis dan posisi makanan, tanpa memperhitungkan kuantitas makanan berdasarkan ukuran, volume, atau berat.

1.4 Tujuan Penelitian

Berdasarkan rumusan masalah diatas, tujuan dari penelitian ini adalah sebagai berikut :

1. Mengevaluasi kinerja algoritma YOLOv8-seg dan algoritma ray casting dalam mendeteksi dan mengelompokkan objek makanan pada sistem self-service prasmanan yang memiliki kompleksitas gambar yang kompleks.
2. Mengimplementasikan algoritma YOLOv8-seg dan ray casting secara real-time pada lingkungan rumah makan prasmanan.

1.5 Manfaat Penelitian

Dalam penelitian ini, implementasi algoritma YOLOv8-seg untuk deteksi makanan dan ray casting untuk pengelompokan berdasarkan wadah diharapkan dapat memberikan kontribusi nyata terhadap efisiensi pelayanan di lingkungan rumah makan prasmanan. Manfaat yang diharapkan dari penelitian ini antara lain sebagai berikut:

1. Adanya penelitian yang menerapkan metode ray casting diharapkan dapat meningkatkan pengalaman pelanggan melalui pengurangan waktu tunggu saat proses transaksi. Hal ini dikarenakan sistem mampu mendeteksi beberapa wadah makanan dalam satu gambar sekaligus serta mengelompokkan objek makanan secara otomatis, sehingga pelayanan menjadi lebih cepat, efisien, dan nyaman.
2. Sistem yang memanfaatkan YOLOv8-seg beserta pengujian variasi parameter mampu mengoptimalkan efisiensi operasional dengan mengurangi beban kerja kasir dalam proses identifikasi makanan dan perhitungan harga secara manual, sehingga transaksi menjadi lebih akurat, cepat, dan transparan, serta tenaga kerja dapat difokuskan pada peningkatan kualitas pelayanan kepada pelanggan.
3. Penelitian ini dapat dijadikan sebagai referensi dalam pengembangan sistem otomatisasi serupa di bidang kuliner lainnya, khususnya dalam penerapan computer vision dan kecerdasan buatan untuk menunjang efisiensi bisnis ataupun pengembangan metode computer vision lainnya.

1.6 Tahapan Penelitian

Tahapan penelitian merupakan serangkaian langkah yang disusun secara sistematis untuk meraih tujuan penelitian. Dalam penelitian ini, tahapan yang dilakukan mencakup

proses identifikasi masalah, pengumpulan data, pelatihan model deteksi, hingga implementasi sistem dan analisis hasil. Tahapan ini terbagi menjadi dua bagian, yaitu:

1.6.1 Metode Penelitian

Menjelaskan tahapan teknis yang dilakukan untuk melatih dan menerapkan model YOLOv8-seg serta algoritma *ray casting* dalam konteks deteksi, pengelompokan, dan perhitungan harga. Berikut metode penelitian yang dilakukan dalam penelitian ini:

1. Studi Literatur

Studi literatur adalah fase permulaan dalam riset yang bermaksud untuk memahami berbagai konsep, teori, serta hasil penelitian sebelumnya yang sesuai dengan topik yang diangkat. Pada fase ini, peneliti menghimpun rujukan dari bermacam sumber seperti buku, jurnal ilmiah, maupun penelitian sebelumnya yang relevan dengan penelitian ini, seperti deteksi makanan, dan model YOLOv8-seg.

2. Pengumpulan Data

Pengumpulan data adalah tahapan dalam riset yang bermaksud untuk memperoleh data yang dibutuhkan guna mendukung proses pelatihan dan pengujian model. Dalam riset ini, data yang dikumpulkan berupa gambar makanan prasmanan yang terdiri dari berbagai jenis makanan yang terdapat pada studi kasus dengan kondisi penyajian yang bervariasi. Riset ini menggunakan data sekunder yang berasal dari internet serta data primer berupa hasil dokumentasi sendiri.

3. Pengolahan Data Awal (*Preprocessing*)

Pengolahan data adalah tahapan dalam penelitian yang bertujuan untuk menyediakan data awal agar dapat dipakai dalam fase *training model*. Melakukan preprocessing data seperti *resizing*, anotasi segmentasi, klasifikasi label makanan, serta augmentasi gambar untuk meningkatkan variasi data. Dataset yang telah terkumpul terbagi menjadi tiga subset: *training* 80%, data *validation* 10%, dan *testing* 10%.

4. Pelatihan Model YOLOv8-seg

Pelatihan model YOLOv8-seg merupakan fase membangun model deteksi objek yang mampu mengenali dan mengklasifikasikan makanan dalam gambar berdasarkan data yang telah dianotasi sebelumnya. Proses ini akan mengoptimalkan parameter yang ada agar meminimalkan fungsi kerugian (*loss function*) dan meningkatkan akurasi deteksi makanan.

5. Penerapan algoritma *Ray Casting*

Penerapan algoritma *ray casting* dalam penelitian ini bertujuan untuk mengelompokkan objek makanan yang telah terdeteksi oleh YOLOv8-seg ke dalam suatu wadah. Setiap wadah direpresentasikan sebagai poligon, dan sistem akan memeriksa letak titik pusat (*centroid*) dari bounding box makanan dengan poligon wadah tersebut dengan menghitung jumlah perpotongan garis horizontal terhadap sisi poligon.

6. Evaluasi Model

Evaluasi model adalah fase guna mengukur performa model YOLOv8-seg setelah melalui proses pelatihan. Mengevaluasi kemampuan model menggunakan metrik seperti mAP, precision, dan recall untuk memastikan akurasi deteksi. Evaluasi ini menggunakan data validasi dengan demikian evaluasi yang digunakan akan lebih objektif karena menggunakan data yang berbeda.

7. Pengujian

Pengujian adalah tahap untuk memastikan bahwa model deteksi makanan dan sistem yang dibangun dapat berlangsung selaras dengan tujuan dan karakteristik yang telah ditetapkan. Uji coba dilakukan dengan data testing yang didapat dari data primer serta secara real-time pada sistem self-service untuk melihat kemampuan model dalam mendeteksi, mengelompokkan dan menghitung harga makanan.

8. Analisis Hasil

Analisis hasil adalah tahap di mana peneliti mengevaluasi dan menginterpretasikan hasil pengujian serta evaluasi model yang telah dilakukan sebelumnya. Hal tersebut diaplikasikan guna menarik kesimpulan dan menyimpulkan kontribusi penelitian ini

1.6.2 Metode Pengembangan Sistem

Menjelaskan proses perancangan dan implementasi sistem berbasis web yang menggunakan hasil model YOLOv8-seg untuk mendukung sistem self-service prasmanan. Berikut metode pengembangan sistem yang dilakukan dalam penelitian ini:

1. Analisis Kebutuhan

Pada fase ini, dilaksanakan penentuan akan keperluan sistem, baik kebutuhan fungsional, seperti akurasi model, perhitungan harga, dan penyajian informasi harga maupun non-fungsional, seperti performa sistem, antarmuka, dan penyimpanan data untuk pencatatan transaksi.

2. Perencanaan Sistem

Mendesain arsitektur sistem yang mencakup kamera yang digunakan, alur proses deteksi makanan, perhitungan harga, dan tampilan hasil ke pengguna, backend, serta frontend.

3. Implementasi Sistem

Menciptakan sistem berlandaskan web yang memadukan kamera, model YOLOv8-seg, backend (Flask), dan antarmuka pengguna (React).

4. Pengujian Sistem

Melakukan uji coba sistem secara menyeluruh untuk memastikan bahwa fitur deteksi dan perhitungan harga mampu berlangsung dengan bagus dan tepat dengan harapan penggunaan.

1.7 Sistematika Penulisan

Sistematika penulisan menyampaikan garis besar terkait riset yang akan dilaksanakan untuk memudahkan pembaca dan peneliti dalam memahami. Dalam penyusunan penelitian ini, sistematika yang digunakan adalah sebagai berikut:

BAB I PENDAHULUAN

Bab I memuat latar belakang, rumusan masalah, batasan masalah, tujuan penelitian, manfaat penelitian, serta sistematika penulisan dalam Implementasi YOLOv8-seg Dan Ray Casting Untuk Deteksi Serta Pengelompokan Makanan Berdasarkan Hubungan Spasial Dengan Wadah.

BAB II TINJAUAN LITERATUR

Bab II terdiri dari teori-teori serta dasar ilmu yang mendukung, khususnya terkait algoritma yang digunakan untuk riset ini. Pada bab ini akan diuraikan terkait image segmentation, CNN, deep learning YOLOv8. Pada bab ini juga akan dibahas mengenai penelitian terkait yang sudah dilakukan untuk referensi penelitian yang akan dibuat.

BAB III METODE PENELITIAN

Bab III membahas mengenai metode yang digunakan dalam penelitian secara sistematis, meliputi rancangan penelitian, tahapan pengumpulan data, proses pengolahan dan pelabelan data, perancangan sistem, hingga metode pengujian dan evaluasi kinerja sistem. Seluruh tahapan disajikan secara rinci dan terorganisir guna memberikan gambaran yang jelas mengenai alur penelitian serta memastikan penelitian dapat dilakukan secara terarah dan terukur.

BAB IV HASIL DAN PEMBAHASAN

Bab IV akan menjelaskan terkait hasil penelitian yang telah dilakukan dengan menerapkan YOLOv8-seg. Pada bab ini juga akan dijelaskan mengenai pengujian dan akurasi dari penelitian ini.

BAB V PENUTUP

Bab V memuat kesimpulan dari penelitian yang telah dibuat serta saran dari penulis untuk pengembangan penelitian selanjutnya. Pada bab ini akan dijelaskan terkait hal-hal yang belum dapat diselesaikan dalam penelitian ini agar dapat dikembangkan pada penelitian dimasa mendatang.

BAB II TINJAUAN LITERATUR

2.1 Self-Service Technology

Self-Service Technology (SST) merupakan struktur inovasi servis yang memberi kesempatan kepada pelanggan untuk berhubungan dengan teknologi dalam memperoleh servis dengan tidak disertai bantuan langsung dari staf. Menurut Meuter et al., (2000), SST diartikan sebagai teknologi berbasis layanan yang memungkinkan pelanggan melakukan sebagian atau seluruh layanan secara mandiri. Dengan perkembangan digital yang pesat, SST menjadi sangat penting dalam berbagai industri karena mampu meningkatkan efisiensi operasional, mempercepat proses pelayanan, serta mengurangi biaya tenaga kerja (Chan & Petrikat, 2022).

Dalam bidang layanan makanan, SST telah banyak diterapkan, seperti pada restoran cepat saji McDonald's (Stanley et al., 2023) dan sistem pemesanan menu secara mandiri (Wijaya, 2020). Penerapan SST ini menunjukkan hubungan positif terhadap *customer experience* dan *customer acceptance* (Jamil et al., 2019; Stanley et al., 2023). Di sisi lain, pelanggan semakin mengharapkan layanan yang efisien dan efektif sebagai bagian dari kepuasan mereka (Ayu et al., 2023). Oleh karena itu, penggunaan SST terus menjadi tren yang berkembang dalam industri makanan karena dinilai mampu menjawab kebutuhan tersebut.

Implementasi SST telah diterapkan dalam industri rumah makan prasmanan, seperti pada proses pengambilan makanan secara mandiri dan pembayaran menerapkan *Quick Response Code Indonesian Standard* (QRIS). Namun, implementasi SST dalam proses perhitungan harga masih menghadapi tantangan. Saat ini, perhitungan harga masih dilakukan secara konvensional, yaitu dengan melibatkan staf untuk memantau dan mencatat secara manual jenis makanan yang diambil oleh pengunjung, sehingga belum sepenuhnya efisien.

2.2 Computer Vision

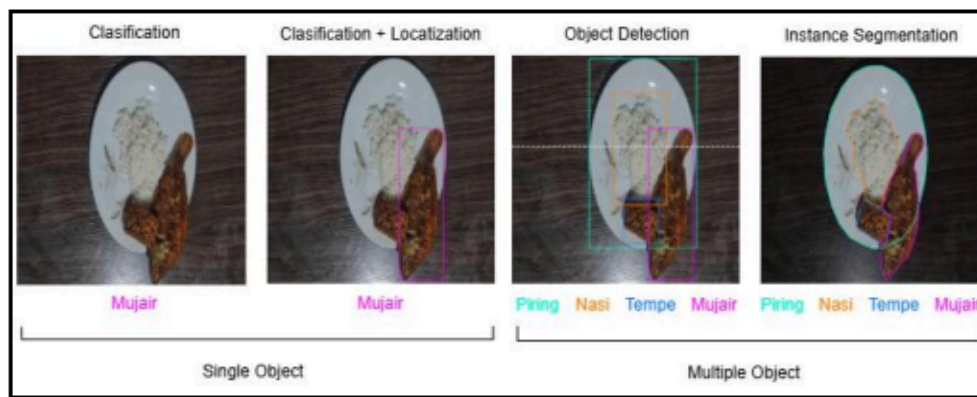
Computer vision merupakan disiplin dalam *artificial intelegent* yang berpusat dalam pembangunan sistem komputer agar mampu mengenali, memahami, dan menganalisis data visual seperti gambar serta video dengan cara pendekatan yang meniru cara kerja penglihatan persepsi manusia. Dengan adanya algoritma dalam *image processing* memungkinkan komputer memahami lingkungan visual (Dani. & Anwar, 2024).

Beberapa penerapan *computer vision* saat ini seperti deteksi wajah, pelacakan objek, hingga membantu dalam sistem kendaraan otomatis. *Computer vision* memiliki beberapa jenis algoritma dengan tujuan dan penerapannya yang berbeda:

1. *Image classification* : Proses mengelompokkan gambar secara otomatis ke dalam jenis tertentu yang telah ditetapkan pada awalnya (Feng et al., 2019).
2. *Object Detection* Metode computer vision guna mendeteksi dan menetapkan posisi objek dalam sebuah data visual (Feng et al., 2019)
3. *Image Segmentation* : Proses membagi citra digital pada tahap piksel yang bertujuan membagi gambar menjadi area bermakna dengan mengklasifikasikan setiap piksel ke dalam objek tertentu untuk analisis lebih lanjut (Feng et al., 2019).

2.3 Instance Segmentation

Instance segmentation merupakan salah satu tugas penting dan menantang dalam *computer vision* dalam menentukan lokasi dan bentuk objek secara detail dalam sebuah gambar atau video. Tujuan utamanya yakni mengklasifikasikan setiap objek dalam gambar dan memisahkan setiap objek sebagai instansi unik berdasarkan tiap pixel nya (Liu et al., 2018). Berbeda dengan *object detection* yang hanya memberikan *bounding box* untuk mendeteksi lokasi objek tanpa memperhatikan bentuk pikselnya, atau *semantic segmentation* yang memberi label piksel namun tidak membedakan antar objek dengan kelas yang sama (Feng et al., 2019). Oleh sebab itu *instance segmentation* bisa dirumuskan bahwa pendekatan *computer vision* yang memecahkan masalah *object detection* dan *semantic segmentation* (Alzubaidi et al., 2021). Contoh gambar tugas *computer vision* dapat dilihat pada gambar 2.1 berikut.



Gambar 2.1 Tugas Computer Vision

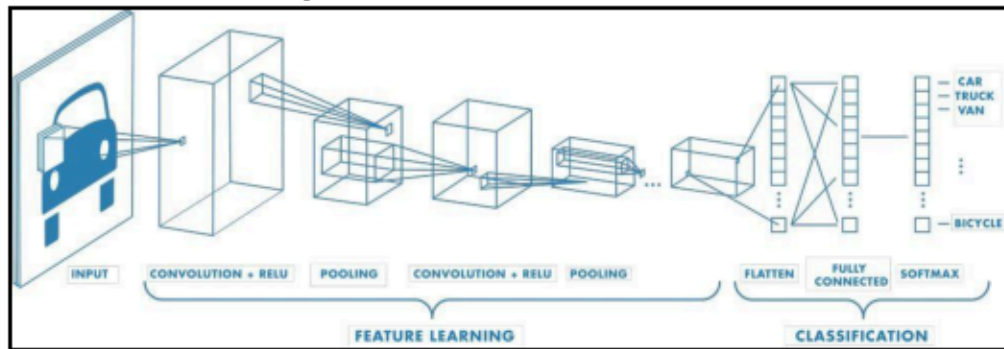
Sebelum berkembangnya *instance segmentation*, *computer vision* terlebih dahulu difokuskan pada pendekatan *image classification*. Pada mulanya *image classification* hanya digunakan untuk mengenali karakter tulisan tangan menggunakan versi awal CNN yaitu LeNet (Lecun et al., 1998). Pada tahun 2012 terjadi terobosan besar ketika Krizhevsky et al., (2012) memperkenalkan AlexNet, ini menginspirasi peneliti lain di berbagai bidang *computer vision* seperti *image classification*, *object detection*, serta *semantic image*. Pendekatan *object detection* muncul seiring meningkatnya kebutuhan sistem, model seperti R-CNN (Hariharan et al., 2014), Fast R-CNN (Girshick, 2015), dan Faster R-CNN (Ren et al., 2016) secara bertahap meningkatkan kecepatan dan akurasi. Selanjutnya *image segmentation* hadir sebagai solusi untuk mengklasifikasikan pixel ke dalam objek tertentu dengan model seperti FCN (Shelhamer et al., 2017), U-Net (Ronneberger et al., 2015), SDS (Hariharan et al., 2014), Mask R-CNN (He et al., 2017), dan YOLACT (Bolya et al., 2019).

Perkembangan teknologi *computer vision*, khususnya pada pendekatan *image segmentation* tipe *instance segmentation*, terus menunjukkan kemajuan yang signifikan. Salah satu model yang saat ini banyak dipakai untuk tugas tersebut adalah YOLOv8-seg, versi terbaru dari model YOLO yang terus mengalami pembaruan dan peningkatan performa seiring waktu. Model ini sering digunakan dalam aplikasi seperti robotika, kendaraan otonom, dan analisis citra medis. Hal ini dikarenakan model ini mampu menggabungkan fungsi deteksi objek dan segmentasi secara bersamaan (Feng et al., 2019).

2.4 Convolutional Neural Network (CNN)

CNN merupakan salah satu rancangan dari Neural Network yang banyak diaplikasikan di bidang *deep learning*, terutama untuk pengolahan data visual seperti gambar dan video. CNN telah menjadi fondasi utama dalam pengembangan berbagai model computer vision, seperti *image classification*, *object detection*, dan *image segmentation* (Khan et al., 2020). Karakteristik CNN adalah mampu mengidentifikasi fitur lokal pada setiap bagian (grid) citra serta secara otomatis melakukan proses ekstraksi fitur.

Secara umum, rancangan CNN terdiri atas dua bagian utama, yaitu *feature extraction layers* dan *classification layers*. Lapisan *feature extraction layers* biasanya mencakup *convolutional layer*, *activation function* (ReLU), dan *pooling layer*, yang bertujuan untuk mengekstraksi dan menyaring fitur esensial dari data input. Sedangkan bagian *classification layers* biasanya berupa satu atau lebih *fully connected layers* yang menghasilkan output berupa kelas prediksi (Alzubaidi et al., 2021; Szegedy et al., 2015). Gambaran umum gambar 2.2 dibawah ini merupakan arsitektur CNN.



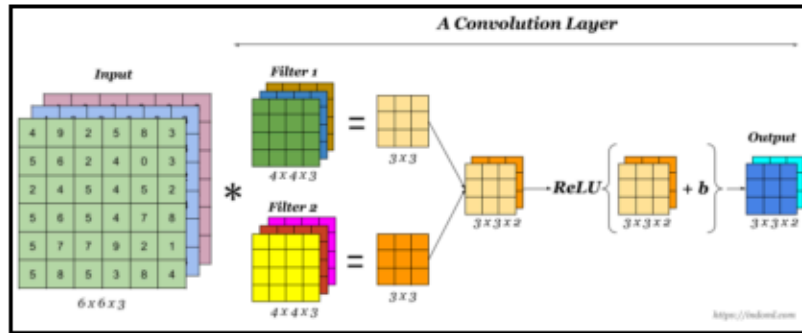
Gambar 2.2 Arsitektur CNN

Sumber: (Matlab MathWorks)

2.4.1 Convolutional layer

Convolutinal layer terdiri dari sekumpulan convolutional filter (kernel). Kernel merupakan nilai diskrit berupa matriks yang memiliki bobot pada setiap elemennya (Laith Alzubaidi et al., 2021). Fungsi utama dari convolutional kernel adalah untuk melakukan *convolutional operation* yakni mengekstraksi setiap *local pattern* dari gambar. Kernel ini akan bergerak melintasi gambar menggunakan convolutional operation dan menerapkan bobot yang sama pada setiap posisi (Khan et al., 2020). Proses ini dilakukan untuk semua kanal warna RGB (Red, Green, Blue) dan diterapkan secara horizontal serta vertikal hingga seluruh matriks citra diproses dan menghasilkan fitur dari proses ekstraksi tersebut. Setiap kanal warna diproses secara terpisah untuk menangkap informasi warna yang berbeda, kemudian hasilnya digabungkan sehingga menghasilkan representasi fitur yang lebih kaya.

Hasil dari convusional operation tersebut akan dilakukan fungsi *activation* ReLU (*Rectified Linear Unit*). Fungsi ini adalah salah satu fungsi aktivasi yang paling umum digunakan dalam CNN. Fungsi ini bekerja dengan cara mengubah nilai negatif menjadi nol dan mempertahankan nilai positif (Yamashita et al., 2018). Hasil dari fungsi ReLU ini akan menghasilkan matriks yang disebut matriks *feature extraction*, matriks ini mengidentifikasi pola tertentu dalam gambar seperti tepi, tekstur, atau sudut, yang kemudian digunakan dalam proses pengenalan atau klasifikasi citra. Penjelasan dari convolutional layer di atas dapat dilihat pada gambar 2.3 berikut.

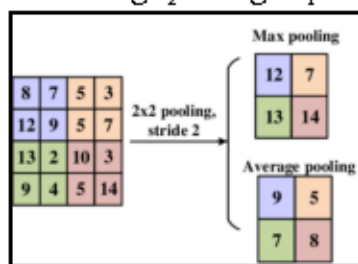


Gambar 2.3 Convolutional Layer

Sumber : (Savas., 2022)

2.4.2 Pooling Layer

Sesudah convolutional layer menghasilkan feature maps, pooling layer dipakai guna menurunkan dimensi spasial dari feature maps tersebut sambil mempertahankan informasi yang sangat essensial. Hal ini digunakan untuk mempercepat komputasi serta mencegah terjadinya overfitting. Secara umum, pooling layer dilakukan dengan cara mengambil nilai tertentu dari suatu area (patch lokal) pada feature map, kemudian menggeser area tersebut secara horizontal dan vertikal ke seluruh bagian feature map. Terdapat beberapa jenis pooling layer, dua teknik pooling yang sering diterapkan adalah max pooling dan average pooling. Max pooling bekerja dengan cara mengambil nilai tertinggi dari setiap patch lokal, sehingga hanya fitur yang paling dominan yang dipertahankan. Sementara itu, average pooling mengestimasi nilai rata-rata dari setiap patch lokal, sehingga mempertahankan informasi secara lebih menyeluruh namun kurang menonjolkan fitur utama. Penggunaan pooling layer tidak hanya berfungsi untuk menurunkan kompleksitas komputasi, tetapi juga memberikan invariansi terhadap translasi kecil pada citra (Alzubaidi et al., 2021). Contoh dari pooling layer *max pooling* dan *average pooling* dapat dilihat pada gambar 2.4 berikut.

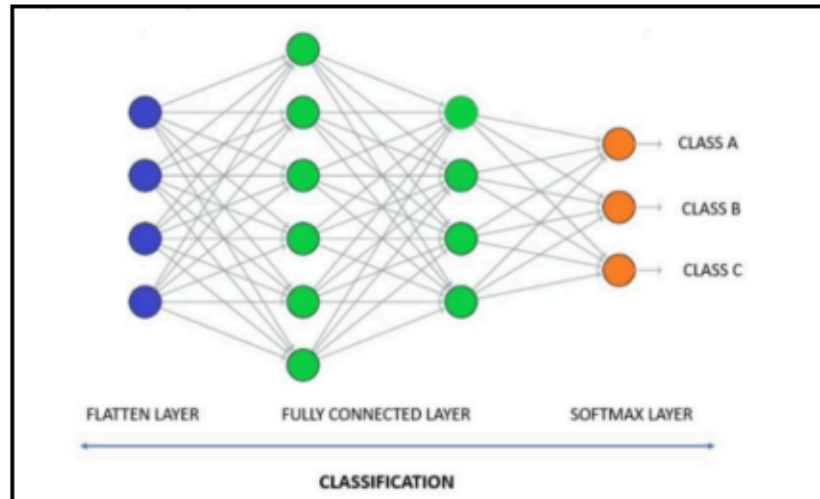


Gambar 2.4 Pooling Layer

Sumber : (Savas., 2022)

2.4.3 Fully Connected layer

Fully connected layer biasanya dipakai saat akhir jaringan khususnya untuk melakukan proses *classification* berdasarkan fitur yang telah diekstraksi pada lapisan sebelumnya. Layer ini mendapatkan input dari tahap *feature extraction* pada *feature learning* dan akan diubah kedalam bentuk vektor satu dimensi (flatten) sebelum dimasukkan kedalam fully connected layer. Neuron pada lapisan ini terkoneksi semua kedalam lapisan sebelumnya. Hasil dari fully connected layer akan digunakan untuk menganalisis output dari seluruh lapisan sebelumnya secara global. Output dari layer ini berupa hasil prediksi yang diperoleh dari proses klasifikasi. (Alzubaidi et al., 2021). Gambar 2.5 berikut merupakan bagian dari *fully connected layer*.



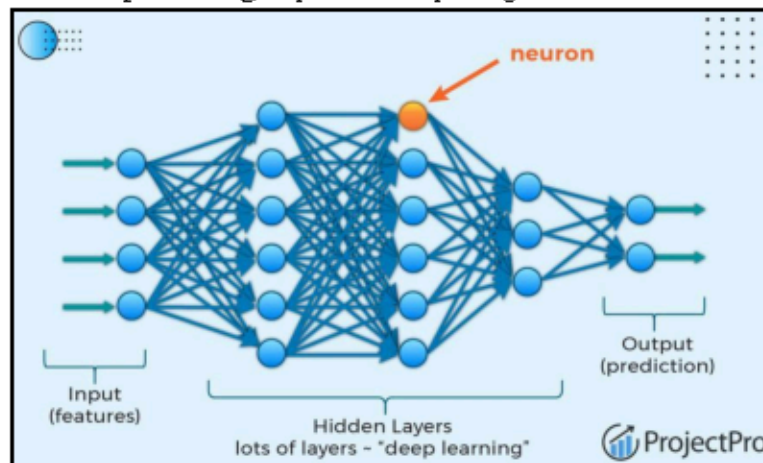
Gambar 2.5 Fully Connected Layer

Sumber : (Meruva et al, 2022)

2.5 Deep Learning

Deep learning adalah salah satu bidang dari machine learning yang memakai arsitektur jaringan saraf tiruan guna menelaah data melalui beberapa tingkat representasi atau abstraksi. Berbeda dengan machine learning konvensional yang umumnya membutuhkan proses *feature engineering* secara manual oleh ahli domain, deep learning memiliki kemampuan untuk mengekstraksi fitur secara otomatis langsung dari data mentah. Hal ini dimungkinkan melalui penggunaan banyak lapisan transformasi non-linear yang dilatih menggunakan algoritma backpropagation untuk meminimalkan kesalahan prediksi (Lecun et al., 2015).

Kemampuan ini memungkinkan deep learning untuk mempelajari data dalam jumlah masif dan tidak terorganisir, seperti citra, suara, dan teks. Beberapa aplikasi deep learning yang telah menunjukkan hasil signifikan meliputi pemahaman bahasa alami (natural language understanding), klasifikasi topik (topic classification), analisis sentimen (sentiment analysis), sistem tanya jawab (question answering), dan penerjemahan bahasa (Lecun et al., 2015). Gambaran umum *deep learning* dapat dilihat pada gambar 2.6 berikut.



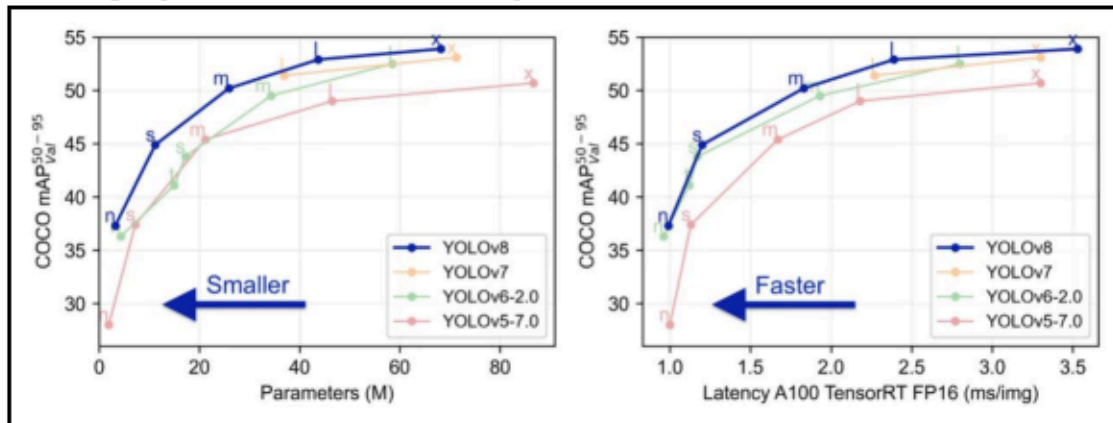
Gambar 2.6 Deep Learning

Sumber : (Lecun et al., 2015)

Secara umum, cara kerja deep learning dimulai dari pemberian input yang kemudian diproses secara berurut melalui setiap lapisan dalam jaringan saraf. Setiap lapisan terdiri dari sejumlah neuron yang mengekstraksi fitur tertentu dari data, mulai dari fitur sederhana seperti *edges*, *shapes*, hingga objek yang rumit. Proses pelatihan dilakukan dengan menggunakan backpropagation, yaitu suatu metode yang memperbarui bobot koneksi antar neuron berdasarkan tingkat kesalahan yang dihasilkan oleh prediksi, sehingga secara bertahap menghasilkan output yang semakin akurat (Lecun et al., 2015).

2.6 YOLOv8-seg

YOLOv8-seg merupakan varian dari algoritma YOLOv8 yang dikembangkan oleh Ultralytics dan dirilis pada tahun 2023 sebagai edisi terkini dalam algoritma YOLO yang populer dengan kemampuannya dalam melakukan deteksi objek secara real-time dan akurat. Varian ini menjadi yang pertama dalam seri YOLO yang mendukung tiga tugas utama dalam visi komputer, yaitu deteksi objek, klasifikasi, dan segmentasi dalam satu model. YOLOv8-seg ini mampu menangani gambar dengan melakukan *instance segmentation*, berbeda dengan pendahulunya YOLOv8 yang hanya mampu melakukan *detection object*. Dibandingkan dengan pendahulunya seperti YOLOv5, YOLOv6, dan YOLOv7, YOLOv8 dirancang dengan arsitektur yang lebih efisien dan akurat, serta lebih fleksibel untuk berbagai implementasi tugas computer vision (Hussain, 2023). Gambar 2.7 merupakan perbandingan performa YOLO dari beberapa versi.



Gambar 2.7 Performa YOLOv5, YOLOv6, YOLOv7, dan YOLOv8

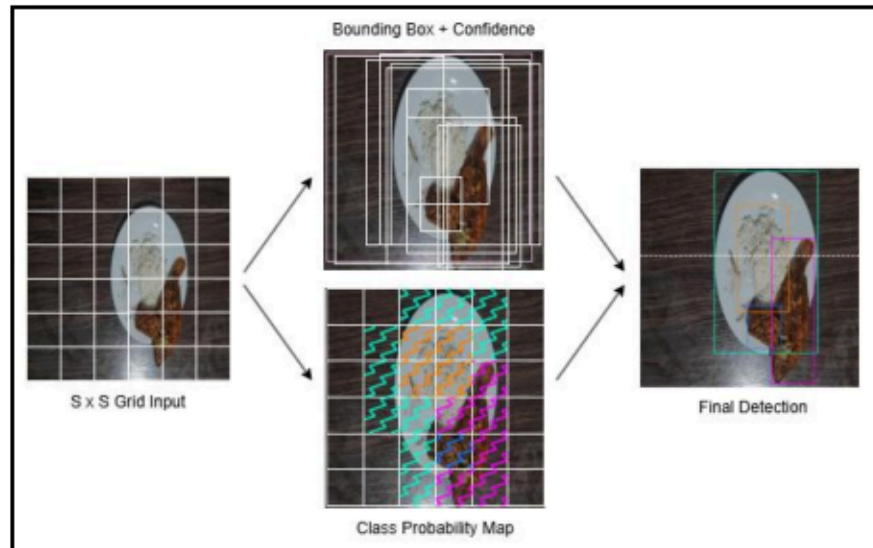
Sumber : (Ultralytics, 2023)

YOLOv8-seg memiliki beberapa konfigurasi model yang dikembangkan untuk menyesuaikan kebutuhan terhadap sumber daya komputasi dan tingkat akurasi. Setiap konfigurasi menunjukkan tingkat kompleksitas dan performa yang berbeda. Varian konfigurasi tersebut meliputi YOLOv8n-seg (nano), YOLOv8s-seg (small), YOLOv8m-seg (medium), YOLOv8l-seg (large), dan YOLOv8x-seg (extra large). Semakin kompleks arsitektur model yang digunakan, umumnya menghasilkan performa segmentasi yang lebih baik, terutama dalam hal akurasi dan presisi. Hasil pengujian pada dataset COCO menunjukkan bahwa terdapat perbedaan performa di antara konfigurasi tersebut, yang menunjukkan *trade-off* antara kecepatan inferensi dan akurasi segmentasi (Ultralytics, 2023). Performa konfigurasi *trade-off* antara varian konfigurasi dapat dilihat pada tabel 2.1 berikut.

Table 2.1 Performa Konfigurasi YOLOv8-seg
Sumber : (Ultralytics, 2023)

Model	Size(px)	mAP^{box}_{50-95}	mAP^{mask}_{50-95}	$Speed_{CPU\ ONNX}(ms)$
YOLOv8n-seg	640	36,7	30,5	96,1
YOLOv8s-seg	640	44,6	36,8	155,7
YOLOv8m-seg	640	49,9	40,8	317,0
YOLOv8l-seg	640	52,3	42,6	572,4
YOLOv8x-seg	640	53,4	43,4	712,1

Cara kerja algoritma YOLO diawali dengan memisahkan citra input ke dalam sejumlah grid cell berukuran tetap ($S \times S$). Setiap grid cell berkewajiban untuk memperkirakan objek yang berada di dalam wilayahnya. Proses ini bertujuan untuk mempermudah model dalam melakukan identifikasi lokasi objek secara lebih terstruktur. Prediksi yang dihasilkan oleh masing-masing *grid cell* terdiri dari beberapa bounding box serta *confidence score* klasifikasi untuk setiap kelas objek yang mungkin terdeteksi. Bounding box mempresentasikan lokasi objek yang terdeteksi dalam bentuk koordinat, sedangkan *confidence score* menunjukkan tingkat keyakinan model terhadap keberadaan objek di suatu area. Disisi lain, model juga akan menghasilkan *probability map* yang menunjukkan kemungkinan setiap grid mengandung kelas objek tertentu. Cara kerja YOLO tersebut ditunjukkan pada Gambar 2.8



Gambar 2.8 Cara Kerja YOLO

Setiap bounding box memiliki tiga komponen utama, yaitu koordinat pusat (x, y), ukuran kotak (width dan height), serta *confidence score*. *confidence score* ini menunjukkan seberapa yakin model bahwa suatu objek memang ada di dalam kotak tersebut, serta seberapa akurat kotak prediksi tersebut terhadap objek sebenarnya. Rumus yang digunakan untuk mengkalulasikan *confidence score* yakni: (Redmon et al., 2016):

$$confidence\ score = P(object) \times IOU_{pred,truth}$$

keterangan:

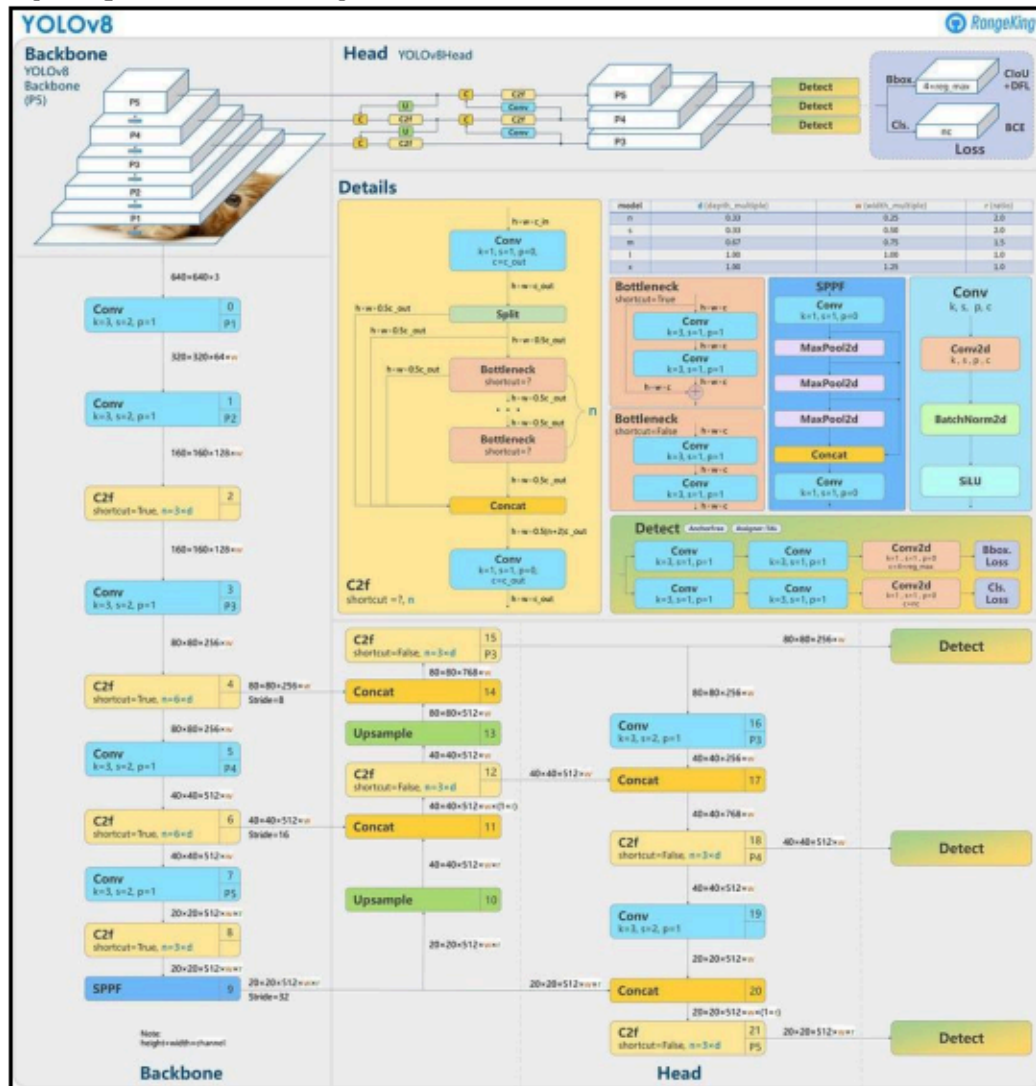
$P(object)$: Probabilitas keberadaan objek di dalam kotak prediksi

$IOU_{pred,truth}$: *Intersection over Union* merupakan rasio antara luas area tumpang tindih antara kotak prediksi dan kotak kebenaran terhadap luas gabungannya.

2.6.1 Arsitektur YOLOv8-seg

Arsitektur YOLOv8-seg menangani tugas instance segmentation dengan tetap mempertahankan efisiensi dan kecepatan deteksi dari arsitektur YOLOv8. Dibandingkan dengan YOLOv7, YOLOv8 memiliki arsitektur yang lebih efisien dan fleksibel. YOLOv7 menggunakan modul ELAN yang kompleks, sedangkan YOLOv8 mengadopsi C2f dan SPPF yang lebih ringan namun tetap efektif dalam ekstraksi fitur (Ultralytics, 2023). Selain itu, YOLOv7 masih menggunakan pendekatan anchor-based yang memerlukan pengaturan anchor box, sementara YOLOv8 beralih ke anchor-free detection yang lebih sederhana dan adaptif. Perubahan ini membuat YOLOv8 lebih cepat, akurat, dan mudah diimplementasikan untuk berbagai tugas visi komputer, termasuk segmentasi (Subedi, 2024).

Pada arsitektur ini terdiri dari tiga komponen utama, yaitu backbone, neck, dan head. Struktur YOLOv8-seg secara keseluruhan hampir serupa dengan YOLOv8 standar, namun dilengkapi dengan *segmentation head* tambahan yang berfungsi menghasilkan *segmentation mask* dan *mask coefficients* untuk setiap objek yang terdeteksi. Arsitektur YOLOv8-seg dapat dilihat pada gambar 2.9 (Ultralytics, 2024).



Gambar 2.9 YOLOv8-seg arsitektur

Sumber : (ultralytics, 2023)

Komponen backbone dalam YOLOv8-seg berperan sebagai ekstraktor fitur utama dari citra input. Proses ini dilakukan melalui serangkaian convolutional block, serta modul C2f (Cross-Stage Partial with 2 fused outputs) dan SPPF (Spatial Pyramid Pooling - Fast), yang membantu mempercepat proses ekstraksi fitur tanpa mengorbankan kedalaman informasi spasial. Citra input diproses melalui lima tahapan utama, yang disebut dengan P1 hingga P5, di mana setiap tahap menangkap informasi dari tingkat resolusi yang berbeda-beda. Konteks visual dalam model dari skala kecil hingga besar dapat dipahami melalui proses ini, sehingga dapat mendeteksi objek dengan ukuran dan posisi yang bervariasi secara lebih akurat (Subedi, 2024).

Selanjutnya, komponen neck digunakan untuk menggabungkan fitur dari berbagai skala hasil ekstraksi dari backbone. Hal ini dilakukan menggunakan mekanisme *Feature Pyramid Network* (FPN) atau *Path Aggregation Network* (PAN) yang bertujuan untuk mempertahankan informasi penting dari berbagai level resolusi (dari dalam ke luar dan sebaliknya), sehingga model dapat mengenali objek secara lebih efektif di berbagai skala serta robust dalam deteksi objek kecil maupun objek besar.

Pada bagian akhir, head bertanggung jawab untuk menghasilkan prediksi akhir dari model. Pada YOLOv8, bagian head hanya menghasilkan dua output utama yakni: bounding box dan class prediction. Namun dalam YOLOv8-seg, head memiliki tiga output utama, yaitu: bounding box prediction, class prediction, serta segmentation output (Subedi, 2024). Penambahan cabang segmentasi ini memungkinkan model untuk tidak hanya mendeteksi lokasi objek, tetapi juga menentukan bentuk objek secara lebih detail hingga tingkat piksel.

Penelitian terdahulu menunjukkan YOLOv8 memiliki performa yang baik dalam melakukan tugas *object detection*, tetapi YOLOv8-seg memberikan hasil terbaik dalam tugas segmentasi objek. Representasi yang dihasilkan model ini lebih jelas dibandingkan dengan model berbasis bounding box. Selain itu YOLOv8-seg juga mampu mengidentifikasi gambar yang lebih kompleks yang lebih baik. Hal ini menjadikan YOLOv8-seg lebih unggul dalam menangani objek yang saling berdekatan atau bertumpang tindih dibandingkan YOLOv8 (Bingol & Al-Raweshidy, 2025).

2.6.2 Backbone

Backbone merupakan komponen kunci pada arsitektur deteksi objek karena bertindak sebagai *feature extractor* utama yang mengubah citra input menjadi peta-peta fitur (feature maps) berlapis dengan tingkat abstraksi berbeda. Fungsi utama backbone meliputi ekstraksi fitur lokal (tepi, tekstur). Komponen ini terdiri dari modul Conv, SPPF, and C2f. Modul Conv digunakan untuk melakukan ekstraksi fitur dari sebuah input citra gambar menggunakan operasi convolutional. Modul ini tersusun atas operasi Conv, Batch Normalization (BN), dan fungsi aktivasi SiLU. Modul SPPF berperan dalam mengurangi beban komputasi serta mempercepat proses dengan melakukan operasi Conv serta menghubungkan tiga lapisan max-pooling secara berturut-turut. Sementara itu, modul C2f menerapkan konsep *gradient diversion* dengan mengombinasikan modul Conv dan residual untuk memperoleh aliran gradien tambahan, sehingga meningkatkan kemampuan model dalam mengekstraksi dan mempelajari sebuah fitur (Subedi, 2024).

1. Convolution

Convolution merupakan proses fundamental dalam Convolutional Neural Network (CNN) yang digunakan untuk mengekstraksi fitur penting dari sebuah citra. Pada proses ini, sebuah *kernel* berukuran kecil yang berisi parameter yang dapat dipelajari selama pelatihan digeser secara sistematis ke seluruh area ukuran gambar input. Pada setiap posisi pergeseran, dilakukan operasi perkalian elemen demi elemen (*element-wise multiplication*) antara nilai pada kernel dan bagian citra yang sedang diproses, kemudian hasilnya dijumlahkan sehingga menghasilkan satu nilai pada *feature map*. Dengan mekanisme ini, convolution mampu mendeteksi pola-pola lokal seperti tepi horizontal dan vertikal, tekstur, bentuk, maupun fitur penting lainnya. Tujuan utama dari operasi convolution adalah mengekstraksi representasi fitur yang relevan dari citra, sehingga jaringan dapat mengenali pola visual dan memanfaatkannya dalam tugas lebih lanjut seperti klasifikasi, segmentasi, dan deteksi objek (Yamashita et al., 2018).

2. Batch Normalization

Batch Normalization merupakan teknik normalisasi yang digunakan untuk mengatasi *internal covariate shift*, yaitu perubahan distribusi aktivasi di setiap layer selama proses pelatihan akibat pembaruan parameter model. Perubahan distribusi ini dapat memperlambat proses pelatihan dan menyebabkan ketidakstabilan gradien. Oleh karena itu, tujuan utama Batch Normalization adalah menstabilkan distribusi aktivasi sehingga proses pembelajaran menjadi lebih cepat, lebih stabil, dan lebih mudah untuk dikonvergensi.

Pada prosesnya, Batch Normalization menghitung statistik mini-batch, berupa nilai rata-rata (*mean*) dan ragam (*variance*), kemudian menormalkan setiap aktivasi agar mengikuti distribusi standar. Distribusi standar ini ditunjukkan dengan nilai mean yang mendekati nol dan variance mendekati satu. Normalisasi ini membantu menjaga agar aliran gradien tetap terkontrol dan tidak mengalami eksplosif atau hilang (*vanishing/exploding gradients*).

Setelah proses normalisasi, Batch Normalization menambahkan dua parameter terlatih, yaitu γ (gamma) sebagai faktor skala dan β (beta) sebagai faktor geser. Kedua parameter ini memungkinkan jaringan untuk menyesuaikan kembali representasi fitur sesuai kebutuhan, sehingga model tetap fleksibel dalam mempelajari pola yang kompleks. Dengan demikian, Batch Normalization tidak hanya mempercepat pelatihan dan meningkatkan stabilitas gradien, tetapi juga berpotensi meningkatkan kemampuan generalisasi model secara keseluruhan. Berikut merupakan persamaan mini-batch mean, mini-batch variance, normalisasi, serta proses scaling dan shifting (Ioffe & Szegedy, 2015).

- a. Mini batch mean

$$\mu_B = \frac{1}{m} \sum_{j=1}^m x_j \dots\dots\dots (2.1)$$

Keterangan:

m = ukuran batch

x_i = nilai matriks

- b. Mini batch variance

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2 \dots\dots\dots (2.2)$$

Keterangan:

μ_B = nilai mini batch mean

- c. Normalize

$$\hat{x}_j = \frac{x_j - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \dots\dots\dots (2.3)$$

Keterangan:

ϵ = konstanta bernilai 10^{-5}

- d. Scaling dan shifting

$$y_j = \gamma \hat{x}_j + \beta \dots\dots\dots (2.4)$$

Keterangan:

γ = Scale bernilai 1,5

β = Shift bernilai 0,5

3. SILU

SILU (*Sigmoid Linear Unit*) adalah fungsi aktivasi non-linear yang digunakan karena kemampuannya menghasilkan transformasi yang lebih halus dibandingkan fungsi aktivasi tradisional seperti ReLU dan ELU. Secara matematis, SILU didefinisikan sebagai perkalian antara input x dengan fungsi sigmoid dari x , sehingga menghasilkan keluaran yang bersifat *smooth*, tidak terpotong, dan tetap mempertahankan informasi pada nilai negatif. Berbeda dengan RELU yang mengabaikan nilai negatif dari sebuah informasi citra. Hal ini membuat SILU meningkatkan performa pembelajaran model dan membantu model dalam mempelajari pola yang lebih kompleks secara lebih efektif. Tujuan utama penggunaan SILU adalah untuk meningkatkan kemampuan model dalam mempelajari pola non-linear yang kompleks serta memperbaiki stabilitas dan akurasi selama proses pelatihan model (Ramachandran, et al., 2017). Rumus fungsi SILU dituliskan sebagai berikut:

$$\text{SILU}(x) = x \times \frac{1}{1 + e^{-x}} \dots\dots\dots (2.5)$$

Keterangan:

$e(\exp) = 2,719292$

$\sigma(x)$ sigmoid dari $x = x \times \frac{1}{1 + e^{-x}}$

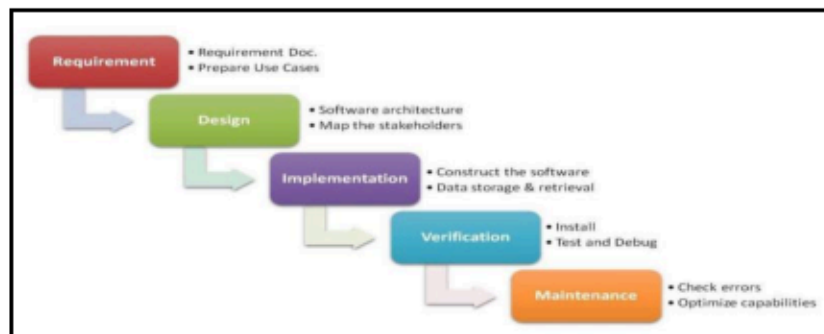
4. Maxpool

MaxPooling adalah salah satu operasi essential dalam Convolutional Neural Network (CNN) yang berfungsi guna mereduksi ukuran spasial fitur tanpa menghilangkan informasi penting di dalamnya. Operasi ini bekerja dengan cara membagi feature map ke dalam region kecil berukuran tetap umumnya 2×2 kemudian memilih nilai maksimum dari setiap region tersebut sebagai output. Pemilihan nilai terbesar memungkinkan jaringan mempertahankan fitur paling dominan, seperti tepi atau pola tekstur yang kuat, sambil mengurangi dimensi data sehingga *computational cost* menjadi lebih efisien dan mengurangi risiko overfitting dengan membuat model lebih fokus pada fitur utama dari objek (Zhao & Zhang, 2024).

2.7 Waterfall

Metode Waterfall adalah salah satu teknik pengembangan *software* yang bersifat terstruktur dan sistematis. Model ini menekankan proses pengembangan yang berjalan secara berurutan, di mana setiap fase harus diselesaikan sepenuhnya sebelum berlanjut ke fase berikutnya, serta tidak memungkinkan untuk kembali ke fase sebelumnya. Adapun tahapan utama dalam metode Waterfall meliputi Requirements, Design, Implementation, Verification, dan Maintenance.

Pada tahap Requirements, seluruh kebutuhan sistem dianalisis dan didokumentasikan secara lengkap sesuai dengan permasalahan yang akan diselesaikan. Tahap berikutnya adalah Design, yaitu proses merancang arsitektur perangkat lunak, alur data, struktur basis data, serta rancangan antarmuka berdasarkan kebutuhan yang telah ditetapkan. Setelah desain disetujui, proses dilanjutkan ke tahap Implementation, yaitu pengembangan dan penulisan kode program sesuai dengan arsitektur yang telah dirancang. Tahap berikutnya adalah Verification, yaitu proses instalasi, pengujian, dan debugging untuk memastikan perangkat lunak berjalan sesuai dengan kebutuhan dan bebas dari kesalahan. Selanjutnya pada tahap Maintenance, sistem yang telah dirilis akan dipelihara, diperbaiki, serta dikembangkan lebih lanjut apabila ditemukan kekurangan atau diperlukan penyesuaian di masa mendatang (Heriyanti. & Ishak, 2020). Gambaran umum dari *waterfall* dapat dilihat dari gambar 2.10 berikut.

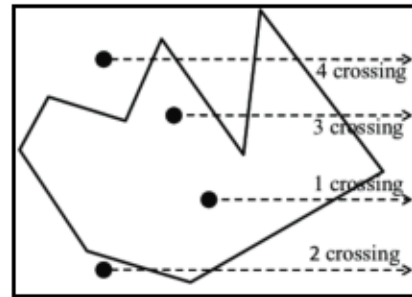


Gambar 2.10 Proses Metode Waterfall

Sumber : (Heriyanti. & Ishak, 2020)

2.8 Analisis Spasial

Analisis spasial merupakan pendekatan dalam pengolahan citra yang melibatkan lokasi serta hubungan spasial antar objek. Salah satu metode dasar analisis spasial yakni Point in Polygon (PIP), yaitu teknik untuk menentukan apakah suatu titik berada di dalam suatu poligon. Terdapat beberapa metode dalam PIP seperti Ray Casting Algorithm (Even-Odd Rule), dan Winding Number, Crossing Number (CN) (Kularathne & Jayarathne, 2018). Gambar 2.11 merupakan gambaran dari algoritma *ray-casting*.



Gambar 2.11 Ray Casting

(Sumber: Galetzka & Glauner, 2017)

Dalam object detection PIP dapat digunakan untuk mengelompokkan objek berdasarkan wilayah spasial tertentu. Metode ray casting merupakan metode umum digunakan dalam PIP, metode ini menarik garis lurus secara horizontal ke kanan/kiri lalu menghitung jumlah perpotongan yang terjadi antara garis tersebut dengan batas poligon. Jika perpotongan yang terjadi berjumlah ganjil maka titik tersebut didalam poligon, sedangkan jika berjumlah genap titik tersebut berada pada luar poligon. (Galetzka & Glauner, 2017). Secara matematis, misalkan poligon P terdiri dari n verteks terurut:

$$P_1, P_2, \dots, P_n = (x_i, y_i) \dots \dots \dots (2.6)$$

dengan sisi $\overline{P_i P_{i+1}}$ untuk $1 \leq i < n$ dan sisi terakhir $\overline{P_n P_1}$.

Untuk setiap sisi poligon, diuji apakah terjadi perpotongan antara sinar horizontal yang ditarik dari titik $Q(x_q, y_q)$ ke arah kanan dengan sisi $\overline{P_i P_{i+1}}$. Suatu sisi dianggap berpotongan jika memenuhi dua kondisi berikut:

$$(y_i > y_q) \neq (y_{i+1} > y_q) \dots \dots \dots (2.7)$$

(Persamaan 1: menentukan apakah sisi poligon dipotong oleh sinar horizontal dari titik uji) Apabila kondisi bahwa titik Q berada di antara rentang vertikal sisi tersebut terpenuhi, maka koordinat titik potong pada sumbu x dapat dihitung menggunakan interpolasi linear sebagai berikut:

$$x_{\text{int}} = x_i + (y_q - y_i) \cdot \frac{x_{i+1} - x_i}{y_{i+1} - y_i} \dots \dots \dots (2.8)$$

(Persamaan 2: menentukan posisi titik potong antara sinar horizontal dan sisi poligon)

Kondisi pertama memastikan bahwa sinar horizontal dari Q berada dalam rentang vertikal sisi (y -range constraint). Kondisi ini penting agar garis tidak dihitung berpotongan ketika berada di atas atau di bawah keseluruhan sisi poligon. Kemudian, titik potong dianggap valid apabila berada di sebelah kanan titik Q , yaitu memenuhi:

$$x_{\text{int}} > x_q \dots \dots \dots (2.9)$$

Jika kedua kondisi tersebut terpenuhi, maka satu perpotongan dicatat. Jumlah total perpotongan dengan seluruh sisi poligon didefinisikan sebagai:

$$k = \sum_{i=1}^n 1 \left((y_i > y_q) \neq (y_{i+1} > y_q) \wedge x_{int} > x_q \right) \dots\dots\dots (2.10)$$

Apabila nilai k merupakan bilangan ganjil, maka titik Q berada di dalam poligon menurut aturan Even-Odd Rule. Sebaliknya, jika k bernilai genap, maka titik Q berada di luar poligon. Metode ini menjadi dasar dari Ray Casting Algorithm yang banyak digunakan dalam analisis spasial dan polygon-based region classification.

2.9 Matriks Evaluasi

Matriks evaluasi adalah kumpulan metrik yang digunakan guna mengukur tingkat efektivitas dan kinerja model machine learning, terutama dalam tugas seperti object detection dan instance segmentation. Salah satu pendekatan utama dalam evaluasi ini adalah penggunaan confusion matrix, yang mengklasifikasikan hasil prediksi model ke dalam beberapa kategori, yaitu (Padilla et al., 2020):

1. True Positive (TP): Model berhasil mendeteksi objek yang ada pada ground truth.
2. False Positive (FP): Model mendeteksi objek padahal objek tersebut tidak ada pada ground truth.
3. False Negative (FN): Model gagal mendeteksi objek yang sebenarnya ada pada ground truth.

3 Kategori tersebut didapatkan dengan cara menghitung IoU dengan mengukur irisan antar bounding box prediksi dengan bounding box ground truth. Jika Nilai IoU lebih dari IoU threshold maka bernilai TP sedangkan jika IoU kurang dari IoU threshold maka bernilai FP. Dari ketiga nilai tersebut, dihitung beberapa metrik penting untuk menilai performa model (Padilla et al., 2020):

1. Precision: Menunjukkan proporsi prediksi yang benar dari seluruh objek yang berhasil dideteksi oleh model.

$$\text{Precision} = \frac{TP}{TP + FP} \times 100 = \frac{TP}{\text{all detection}} \times 100 \dots\dots\dots (2.11)$$

2. Recall: Menunjukkan seberapa banyak objek yang berhasil terdeteksi dari seluruh objek yang ada pada *ground truth*.

$$\text{Recall} = \frac{TP}{TP + FN} \times 100 = \frac{TP}{\text{all ground truth}} \times 100 \dots\dots\dots (2.12)$$

3. F1Score: F1-score merupakan metrik evaluasi performa model klasifikasi yang digunakan untuk mengukur keseimbangan antara precision dan recall.

$$\text{F1 - score} = 2 \times \frac{\text{Precision} + \text{Recall}}{\text{Precision} \times \text{Recall}} \dots\dots\dots (2.13)$$

4. Average Precision (AP): Merupakan ukuran area di bawah kurva Precision-Recall untuk satu kelas, yang menunjukkan keseimbangan antara precision dan recall pada berbagai threshold.

5. mean Average Precision (mAP): Merupakan rata-rata dari nilai AP untuk semua kelas. Dalam object detection, mAP dihitung pada IoU threshold tertentu, misalnya:

- $mAP@0.5$: rata – rata AP dari $IoU \geq 0.5$
- $mAP@0.5: 0.95$: rata-rata AP dari IoU 0.5 hingga 0.95 dengan interval 0.05.

2.10 Tinjauan Literatur

Terdapat beberapa penelitian terdahulu yang membahas mengenai deteksi makanan yang menjadi dasar dalam pembuatan penelitian yang akan dilaksanakan. Penelitian terdahulu yang dijadikan sebagai referensi dapat dilihat pada tabel 2.2 berikut:

Table 2.2 State of Chart

No.	Penelitian	Metode	Dataset	Hasil Penelitian
1.	Implementasi You Only Look Once v8 Dalam Deteksi Makanan Warung Tegal Untuk Sistem Perhitungan Harga Otomatis. (Purnomo et al., 2025)	YOLO v8	Dataset primer dikumpulkan menggunakan smartphone dengan jumlah 400 gambar dengan 40 class	Precision, Recall, F1-Score, box_loss, dan mAP@0.5 berturut turut yakni 0.602, 0.176, 0.32, 1.756, dan 0.229. Akurasi rendah karena keterbatasan data serta kompleksitas latar blekang.
2.	IndianFoodNet: Detecting Indian Food Items Using Deep Learning (Agarwal et al., 2023)	YOLO v5, YOLO v7, dan YOLO v8	IndianFoodNet dengan 5500 gambar dengan 30 class	Hasil perbandingan model mAP nya yakni YOLOv5 0.671, YOLOv7 0.291, dan YOLOv8 0.807.
3.	Comparing YOLOv8 and Mask R-CNN for instance segmentation in complex orchard environments (Sapkota et al., 2024)	YOLOv8 dan Mask R-CNN	Dikumpulkan dari kebun apel komersial washinton state USA. Dataset I musin istirahat (474 gambar, 3 kelas), dataset II awal musim tanam (1079 gambar, 1 kelas)	YOLOv8 menghasilkan akurasi dan kecepatan yang lebih baik. Dataset I dengan kecepatan 10.9ms, precision 90% dan recall of 95% sedangkan pada dataset II kecepatan 7.8ms, precision 93% dan recall 97%.
4	A Simple and Correct Even-Odd Algorithm for the Point-in-Polygon Problem for Complex Polygons (Galetzka & Glauner, 2017)	Even Odd Algorithm	Gambar complex polygon	Merumuskan algoritma Even Odd untuk menentukan kondisi titik dalam suatu polygon tertentu
5	Klasifikasi Jajanan Tradisional Indonesia berbasis Deep Learning dan Metode Transfer Learning (Fahturaman et al., 2023)	MobilenetV2, ResNet50 dan VGG16.	Dataset diperoleh dari unduhan google dengan 1500 gambar dan 5 kelas	Didapatkan hasil terbaik yakni mobilenetV2 dengan akurasi 98%, diikuti metode ResNet50 dan VGG16 dengan akurasi masing masing 97.33% dan 93.33%.
6	Food Image Recognition Using CNN, Faster R-CNN and YOLO (Chopra. & Purwar, 2022)	CNN, Faster R-CNN and YOLO	Dataset berasal dari UEC FOOD-100 yang terdiri dari 100 kelas berbeda dengan total gambar berkisar 14500	Yolo menghasilkan akurasi yang lebih baik daripada CNN, dan Faster R-CNN dengan akurasi 0.741, precision 95.82% dan recall 91.23%

Penelitian Implementasi You Only Look Once v8 dalam Deteksi Makanan Warung Tegal untuk Sistem Perhitungan Harga Otomatis (Purnomo et al., 2025) telah mengkaji permasalahan deteksi makanan pada lingkungan prasmanan dengan dataset primer sebanyak 400 citra dan 40 kelas yang dikumpulkan menggunakan smartphone. Hasil pengujian menunjukkan nilai precision 0,602, recall 0,176, F1-score 0,32, box loss 1,756, serta

mAP@0,5 sebesar 0,229. Meskipun demikian, penelitian tersebut mengidentifikasi bahwa kompleksitas latar belakang menjadi faktor utama yang memengaruhi performa deteksi sehingga perlu dikaji lebih lanjut.

Food Image Recognition Using CNN, Faster R-CNN and YOLO (Chopra. & Purwar, 2022). Menunjukkan bahwa Yolo menghasilkan akurasi yang lebih baik daripada CNN, dan Faster R-CNN dengan akurasi 0.741, precision 95.82% dan recall 91.23%. Dataset tersebut berasal dari UEC FOOD-100 yang terdiri dari 100 kelas berbeda dengan total gambar berkisar 14500.

Penelitian IndianFoodNet: Detecting Indian Food Items Using Deep Learning (Agarwal et al., 2023) membandingkan performa YOLOv5, YOLOv7, dan YOLOv8 pada dataset IndianFoodNet yang terdiri dari 5.500 citra dan 30 kelas. Hasil evaluasi menunjukkan bahwa YOLOv8 memperoleh nilai mAP tertinggi sebesar 0,807, sehingga menunjukkan kinerja yang lebih baik dibandingkan versi sebelumnya. Selain itu, penelitian Comparing YOLOv8 and Mask R-CNN for Instance Segmentation in Complex Orchard Environments (Sapkota et al., 2024) menunjukkan bahwa YOLOv8 memiliki akurasi dan kecepatan inferensi yang lebih unggul dibandingkan Mask R-CNN pada kondisi objek yang saling *overlapping*.

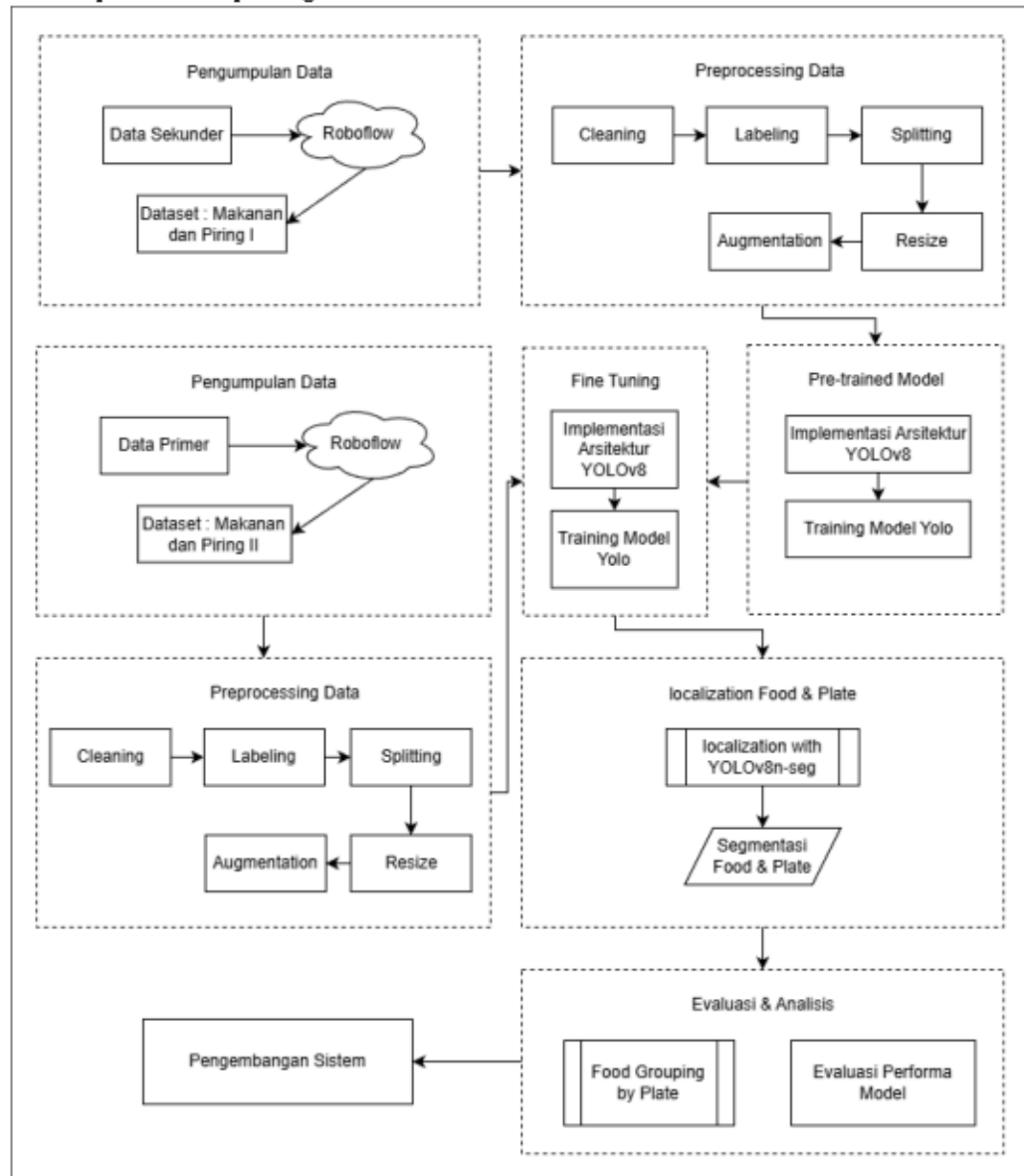
Untuk penentuan objek berdasarkan area wadah, penelitian A Simple and Correct Even-Odd Algorithm for the Point-in-Polygon Problem for Complex Polygons (Galetzka & Glauner, 2017) menunjukkan bahwa algoritma even-odd rule mampu menentukan apakah suatu titik berada di dalam poligon secara akurat. Pendekatan ini relevan untuk mengatasi permasalahan multiwadah, yaitu mencegah penggabungan dua porsi makanan yang berbeda menjadi satu porsi apabila berada pada wadah yang berbeda.

Sementara itu, penelitian Klasifikasi Jajanan Tradisional Indonesia berbasis Deep Learning dan Metode Transfer Learning (Fahturaman et al., 2023) menggunakan MobileNetV2, ResNet50, dan VGG16 dengan akurasi terbaik 98% pada MobileNetV2. Namun, penelitian tersebut hanya melakukan klasifikasi citra tanpa melakukan deteksi objek, sehingga belum mampu menangani permasalahan lokasi dan jumlah objek dalam satu citra.

Berdasarkan kajian tersebut, penelitian ini menggunakan YOLOv8n-seg karena memiliki performa deteksi yang lebih baik pada objek dengan latar belakang kompleks dan kondisi *overlapping*. Selain itu, algoritma even-odd rule diterapkan untuk menentukan keterkaitan objek makanan dengan wadahnya sehingga dapat mencegah kesalahan penggabungan porsi dan mendukung perhitungan harga otomatis secara lebih akurat.

BAB III METODOLOGI PENELITIAN

Penelitian ini bertujuan untuk mendeteksi makanan serta mengelompokkannya berdasarkan object wadah menggunakan model yolov8 serta algoritma ray casting. Metode penelitian ini dapat dilihat pada gambar 3.1 dibawah ini.



Gambar 3.1 Alur Metodologi Penelitian

3.1 Pengumpulan Data

Data yang digunakan pada proses pelatihan model pre trained merupakan data sekunder yang diperoleh melalui proses pengunduhan gambar dari situs penyedia gambar makanan, seperti Shutterstock dan iStock. Dataset makanan dan wadah tersebut berjumlah 4608 gambar yang telah dianotasi secara mandiri menggunakan platform Roboflow sebagai alat bantu anotasi. Proses anotasi dilakukan untuk memberikan label pada setiap objek yang akan digunakan dalam proses pelatihan model deteksi. Beberapa contoh gambar yang terdapat pada dataset dapat dilihat pada Gambar 3.2.



Gambar 3.2 Data Sekunder

Dataset yang digunakan untuk tahap fine-tuning merupakan dataset primer yang diperoleh melalui proses pengambilan gambar secara langsung menggunakan kamera Xiaomi 12 Lite. Proses pengambilan gambar disesuaikan dengan kondisi lingkungan pembayaran yang sebenarnya untuk meminimalkan kesalahan prediksi objek dan kesalahan pengelompokan harga. Dataset primer ini mencakup beragam variasi kondisi agar model dapat beradaptasi dengan situasi di lapangan.

Pada pengambilan data makanan single-class terdapat 10 jenis objek makanan dengan masing masing jenis terdiri dari 240 gambar, gambar ini diambil untuk keperluan training dan validation dengan kondisi variasi latar belakang, objek yang berbeda serta kondisi *overlapping* objek sebagai berikut:

- a. Satu wadah berisi satu objek, sebanyak 60 gambar menggunakan wadah A untuk menambah variasi bentuk wadah dan objek.
- b. Satu wadah berisi satu objek, sebanyak 60 gambar menggunakan wadah B.
- c. Satu wadah berisi dua objek, sebanyak 60 gambar menggunakan wadah B.
- d. Dua wadah berisi masing-masing satu objek (1 objek – 1 objek), sebanyak 60 gambar, untuk menambah variasi kondisi objek terpisah maupun kemungkinan tumpang tindih.

Sementara itu, pengambilan data makanan multi-class digunakan untuk proses training, validation, dan testing dengan kondisi sebagai berikut:

- a. Satu wadah dengan satu objek, sebanyak 9 gambar.
- b. Satu wadah dengan dua objek, sebanyak 36 gambar.
- c. Satu wadah dengan tiga objek, sebanyak 84 gambar.
- d. Dua wadah dengan masing-masing satu objek, sebanyak 45 gambar.
- e. Dua wadah dengan masing-masing dua objek, sebanyak 666 gambar.
- f. Tiga wadah dengan masing-masing satu objek, sebanyak 9 gambar.
- g. Tiga wadah dengan masing-masing dua objek, sebanyak 15 gambar.

Sedangkan untuk data testing untuk pengujian berjumlah 360 gambar yang berbeda beda dengan kondisi jumlah gambar sebagai berikut :

- a. Satu piring tidak tumpang tindih sebanyak 30 Gambar
- b. Dua piring tidak tumpang tindih sebanyak 30 Gambar
- c. Tiga piring tidak tumpang tindih sebanyak 30 Gambar
- d. Empat piring tidak tumpang tindih sebanyak 30 Gambar
- e. Lima piring tidak tumpang tindih sebanyak 30 Gambar
- f. Enam piring tidak tumpang tindih sebanyak 30 Gambar

- g. Satu piring tumpang tindih sebanyak 30 Gambar
- h. Dua piring tumpang tindih sebanyak 30 Gambar
- i. Tiga piring tumpang tindih sebanyak 30 Gambar
- j. Empat piring tumpang tindih sebanyak 30 Gambar
- k. Lima piring tumpang tindih sebanyak 30 Gambar
- l. Enam piring tumpang tindih sebanyak 30 Gambar

Untuk jumlah objek dari masing masing makanan yang ada dalam dataset ini dapat dilihat pada tabel 3.1 berikut:

Table 3.1 Jumlah Objek Setiap Jenis Makanan

Class Name	Count
ayam_goreng_crispy	1880
bakwan_sayur	2926
ikan_mujair	1487
lele_goreng	2019
mendoan	2636
nasi_putih	4127
piring	10598
tahu	2906
telur_bacem	1869
tempe	2864
turnis_kangkung	1520

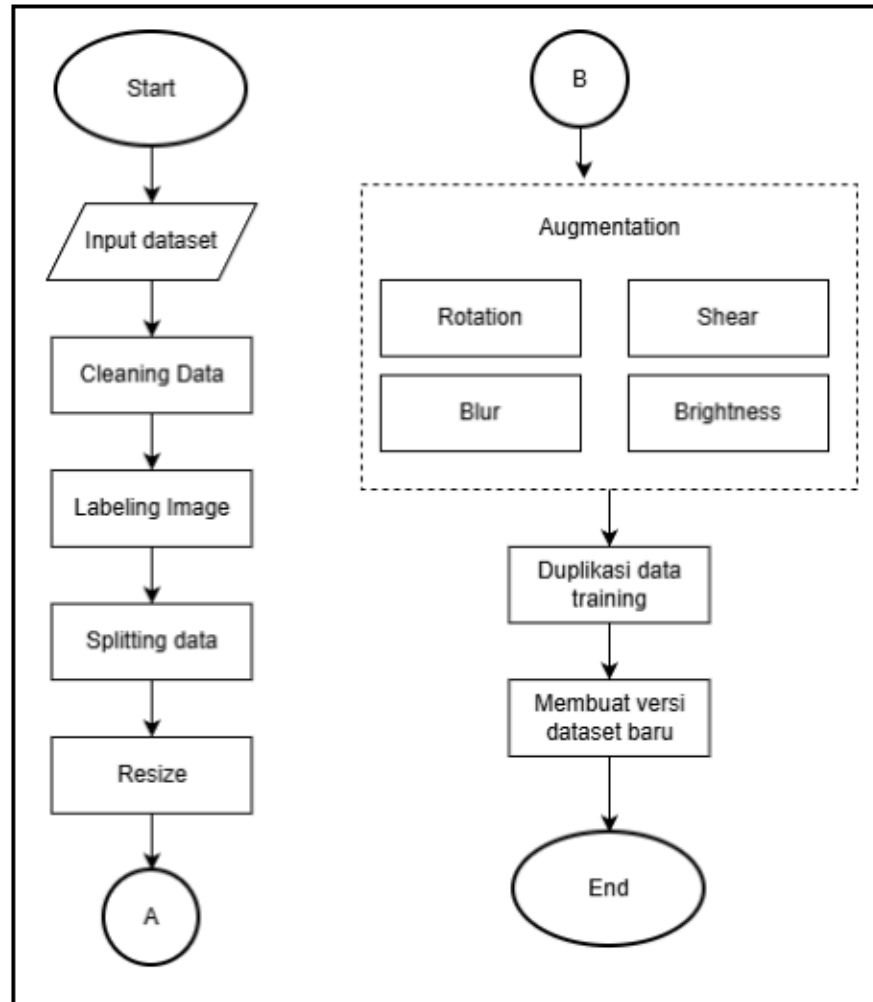
Pendekatan ini dilakukan agar dataset primer dapat mencakup berbagai variasi kondisi yang mungkin ditemui pada skenario nyata di lingkungan pembayaran. Dengan memanfaatkan dataset sekunder sebagai pre-trained model serta dataset primer untuk proses fine-tuning, diharapkan model mampu menjadi lebih robust dalam menghadapi domain gap antara data internet dan kondisi data di lapangan. Contoh data pelatihan dan validation yang digunakan ditampilkan pada Gambar 3.3.



Gambar 3.3 Data Validation

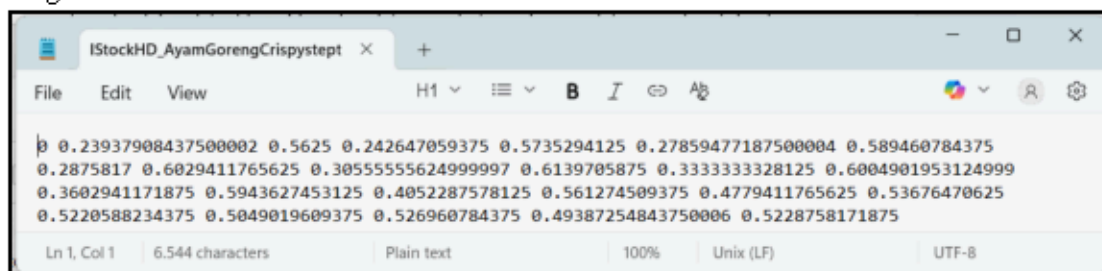
3.2 Pre-Processing

Pada bagian ini dilakukan proses persiapan data sebelum digunakan dalam pelatihan model. Tahapan *preprocessing* terbagi menjadi lima langkah utama, yaitu *cleaning*, *labeling*, *splitting*, *resize*, dan *augmentasi*. Seluruh tahap ini dilakukan melalui platform Roboflow saat melakukan pembuatan versi dataset. Alur lengkap proses *preprocessing* dapat dilihat pada Gambar 3.4.



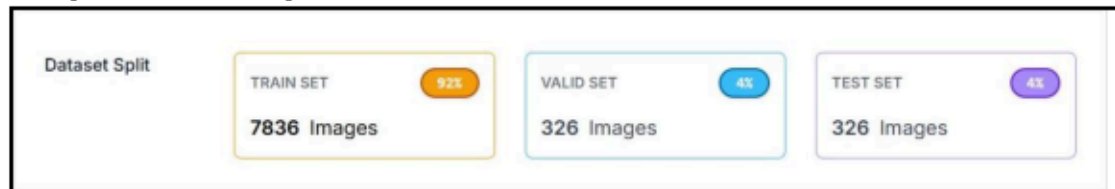
Gambar 3.4 Alur Metodologi Penelitian

Tahap pertama adalah cleaning data, yaitu proses membersihkan dataset dari gambar-gambar yang tidak relevan, buram, duplikat, atau tidak sesuai dengan kasus yang ditangani. Tujuan dari tahap ini adalah memastikan bahwa hanya data yang berkualitas dan sesuai konteks yang digunakan pada pelatihan model. Tahap selanjutnya adalah labeling, yaitu proses memberi anotasi pada setiap objek yang ingin dideteksi. Pada tahap ini, setiap gambar dilabeli dengan kelas objek dan koordinat *bounding box* atau segmentasi yang menggambarkan posisi objek secara presisi. Proses labeling bertujuan agar model dapat belajar mengenali lokasi dan kelas objek secara akurat. Contoh hasil labeling dapat dilihat pada gambar 3.5 berikut.



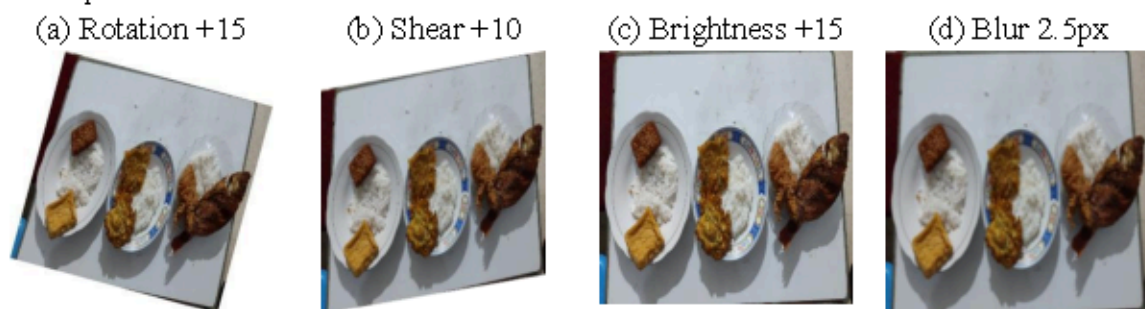
Gambar 3.5 Contoh hasil labelling

Setelah proses anotasi selesai, dataset melalui tahap splitting, yaitu pembagian data menjadi tiga subset: *training*, *validation*, dan *testing*. Pada penelitian ini, pembagian dataset dilakukan dengan proporsi 80% data training (2608 gambar), 10% data validation (326 gambar), dan 10% data testing (326 gambar). Data training dan validation digunakan dalam proses pelatihan model, sedangkan data testing digunakan sebagai evaluasi akhir performa model setelah pelatihan selesai. Gambar 3.6 berikut merupakan pembagian dataset untuk proses pelatihan model *fine tune*.



Gambar 3.6 Splitting Dataset

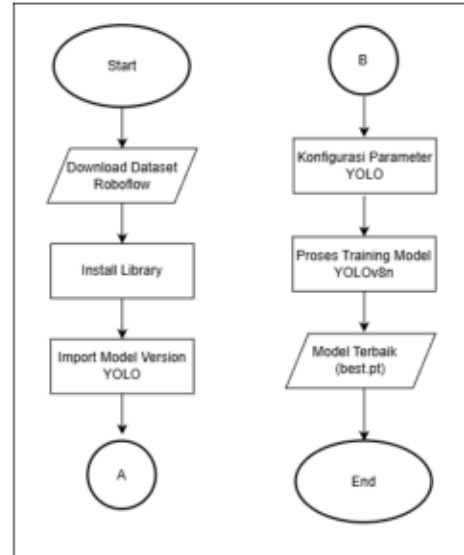
Tahap berikutnya adalah *resize*, yaitu mengubah ukuran seluruh gambar menjadi 640×640 piksel untuk menyesuaikan format input standar YOLOv8. Setelah itu dilakukan tahap augmentasi, yang bertujuan meningkatkan variasi data sehingga model lebih *robust* dalam menghadapi kondisi nyata. Proses augmentasi meliputi teknik seperti rotasi, perubahan pencahayaan, *flip*, dan perubahan skala. Setelah augmentasi diterapkan, jumlah data pelatihan yang semula sebanyak 2608 gambar meningkat menjadi 7836 gambar, sehingga total keseluruhan dataset menjadi 8488 gambar. Contoh hasil *resize* dan augmentasi dapat dilihat pada Gambar 3.7.



Gambar 3.7 Contoh Resize dan Augmentation

3.3 Implementasi Model

Proses pelatihan model dilakukan menggunakan lingkungan pengembangan Visual Studio Code dengan memanfaatkan dua pustaka utama, yaitu Roboflow untuk pengelolaan dataset dan Ultralytics sebagai framework model YOLOv8-seg. Setelah melewati tahapan *preprocessing*, dataset dilatih menggunakan konfigurasi parameter berupa jumlah *epoch* sebanyak 100, ukuran input gambar 640×640 piksel, *batch size* 16, serta penggunaan algoritma optimizer Adam. Pada tahap awal, model YOLOv8-seg dilatih menggunakan dataset sekunder untuk memperoleh kemampuan dasar sebagai *pre-trained model*. Selanjutnya, model tersebut di-*fine-tune* menggunakan dataset primer yang telah disiapkan untuk menyelesaikan tugas deteksi yang lebih spesifik. Alur proses pembuatan model ditampilkan pada Gambar 3.8.



Gambar 3.8 Alur Implementasi Model YOLOv8

Model hasil *fine-tuning* memiliki struktur jaringan yang terdiri dari 151 layer, 3265761 parameter, dan 3265745 gradient, dengan beberapa komponen utama seperti lapisan konvolusi, *batch normalization*, *activation function* SiLU, serta *max pooling* pada bagian tertentu. Gambaran lengkap arsitektur model YOLOv8-segn yang dihasilkan setelah proses pelatihan ditunjukkan pada Gambar 3.9.

	from	N	Params	Module	Arguments
0	-1	1	464	Ultralytics.nn.modules.conv.Conv	[3, 16, 3, 2]
1	-1	1	4677	Ultralytics.nn.modules.conv.Conv	[16, 32, 3, 2]
2	-1	1	7360	Ultralytics.nn.modules.block.C2f	[32, 32, 1, true]
3	-1	1	18560	Ultralytics.nn.modules.conv.Conv	[32, 64, 3, 2]
4	-1	1	49664	Ultralytics.nn.modules.block.C2f	[64, 64, 2, true]
5	-1	1	74984	Ultralytics.nn.modules.conv.Conv	[64, 128, 3, 2]
6	-1	1	197632	Ultralytics.nn.modules.block.C2f	[128, 128, 2, true]
7	-1	1	295424	Ultralytics.nn.modules.conv.Conv	[128, 256, 3, 2]
8	-1	1	460288	Ultralytics.nn.modules.block.C2f	[256, 256, 1, true]
9	-1	1	164608	Ultralytics.nn.modules.conv.SPFF	[256, 256, 5]
10	-1	1	0	Torch.nn.upsample.upsampling	[none, 2 'nearest']
11	[-1, 6]	1	0	Ultralytics.nn.modules.conv.Concat	[1]
12	-1	1	148224	Ultralytics.nn.modules.block.C2f	[384, 128, 1]
13	-1	1	0	Torch.nn.upsample.upsampling	[none, 2 'nearest']
14	[-1, 4]	1	0	Ultralytics.nn.modules.conv.Concat	[1]
15	-1	1	37428	Ultralytics.nn.modules.block.C2f	[192, 64, 1]
16	-1	1	36992	Ultralytics.nn.modules.conv.Conv	[64, 64, 3, 2]
17	[-1, 12]	1	0	Ultralytics.nn.modules.conv.Concat	[1]
18	-1	1	123648	Ultralytics.nn.modules.block.C2f	[192, 128, 1]
19	-1	1	147712	Ultralytics.nn.modules.conv.Conv	[128, 128, 3, 2]
20	[-1, 9]	1	0	Ultralytics.nn.modules.conv.Concat	[1]
21	-1	1	493056	Ultralytics.nn.modules.block.C2f	[384, 256, 1]
22	[15, 18, 21]	1	1006225	Ultralytics.nn.modules.head.segment	[11, 32, 64, [64, 128, 256]]

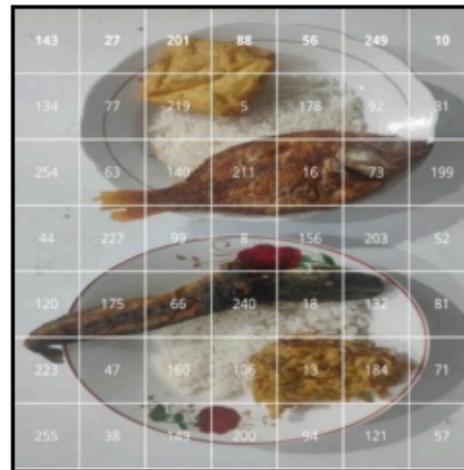
YOLOv8n-seg summary: 151 layers, 3.265.761 parameters, 3.265.745 gradients, 12.1 GFLOPs

Gambar 3.9 Arsitektur Model YOLOv8n

Gambar 3.9 juga menjelaskan detail struktur jaringan, termasuk jumlah lapisan (*layers*), jumlah parameter (*params*), serta jumlah *gradient updates* yang terlibat pada proses pelatihan. Pada bagian selanjutnya, ditampilkan contoh perhitungan matematis yang berkaitan dengan arsitektur model YOLOv8-segn, meliputi perhitungan pada lapisan konvolusi, batch normalization dengan proses normalisasi dan scale-shift, fungsi aktivasi SiLU, serta lapisan max pooling.

a. Lapisan Konvolusi

Lapisan konvolusi bertujuan untuk menangkap pola lokal pada citra, seperti tepi, tekstur, dan bentuk-bentuk berukuran kecil yang menjadi dasar pembentukan fitur. Pada tahap awal, citra input dibagi ke dalam elemen-elemen piksel yang akan diproses oleh kernel konvolusi. Dalam penelitian ini, citra contoh dibagi ke dalam grid berukuran 7×7 piksel sebagaimana ditunjukkan pada Gambar 3.10. Nilai pixel disini diambil dari contoh kernel red dari red green blue (RGB). Pembagian ini memungkinkan kernel untuk melakukan operasi ekstraksi fitur secara lebih terfokus pada area lokal citra. Gambar 3.10 adalah contoh dari pembagian citra grid.



Gambar 3.10 Pembagian Grid Citra

Pada lapisan konvolusi, digunakan kernel berukuran 3×3 yang berfungsi sebagai filter untuk mengekstraksi fitur lokal dari citra. Nilai-nilai pada kernel pada awalnya diinisialisasi secara acak dan kemudian diperbarui selama proses pelatihan melalui mekanisme backpropagation. Pembaruan nilai ini memungkinkan model untuk belajar pola yang relevan terhadap tugas deteksi. Adapun nilai kernel yang digunakan pada contoh perhitungan diambil dari sharpen kernel, penggunaan sharpen kernel bertujuan untuk menonjolkan tepi objek dan meningkatkan ketajaman citra, karena bobot positif terbesar berada pada piksel tengah, sedangkan piksel di sekelilingnya diberi bobot negatif. Dengan demikian, perbedaan intensitas antar piksel akan diperkuat sehingga batas objek terlihat lebih jelas. Sharpen kernel dapat dilihat pada matrik berikut.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Setelah itu, matriks input diberikan *zero padding* pada seluruh sisi dengan menambahkan nilai nol di bagian luar matriks, sehingga ukuran citra yang semula 7×7 bertambah menjadi 9×9 . Penerapan *zero padding* dilakukan agar dimensi keluaran setelah proses konvolusi tetap terjaga serta untuk mempertahankan informasi pada bagian tepi citra. Tanpa adanya padding, piksel-piksel yang berada di area pinggir hanya akan diproses sebagian atau

bahkan terabaikan, sehingga berpotensi menyebabkan hilangnya fitur penting yang berada di batas objek. Oleh karena itu, zero padding menjadi langkah penting agar seluruh area citra, termasuk bagian tepi, tetap memperoleh perlakuan yang seimbang selama proses ekstraksi fitur. Contoh perhitungan proses konvolusi menggunakan kernel 3×3 dapat dilihat pada Gambar 3.11.

0	0	0	0	0	0	0	0	0
0	143	27	201	88	56	249	10	0
0	134	77	219	5	178	92	31	0
0	254	63	140	211	16	73	199	0
0	44	227	90	8	156	260	52	0
0	120	175	66	240	18	132	81	0
0	223	447	180	106	13	184	71	0
0	255	38	149	200	94	121	57	0
0	0	0	0	0	0	0	0	0

$\times$

0	-1	0
-1	5	-1
0	-1	0

$=$

554	671	-235	-230
1029	108	-538	794
158	-344	-451	150
1014	347	136	93

Gambar 3.11 Ilustrasi Proses Lapisan Konvolusi

Gambar 3.11 tersebut menunjukkan matriks pembagian citra grid yang terdapat pada gambar 3.10 dengan dot product sharpen kernel. Kernel berukuran 3×3 digeser secara sistematis ke seluruh area citra dengan pola pergeseran dari kiri ke kanan dan dari atas ke bawah. Operasi konvolusi dilakukan dengan *stride* sebesar 2 sehingga kernel bergeser dua piksel pada setiap langkah perhitungan. Pengaturan *stride* ini menyebabkan ukuran *feature map* yang dihasilkan menjadi lebih kecil dibandingkan ukuran input awal.

Pada setiap posisi, kernel akan menutupi sebagian area citra berukuran 3×3 , kemudian dilakukan proses *dot product*, yaitu perkalian antara setiap elemen kernel dengan nilai piksel yang bersesuaian pada area tersebut. Seluruh hasil perkalian tersebut kemudian dijumlahkan untuk menghasilkan satu nilai keluaran. Nilai ini ditempatkan pada posisi yang sesuai pada matriks hasil atau feature map. Untuk proses perhitungan tahap pertama konvolusi *dot product* antara pembagian citra grid dengan kernel 3×3 dapat dilihat pada tabel 3.2 berikut ini.

Table 3.2 Perhitungan Proses Konvolusi

Proses	Perhitungan	Hasil
1	$(0 \times 0) + (0 \times -1) + (0 \times 0) + (0 \times -1) + (143 \times 5) + (27 \times -1) + (0 \times 0) + (134 \times -1) + (77 \times 0)$	554
2	$(0 \times 0) + (0 \times -1) + (0 \times 0) + (27 \times -1) + (201 \times 5) + (88 \times -1) + (77 \times 0) + (219 \times -1) + (5 \times 0)$	671
3	$(0 \times 0) + (0 \times -1) + (0 \times 0) + (88 \times -1) + (56 \times 5) + (249 \times -1) + (5 \times 0) + (178 \times -1) + (92 \times 0)$	-235
4	$(0 \times 0) + (0 \times -1) + (0 \times 0) + (249 \times -1) + (10 \times 5) + (0 \times -1) + (92 \times 0) + (31 \times -1) + (0 \times 0)$	-230

Proses tersebut dilakukan berulang hingga kernel menjangkau seluruh bagian citra. Hasil akhirnya adalah sebuah feature map baru yang berisi representasi fitur-fitur lokal dari citra asli. Nilai-nilai pada feature map

menunjukkan respons kernel terhadap pola tertentu pada citra. Jika kernel yang digunakan adalah sharpen kernel, maka feature map akan lebih menonjolkan perubahan intensitas piksel, seperti batas objek, garis tepi, serta detail tekstur. Dengan demikian, informasi visual penting pada citra menjadi lebih jelas dan lebih mudah dikenali pada tahap pemrosesan berikutnya.

Dalam konteks penelitian ini, proses konvolusi berperan penting sebagai tahap awal ekstraksi fitur pada citra makanan. Feature map yang dihasilkan akan membantu model dalam mengenali karakteristik objek, seperti bentuk makanan, tekstur permukaan, serta pemisahan antar objek yang saling berdekatan, sehingga sistem dapat melakukan deteksi serta segmentasi objek secara lebih akurat. Hasil perhitungan konvolusi pada setiap posisi kernel terhadap citra input kemudian membentuk sebuah lapisan keluaran (*convolution layer*), yang ditampilkan pada matriks berikut.

$$\begin{bmatrix} 554 & 671 & -235 & -230 \\ 1029 & 108 & -538 & 794 \\ 158 & -344 & -451 & 150 \\ 1014 & 347 & 136 & 93 \end{bmatrix}$$

b. Batch Normalization

Batch Normalization (BN) digunakan untuk mempercepat sekaligus menstabilkan proses pelatihan model. Selain meningkatkan stabilitas, BN juga memberikan efek regularisasi ringan sehingga model menjadi lebih tahan terhadap *overfitting*. Proses BN terdiri dari beberapa tahapan utama, yaitu perhitungan *mini-batch mean*, perhitungan *mini-batch variance*, normalisasi nilai berdasarkan kedua hasil perhitungan sebelumnya, serta tahap *scaling* dan *shifting* menggunakan parameter terlatih γ (gamma) dan β (beta). Pada penelitian ini, perhitungan *mini-batch mean* dilakukan menggunakan nilai-nilai keluaran konvolusi dan dihitung menggunakan Persamaan 2.1.

$$\begin{aligned} \mu_B &= \frac{1}{16} \sum_{i=1}^{16} x_1 + x_2 + \dots + x_{15} + x_{16} \\ \mu_B &= \frac{1}{16} (554 + 671 + \dots + 136 + 93) \\ \mu_B &= 203,5 \end{aligned}$$

Setelah nilai *mini-batch mean* diperoleh, langkah berikutnya adalah menghitung *mini-batch variance*. Nilai *mini-batch variance* dihitung menggunakan nilai-nilai keluaran konvolusi, bersama dengan hasil *mini-batch mean* yang telah diperoleh pada tahap sebelumnya, dan dirumuskan menggunakan Persamaan 2.2.

$$\begin{aligned} \sigma_B^2 &= \frac{\sum_{i=1}^{16} (x_i - \mu_B)^2}{16} \\ \sigma_B^2 &= \frac{(554 - 203,5)^2 + \dots + (93 - 203,5)^2}{16} \\ \sigma_B^2 &= 233626,375 \end{aligned}$$

$$\sigma_B = 483,349$$

Setelah proses perhitungan *mini-batch variance* selesai, langkah berikutnya adalah melakukan normalisasi terhadap setiap elemen pada feature map. Sebagai contoh, perhitungan normalisasi untuk elemen matriks pada posisi (0,0) ditunjukkan sesuai dengan Persamaan 2.3.

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

$$\hat{x} = \frac{554 - 203,5}{\sqrt{233626,375 + 10^{-5}}}$$

$$\hat{x} = 0,725$$

Proses ini diulang hingga element sel terakhir sehingga mendapatkan hasil normalisasi yang dapat dilihat sesuai dengan matriks berikut ini.

$$\begin{bmatrix} 0.725 & 0.967 & -0.907 & -0.897 \\ 1.708 & -0.198 & -1.534 & 1.222 \\ -0.094 & -1.133 & -1.354 & -0.111 \\ 1.677 & 0.297 & -0.140 & -0.229 \end{bmatrix}$$

Tahap terakhir dalam proses Batch Normalization adalah *scaling* dan *shifting*. Setelah seluruh elemen pada feature map dinormalisasi, nilai-nilai tersebut kemudian dikalibrasi ulang menggunakan parameter γ (*gamma*) dan β (*beta*) yang dapat dipelajari oleh model selama proses pelatihan. Contoh perhitungan untuk elemen matriks pada posisi (0,0) ditunjukkan sesuai dengan Persamaan 2.4.

$$y = \gamma \hat{x} + \beta$$

$$y = 1,5(0,725) + 0,5$$

$$y = 1,588$$

Proses ini diulang sampai pada element sel terakhir pada matriks sehingga menghasilkan matriks *scaling shifting* berikut ini.

$$\begin{bmatrix} 1.558 & 1.951 & -0.861 & -0.846 \\ 3.052 & -0.203 & -1.901 & 2.333 \\ 0.359 & -1.199 & -1.531 & -0.334 \\ 3.016 & 0.946 & 0.290 & 0.156 \end{bmatrix}$$

c. SiLU

Aktivasi SiLU merupakan fungsi aktivasi non-linear yang digunakan untuk meningkatkan kemampuan model dalam mengekstraksi fitur secara lebih efektif. SiLU bekerja dengan cara mengalikan nilai input dengan nilai sigmoid dari input tersebut, sehingga menghasilkan aktivasi yang lebih halus dan stabil dibandingkan fungsi aktivasi tradisional seperti ReLU. Pada penelitian ini, perhitungan nilai SiLU dilakukan menggunakan output hasil matriks *Batch Normalization* serta Persamaan 2.6. Berikut merupakan contoh perhitungan fungsi SiLU pada elemen sel matriks pertama setelah melalui proses normalisasi.

$$SiLU(1,558) = 1,558 \left(\frac{1}{1 + e^{-1,558}} \right)$$

$$SiLU(1,558) = 1,287$$

Proses ini diulang sampai pada element sel terakhir pada matriks sehingga menghasilkan matriks SILU berikut

$$\begin{bmatrix} 1.287 & 1.708 & -0.256 & -0.254 \\ 2.925 & 0.112 & -0.256 & 2.125 \\ 0.212 & -0.277 & -0.272 & 0.195 \\ 2.874 & 0.681 & 0.166 & 0.084 \end{bmatrix}$$

d. MaxPool2D

MaxPooling merupakan proses yang digunakan untuk mengecilkan ukuran citra atau *feature map* sambil mempertahankan informasi yang paling penting di dalamnya. Operasi ini bekerja dengan mengambil nilai terbesar dari setiap region berukuran 2×2 pada *feature map*. Region tersebut kemudian digeser secara berkala ke seluruh elemen matriks sesuai dengan ukuran stride yang digunakan. Dengan melakukan reduksi ukuran seperti ini, MaxPooling tidak hanya mengurangi jumlah komputasi pada lapisan selanjutnya, tetapi juga membantu model menjadi lebih robust terhadap variasi posisi dan perubahan kecil pada citra. Ilustrasi proses MaxPool2D dapat dilihat pada Gambar 3.12.

1,287	1,708	-0,256	-0,254
2,925	0,112	-0,256	2,125
0,212	-0,277	-0,272	0,195
2,874	0,681	0,166	0,084

=

2,925	1,708	2,125
2,925	0,112	2,125
2,874	0,681	0,195

Gambar 3.12 Ilustrasi Proses MaxPool2D

e. Upsample

Upsample merupakan kebalikan dari operasi MaxPooling, yaitu proses untuk meningkatkan ukuran citra atau *feature map* sehingga menghasilkan resolusi yang lebih tinggi. Operasi ini umum digunakan pada arsitektur model deteksi dan segmentasi untuk mengembalikan informasi spasial yang hilang setelah proses downsampling. Pada tahap ini, setiap elemen pada *feature map* diperbesar dengan cara mereplikasi nilai piksel ke area yang lebih luas sesuai dengan faktor skala yang digunakan. Dengan demikian, *feature map* yang dihasilkan memiliki ukuran yang lebih besar dan mampu mempertahankan pola penting dari lapisan sebelumnya. Ilustrasi proses upsample dapat dilihat pada Gambar 3.13.

2,925	1,708	2,125
2,925	0,112	2,125
2,874	0,681	0,195

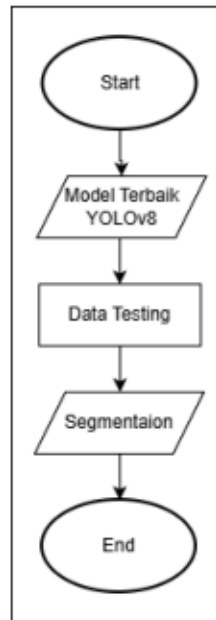
=

2,925	2,925	1,708	1,708	2,125	2,125
2,925	2,925	1,708	1,708	2,125	2,125
2,925	2,925	0,112	0,112	2,125	2,125
2,925	2,925	0,112	0,112	2,125	2,125
2,874	2,874	0,681	0,681	0,195	0,195
2,874	2,874	0,681	0,681	0,195	0,195

Gambar 3.13 Ilustrasi Proses Upsample

3.4 Proses Lokalisasi Makanan

Proses lokalisasi makanan merupakan tahapan inti dalam sistem deteksi dan pengelompokan objek pada skenario *self-service* prasmanan. Tahapan ini bertujuan untuk menentukan posisi spasial dari setiap objek makanan yang berhasil dideteksi oleh model, sehingga sistem dapat mengidentifikasi letak objek secara akurat dalam suatu area pengamatan. Informasi posisi ini menjadi dasar untuk proses pengelompokan, perhitungan jumlah, serta penentuan harga berdasarkan wadah atau area tertentu. Alur proses lokalisasi makanan secara keseluruhan ditunjukkan pada Gambar 3.14.



Gambar 3.14 Proses Lokalisasi Makanan

Langkah awal dalam proses lokalisasi makanan dimulai dari pemilihan model YOLOv8-seg terbaik yang telah diperoleh pada tahap pelatihan. Selanjutnya, pengguna memberikan masukan berupa gambar atau video *real-time* sebagai data uji ke dalam model tersebut. Model YOLOv8-seg digunakan untuk mendeteksi keberadaan objek makanan dan wadah pada setiap *frame* yang ditangkap oleh kamera. Setiap deteksi menghasilkan informasi berupa *bounding box*, kelas objek, serta nilai kepercayaan (*confidence score*). Informasi ini kemudian diproses untuk menentukan posisi awal setiap objek makanan di dalam ruang dua dimensi citra.

Model YOLOv8-seg menghasilkan empat komponen koordinat *bounding box*, yaitu x_{min} , y_{min} , x_{max} , dan y_{max} . Sebagai contoh, koordinat (260, 340, 340, 420) merepresentasikan area persegi panjang yang membatasi objek makanan secara akurat pada gambar. Selain itu, titik pusat (*centroid*) dari *bounding box* dihitung menggunakan persamaan:

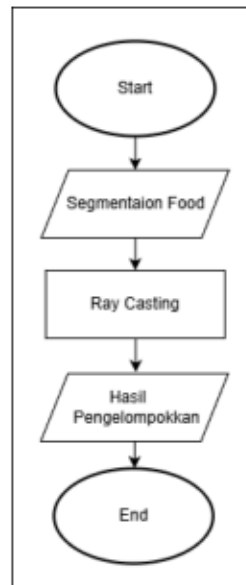
$$x_c = \frac{x_{min} + x_{max}}{2} = \frac{260 + 340}{2} = 300$$
$$y_c = \frac{y_{min} + y_{max}}{2} = \frac{340 + 420}{2} = 380$$

Titik pusat ini digunakan sebagai representasi posisi objek yang lebih ringkas dalam proses penentuan hubungan dengan wadah. Dengan demikian, output

YOLOv8-seg tidak hanya berupa identifikasi objek, tetapi sekaligus informasi koordinat yang diperlukan untuk proses lokalisasi spasial berikutnya.

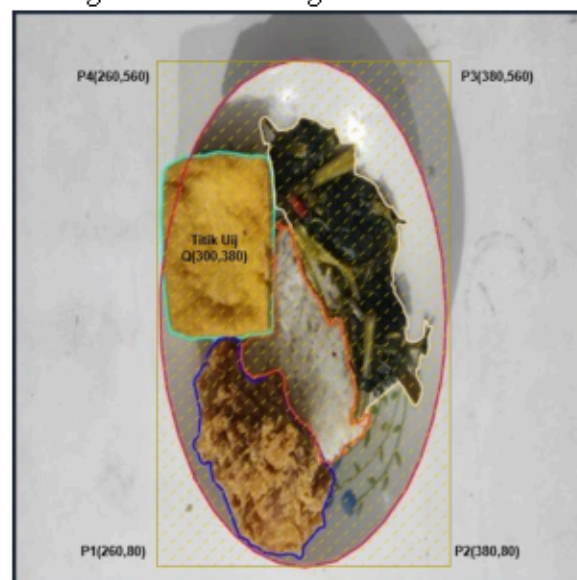
3.5 Proses Pengelompokkan Makanan

Setelah koordinat pusat objek makanan diperoleh, langkah selanjutnya adalah menentukan apakah objek tersebut berada di dalam atau di luar area wadah. Tahap ini dilakukan menggunakan algoritma Ray Casting dengan alur proses seperti gambar 3.15 berikut.



Gambar 3.15 Proses Pengelompokkan Makanan

Pada penelitian ini, poligon yang dimaksud adalah bentuk batas wadah atau wadah yang dihasilkan oleh model segmentasi atau mask dari YOLOv8-seg. Titik pusat bounding box makanan digunakan sebagai *query point*. Algoritma Ray Casting kemudian menarik sebuah garis imajiner (*ray*) dari titik tersebut menuju salah satu arah. Dimisalkan hasil segmentasi sesuai gambar 3.16 berikut.



Gambar 3.16 Hasil Segmentasi

Dalam gambar 3.16 diatas dapat kita lihat bahwa hasil segmentasi berupa poligon dengan sisi sesuai dengan persamaan 2.10. Dalam gambar tersebut nilai titik tengah objek nasi, ayam crispy, tumis kangkung dan tahu memiliki nilai (x_c, y_c) yang berbeda beda sesuai dengan perhitungan pada sub bab 3.4, Perhitungan ray casting algorithm melibatkan mask segmentation wadah dengan titik tengah bounding box makanan.

Untuk mengilustrasikan cara kerja metode Ray Casting dalam menentukan apakah suatu titik berada di dalam atau di luar suatu poligon, digunakan contoh poligon berbentuk segiempat dengan koordinat sebagai berikut $P_1(260,80)$, $P_2(380,80)$, $P_3(380,560)$, $P_4(260,560)$. Titik uji yang akan diuji posisinya Adalah objek tahu $Q(300,380)$. Perhitungan berikut menunjukkan bagaimana proses penentuan jumlah perpotongan dilakukan pada setiap sisi poligon.

1. Sisi 1: $P_1(260,80) \rightarrow P_2(380,80)$

Cek apakah garis sisi melintasi sumbu y menggunakan persamaan 2.7:

$$(80 > 380) \neq (80 > 380) \Rightarrow \text{false} \neq \text{false} \Rightarrow \text{false}$$

Maka tidak terjadi perpotongan.

2. Sisi 2: $P_2(380,80) \rightarrow P_3(380,560)$

Cek apakah garis sisi melintasi sumbu y menggunakan persamaan 2.7:

$$(80 > 380) \neq (560 > 380) \Rightarrow \text{false} \neq \text{true} \Rightarrow \text{true}$$

Karena kondisi terpenuhi, maka terdapat perpotongan. Titik potong pada sumbu x dihitung menggunakan rumus interpolasi pada persamaan 2.8:

$$x_{\text{int}} = 380 + (380 - 80) \cdot \frac{380 - 80}{560 - 80} = 380$$

Bandingkan dengan $x_Q = 300$ menggunakan persamaan 2.9:

$$380 > 300$$

Maka terjadi satu perpotongan.

3. Sisi 3: $P_3(380,560) \rightarrow P_4(260,560)$

Cek apakah garis sisi melintasi sumbu y menggunakan persamaan 2.7:

$$(560 > 380) \neq (560 > 380) \Rightarrow \text{false} \neq \text{false} \Rightarrow \text{false}$$

Maka tidak ada perpotongan.

4. Sisi 4: $P_4(260,560) \rightarrow P_1(260,80)$

Cek apakah garis sisi melintasi sumbu y menggunakan persamaan 2.7:

$$(560 > 380) \neq (80 > 380) \Rightarrow \text{true} \neq \text{false} \Rightarrow \text{true}$$

Karena kondisi terpenuhi, maka terdapat perpotongan. Titik potong pada sumbu x dihitung menggunakan rumus interpolasi pada persamaan 2.8:

$$x_{\text{int}} = 260 + (380 - 560) \cdot \frac{260 - 260}{80 - (-5600)} = 260$$

Bandingkan dengan $x_Q = 300$ menggunakan persamaan 2.9:

$$260 > 300 \Rightarrow \text{false}$$

Maka tidak ada perpotongan.

Dengan Demikian nilai k dapat dihitung menggunakan persamaan 2.10

$$\begin{aligned}
k &= \sum_{i=1}^n 1 \left((y_i > y_q) \neq (y_{i+1} > y_q) \wedge x_{int} > x_q \right) \\
&= 1(0) + 1(1) + 1(0) + 1(0) \\
&= 1
\end{aligned}$$

Berdasarkan hasil perhitungan menggunakan algoritma Ray Casting (Even–Odd Rule), titik $Q(4,3)$ memiliki jumlah perpotongan $k = 1$ dengan sisi-sisi poligon. Karena nilai ini bersifat ganjil, maka titik tersebut berada di dalam poligon. Hasil ini konsisten dengan definisi formal algoritma bahwa jumlah perpotongan ganjil menunjukkan posisi didalam, sedangkan jumlah perpotongan genap menunjukkan posisi diluar.

3.6 Evaluasi Model

Tahap evaluasi model merupakan proses penting untuk mengukur performa sistem deteksi makanan dan wadah yang telah dikembangkan menggunakan YOLOv8. Evaluasi ini bertujuan untuk memastikan bahwa model mampu menghasilkan deteksi yang konsisten, akurat, dan dapat diandalkan ketika diterapkan pada kondisi nyata di lingkungan *self-service* prasmanan. Metode evaluasi yang digunakan dalam penelitian ini adalah confusion matrix, yang berfungsi untuk menggambarkan perbandingan antara hasil prediksi model dengan kondisi sebenarnya. Melalui confusion matrix, nilai recall, precision, dan F1-score dapat dihitung sehingga memberikan gambaran menyeluruh mengenai kualitas kinerja model. Contoh format tabel confusion matrix yang digunakan dalam proses evaluasi disajikan pada tabel 3.3 berikut.

Table 3.3 Confussion Matrix

Prediksi / Aktual	Positive(P)	Negative(N)
Positive(P)	TP	FP
Negative(N)	FN	TN

3.7 Metodologi Pengembangan Sistem

Pada penelitian ini, proses pengembangan sistem dilakukan menggunakan metodologi Waterfall, yaitu salah satu model pengembangan perangkat lunak yang memiliki alur kerja terstruktur, sistematis, dan bersifat sekuensial. Setiap tahapan dalam metode ini harus diselesaikan terlebih dahulu sebelum melanjutkan ke tahap berikutnya. Metode Waterfall pada penelitian ini terdiri dari beberapa tahapan utama, yaitu Analisis Kebutuhan, Desain Sistem, Implementasi, Pengujian, serta Deploy dan Pemeliharaan, yang secara keseluruhan membentuk alur kerja pengembangan sistem secara teratur dan mudah dikendalikan.

3.7.1 Analisis Kebutuhan

Analisis kebutuhan merupakan tahap penting untuk memastikan bahwa sistem yang dibangun sesuai dengan tujuan, fungsi, dan lingkungan operasional. Pada penelitian ini, analisis kebutuhan dibagi menjadi dua kategori utama: kebutuhan fungsional dan kebutuhan perangkat.

1. Kebutuhan Fungsional

Kebutuhan fungsional Adalah kebutuhan yang harus disediakan dan dikembangkan untuk memenuhi tujuan deteksi makanan. Berikut kebutuhan system yang perlu disediakan.

- Sistem dapat menerima inputan gambar ataupun kamera real time
- Sistem dapat menampilkan bounding box dan mask hasil deteksi
- Sistem dapat mengelompokkan makanan dan menampilkan harga

2. Kebutuhan Perangkat

Kebutuhan perangkat adalah analisis teknis perangkat lunak dan keras yang dibutuhkan dalam membangun sistem ini. Berikut kebutuhan perangkat pada system ini.

a) Analisis Perangkat Lunak (Software)

analisis perangkat lunak adalah proses untuk mengidentifikasi dan menentukan seluruh kebutuhan perangkat lunak yang diperlukan agar sistem dapat berjalan dengan baik. Analisis perangkat lunak yang diperlukan dalam penelitian ini dapat dilihat pada tabel 3.4 berikut :

Table 3.4 Analisis Perangkat Lunak

No	Perangkat Lunak	Keterangan
1	Windows 11	Sistem Operasi Laptop
2	Python	Bahasa Pemrograman
3	React	Framework Frontend
4	Draw io	Flowchart
5	Visual Studio Code	Kode editor
6	Google Colab	Notebook Environment
7	Google Chrome	Web Browser

b) Analisis Perangkat Keras (Hardware)

Analisis perangkat keras adalah proses untuk mengidentifikasi kebutuhan fisik berupa alat dan komponen elektronik yang dibutuhkan agar sistem dapat dijalankan dan diuji. Analisis perangkat keras yang diperlukan dalam penelitian ini dapat dilihat pada tabel 3.5 berikut

Table 3.5 Analisis Perangkat Keras

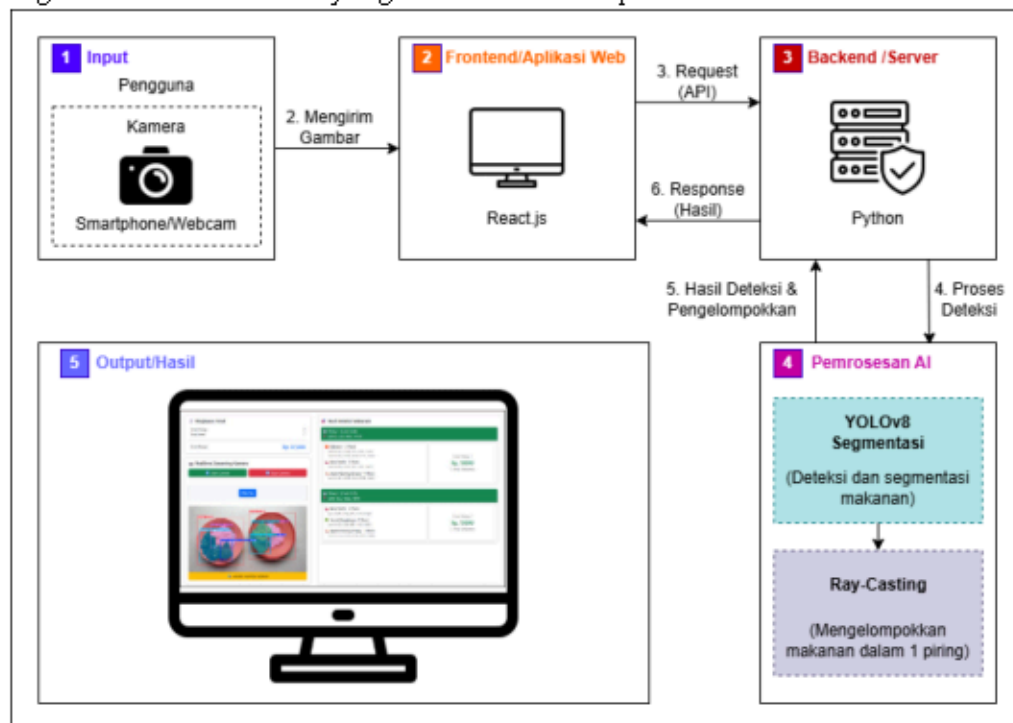
No	Jenis Perangkat	Perangkat Keras	Keterangan
1	Laptop	Processor	Ryzen 7 4800H
2		RAM	8GB
3		SSD	512GB
4		Perangkat I/O	Keyboard/Mouse
5	Kamera	EYD PC03	3840 x 2160p

3.7.2 Desain Sistem

Pada tahap perancangan desain sistem, proses dilakukan untuk menyusun arsitektur secara terstruktur guna meminimalkan potensi kesalahan yang dapat terjadi pada tahap implementasi. Perancangan komponen ini bertujuan membentuk alur kerja sistem deteksi makanan dan proses pengelompokkannya agar berjalan secara efektif dan efisien. Secara umum, perancangan sistem ini dilakukan melalui dua tahapan utama, yaitu perancangan arsitektur sistem dan perancangan antarmuka pengguna. Kedua tahapan tersebut berperan penting dalam memastikan sistem memiliki struktur yang jelas, mudah diimplementasikan, serta mampu memberikan pengalaman penggunaan yang optimal.

1. Perancangan Arsitektur Sistem

Perancangan arsitektur pada penelitian ini dilakukan untuk menggambarkan alur proses deteksi dan pengelompokan makanan yang dijalankan oleh sistem. Arsitektur tersebut disusun untuk menunjukkan hubungan antara input gambar dari pengguna, proses pengolahan oleh sistem, hingga output yang dihasilkan. Dengan demikian, diagram arsitektur berfungsi sebagai representasi visual dari seluruh alur kerja sistem secara menyeluruh. Gambar 3.17 menyajikan ilustrasi lengkap mengenai arsitektur sistem yang diusulkan dalam penelitian ini.



Gambar 3.17 Arsitektur Sistem

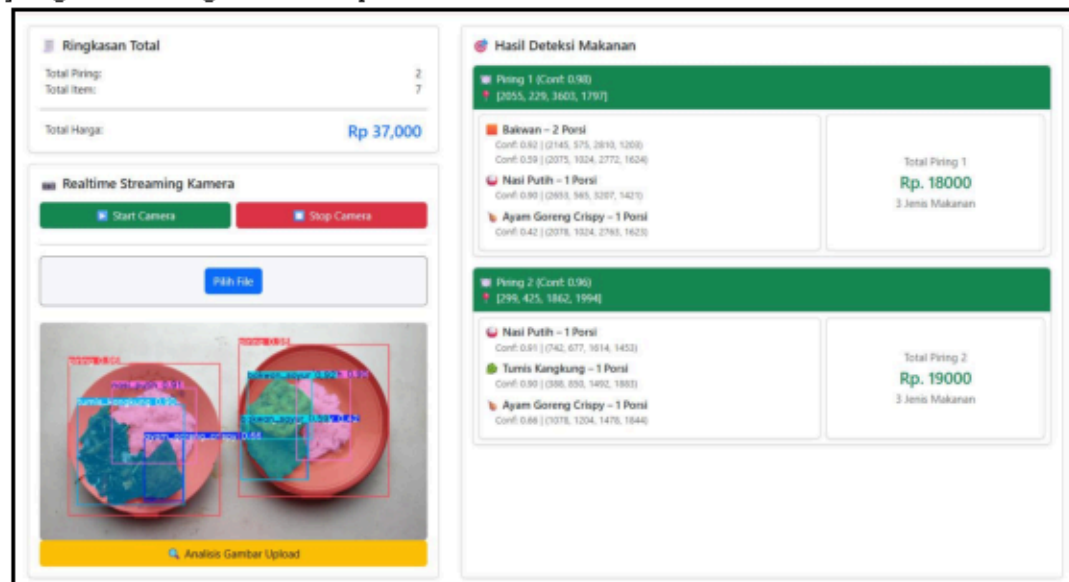
Berdasarkan Gambar Arsitektur Sistem, proses dimulai dari pengguna yang melakukan input data berupa gambar atau video secara real-time menggunakan kamera webcam. Data visual tersebut kemudian dikirimkan ke aplikasi web frontend yang dibangun menggunakan framework React.js sebagai antarmuka utama sistem. Frontend berfungsi untuk menerima input pengguna, menampilkan streaming kamera, serta mengirimkan data gambar ke backend server melalui mekanisme Application Programming Interface (API).

Selanjutnya, backend server yang dikembangkan menggunakan Python menerima request dari frontend dan melakukan pemrosesan citra menggunakan model YOLOv8-seg. Pada tahap ini sistem melakukan deteksi dan segmentasi objek makanan sehingga menghasilkan informasi berupa label kelas makanan, bounding box, mask segmentasi, koordinat objek, serta nilai confidence score. Hasil deteksi tersebut kemudian diproses lebih lanjut menggunakan metode Ray Casting untuk menentukan posisi objek terhadap area piring dan mengelompokkan makanan yang berada pada wadah atau piring yang sama.

Setelah seluruh proses selesai, backend mengirimkan hasil pengolahan kembali ke frontend dalam bentuk response data. Frontend kemudian menampilkan informasi deteksi kepada pengguna, seperti jumlah piring, jenis makanan yang terdeteksi, jumlah tiap makanan, serta total harga secara otomatis. Dengan arsitektur ini, sistem mampu melakukan proses identifikasi makanan dan pengelompokan objek secara terintegrasi melalui antarmuka web berbasis real-time.

2. Perancangan Antarmuka Pengguna

Perancangan antarmuka pengguna dilakukan untuk memastikan bahwa sistem dapat beroperasi dengan mudah, intuitif, dan efektif bagi pengguna akhir. Tahap ini bertujuan untuk merancang tampilan dan alur interaksi yang memudahkan pengguna dalam melakukan proses input, melihat hasil deteksi, serta memahami informasi yang disajikan sistem. Gambar 3.18 menunjukkan rancangan antarmuka pengguna yang dikembangkan dalam penelitian ini.



Gambar 3.18 Rancangan Antarmuka Pengguna

3.7.3 Implementasi

Tahap implementasi merupakan proses menerjemahkan hasil perancangan ke dalam bentuk kode program yang dapat dijalankan. Pada penelitian ini, implementasi dilakukan melalui beberapa langkah utama. Pertama, model YOLOv8-seg dilatih menggunakan dataset makanan dan wadah yang telah dianotasi secara manual, dengan memanfaatkan bahasa pemrograman Python dan framework Ultralytics. Selanjutnya, algoritma Ray Casting diimplementasikan untuk menentukan apakah

titik pusat objek makanan berada di dalam area poligon yang merepresentasikan wadah, sehingga proses pengelompokan objek dapat dilakukan secara akurat. Terakhir, aplikasi utama dibangun menggunakan Python dan React, dengan bantuan kode editor Visual Studio Code, untuk menampilkan hasil deteksi dan segmentasi secara real-time kepada pengguna.

3.7.4 Pengujian

Pengujian sistem dilakukan secara bertahap untuk memastikan bahwa seluruh fungsi dapat berjalan sesuai dengan kebutuhan yang telah ditentukan. Metode pengujian yang digunakan adalah black box testing, yaitu metode pengujian yang berfokus pada fungsi dan keluaran program tanpa memperhatikan kode atau proses internal yang digunakan untuk menghasilkan keluaran tersebut. Melalui pengujian ini, setiap fitur diuji berdasarkan input dan output yang diharapkan. Apabila ditemukan bug atau ketidaksesuaian dalam fungsionalitas sistem, maka dilakukan perbaikan agar sistem dapat berjalan dengan optimal. Rincian pengujian black box testing yang digunakan dalam penelitian ini disajikan pada Tabel 3.6 berikut.

Table 3.6 Pengujian Black Box Testing

No	Test Case	Parameter Uji	Hasil
1	Pengguna dapat melakukan input gambar ataupun video secara real time kedalam system	Pengguna berhasil melakukan input gambar	
2	Pengguna dapat melihat hasil dari proses pengelompokan makanan dalam tampilan system tersebut	Pengguna berhasil melihat hasil pengelompokan makanan	

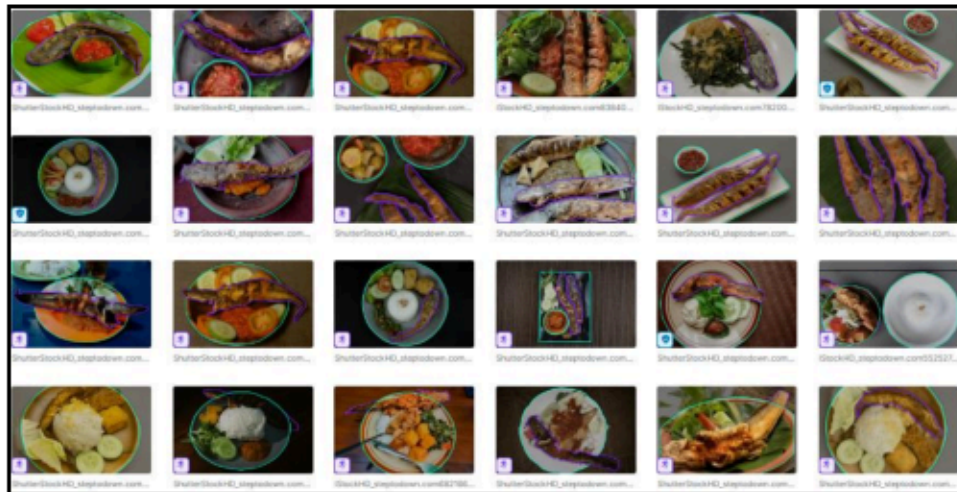
BAB IV HASIL PENGUJIAN DAN PEMBAHASAN

4.1 Hasil

Pada bagian ini akan dijelaskan mengenai bagian setiap proses dari penelitian yang dilakukan. Hasil akan disusun sesuai dengan tahapan pada yang telah dijabarkan pada metodologi penelitian.

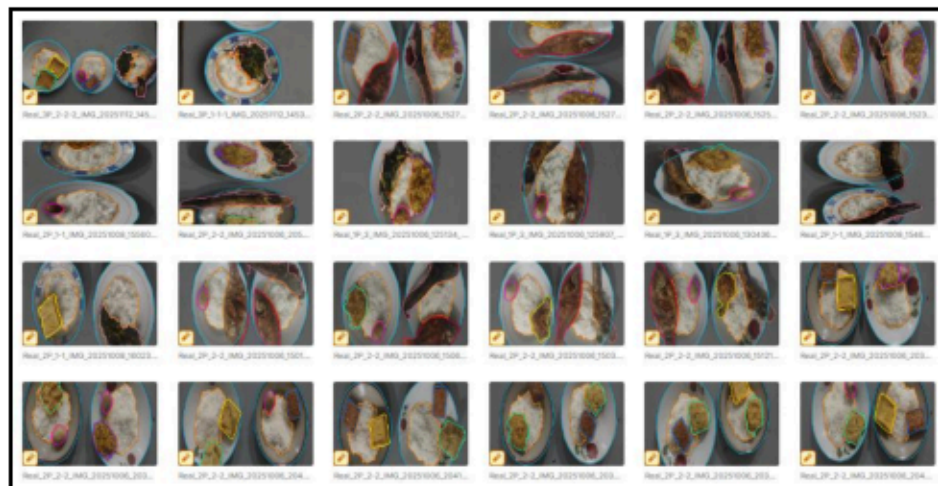
4.1.1 Pengumpulan Data

Dalam penelitian ini data yang digunakan yakni dataset untuk pre-trained serta dataset untuk fine tuning. Data yang digunakan dalam model pre-trained terdiri dari data sekunder yang didapatkan dalam website shutterstock dan istock dengan jumlah 4068 gambar. Pengumpulan data ini dapat dilihat pada gambar 4.1 berikut.



Gambar 4.1 Pengumpulan data model pre trained

Sedangkan untuk data untuk model fine tuning digunakan data dari data primer yang didapatkan dari hasil pengambilan gambar menggunakan kamera xiaomi 12 lite dengan simulasi sama seperti studi kasus yang diharapkan dengan jumlah 3264 gambar. Berikut data testing yang diambil dapat dilihat pada gambar 4.2 berikut.



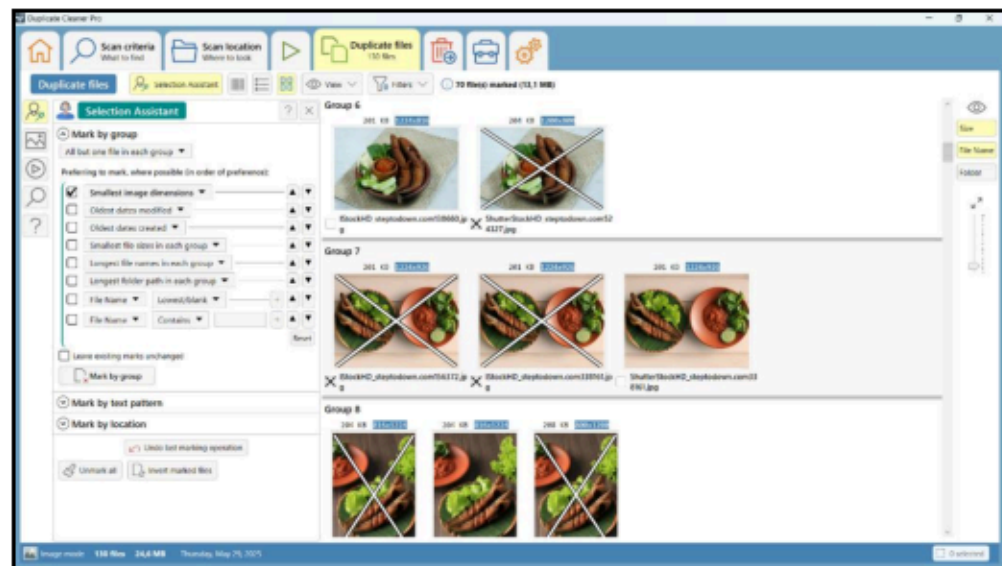
Gambar 4.2 Pengumpulan data model fine tuning

4.1.2 Data Preprocessing

Setelah data berhasil dikumpulkan, tahap berikutnya adalah *data preprocessing* yang bertujuan untuk mengoptimalkan kualitas data sebelum digunakan dalam proses pelatihan model. Tahapan preprocessing yang dilakukan pada penelitian ini meliputi beberapa langkah berikut:

A. Cleaning Data

Pada dataset *pre-trained*, dilakukan proses pembersihan data untuk menghilangkan data yang buruk, tidak sesuai kategori, maupun data duplikat yang dapat menurunkan akurasi model. Penghapusan duplikasi gambar dilakukan secara otomatis menggunakan *Duplicate Cleaner*. Jumlah data awal pada dataset *pre-trained* adalah 5.211 gambar, dan setelah proses pembersihan diperoleh 4.068 gambar. Proses penghapusan duplikasi gambar ditunjukkan pada Gambar 4.3.

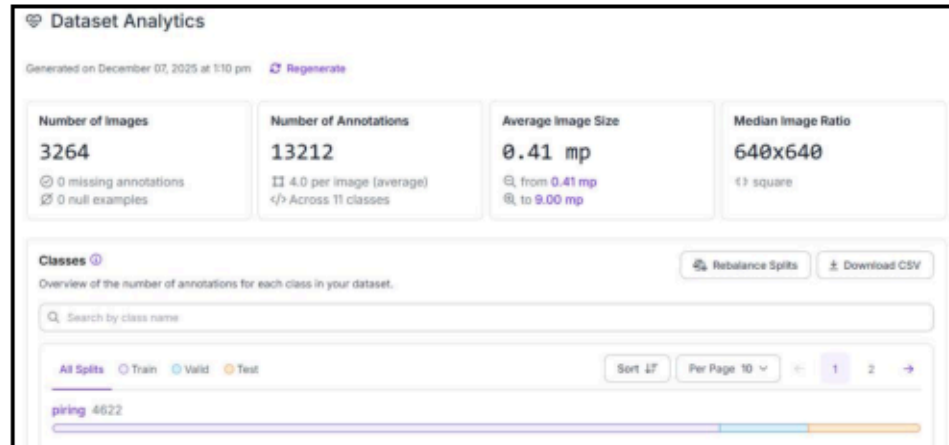


Gambar 4.3 Cleaning Data

B. Anotating Data

Tahap anotasi dilakukan untuk memberikan label pada setiap objek makanan yang ada dalam citra, seperti nasi putih, lele goreng, ikan mujair, bakwan, mendoan, tahu, tempe, telur bacem, dan tumis kangkung. Proses ini bertujuan agar model YOLOv8-seg dapat mengenali posisi dan area objek secara tepat berdasarkan batas piksel yang telah ditandai. Setiap objek akan menghasilkan *segmentation mask* dengan warna unik sesuai kelasnya.

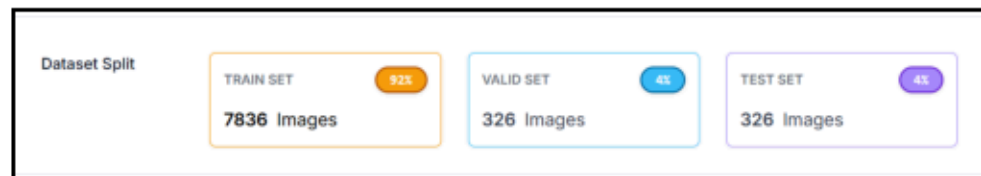
Pada penelitian ini, proses anotasi dilakukan secara manual menggunakan Roboflow, yang mempermudah pelabelan dataset dalam jumlah besar. Tahap anotasi membutuhkan tingkat ketelitian yang tinggi karena hasil anotasi akan menjadi acuan utama model dalam mempelajari objek yang akan dideteksi. Pada dataset *fine-tuning*, terdapat 3.264 gambar dengan total 13.212 anotasi. Analytics dataset *fine-tuning* dari proses anotasi di Roboflow dapat dilihat pada Gambar 4.4



Gambar 4.4 Dataset Analitys

C. Splitting Data

Tahap *splitting data* dilakukan untuk membagi dataset ke dalam tiga subset, yaitu data training, validation, dan testing. Pembagian dilakukan dengan rasio 80% untuk training, 10% untuk validation, dan 10% untuk testing. Data training digunakan untuk melatih model, data validation digunakan untuk mengevaluasi model saat pelatihan berlangsung, sedangkan data testing digunakan untuk pengujian akhir model. Pembagian dataset secara rinci dapat dilihat pada Gambar 4.5.



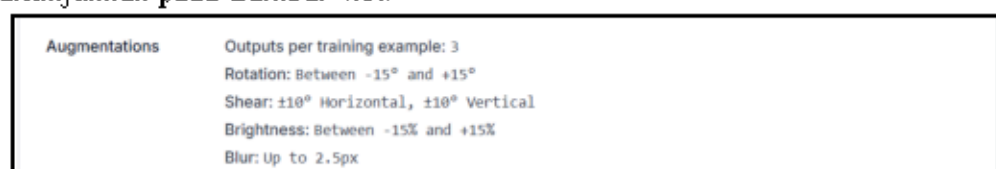
Gambar 4.5 Pembagian Dataset

D. Resize Data

Tahap *resize data* dilakukan untuk menyeragamkan resolusi seluruh gambar agar memiliki ukuran tetap. Langkah ini penting karena YOLO membutuhkan input dengan resolusi yang konsisten untuk mendukung kinerja dan akurasi yang optimal. Ukuran *resize* yang digunakan pada penelitian ini adalah 640×640 piksel.

E. Augmentasi Data

Tahap *augmentasi data* dilakukan untuk menambah variasi data melalui berbagai transformasi seperti rotasi, *shear*, perubahan kecerahan, dan *blurring* agar model lebih robust. Jumlah data training awal adalah 2.612 gambar dan setelah proses augmentasi meningkat menjadi 7.836 gambar. Dengan demikian, total dataset *fine-tuning* setelah augmentasi menjadi 8.488 gambar. Proses augmentasi data ditunjukkan pada Gambar 4.6..



Gambar 4.6 Augmentasi Data

4.1.3 Implementasi dan Evaluasi Model

Setelah tahap preprosesing selanjutnya dilakukan tahap pelatihan model YOLOv8-seg-n dengan google colab. Pelatihan pada google colab menggunakan L4-GPU, untuk rata rata model yang dilatih membutuhkan waktu 2 jam dengan tahapan training seperti berikut.

1. Import Library

Tahapan ini memuat proses mempersiapkan seluruh library yang diperlukan untuk menjalankan proses pelatihan. Pada tahap ini, peneliti mengimpor library utama yaitu *Ultralytics*, yang menyediakan fungsi-fungsi untuk melakukan pelatihan model YOLOv8-seg. Selain itu, library ini juga membutuhkan beberapa dependensi pendukung seperti *PyTorch* untuk menjalankan operasi komputasi selama proses training. Pseudocode untuk tahap import library dapat dilihat pada pseudocode 1 berikut.

Pseudocode 1 : Import Library

START

```
# Install library Ultralytics untuk menggunakan YOLOv8
RUN command to install 'ultralytics' library:
- pip install ultralytics
```

```
# Import kelas YOLO dari library Ultralytics
IMPORT YOLO from ultralytics
```

STOP

2. Import Dataset

Tahapan ini memuat proses mempersiapkan dataset yang akan digunakan dalam pelatihan model. Pada tahap ini, dataset didownload secara manual melalui zip. Jika menggunakan Google Colab yakni dengan mengimpor library Roboflow untuk menghubungkan akun Roboflow dengan Google Colab. Setelah itu, digunakan API key untuk mengakses proyek dan mengunduh dataset berdasarkan versi proyek tersebut. Pseudocode proses import dataset dari Roboflow ditampilkan pada pseudocode 2 dan 3 berikut.

Pseudocode 2 : Import Dataset1

START

```
# Install library Roboflow
RUN command:
- pip install roboflow
```

```
# Import kelas Roboflow
IMPORT Roboflow FROM roboflow
```

```
# Inisialisasi koneksi menggunakan API key Roboflow
SET rf = Roboflow(api_key = "YOUR_API_KEY")
# Akses workspace dan project pada Roboflow
SET project = rf.workspace|"deteksi-makanan-bapakg".project|"makanan-prasmanan-domain-gap-frvjt"
```

```
# Pilih versi dataset
SET version = project.version[3]
```

```
Pseudocode 3 : Import Dataset1 Lanjutan
# Download dataset dalam format YOLOv8
SET dataset = version.download|"yolov8"|
```

```
STOP
```

3. Inisialisasi Dataset

Tahapan ini memuat proses menyiapkan berkas *data.yaml* yang berfungsi sebagai panduan bagi model dalam memahami struktur dataset, termasuk lokasi direktori *train*, *val*, *test*, jumlah kelas, serta nama-nama kelas yang digunakan dalam proses pelatihan. Pseudocode untuk proses inisialisasi dataset ditampilkan pada pseudocode 4 berikut.

```
Pseudocode 4 : Inisialisasi Dataset
START
```

```
# Menentukan lokasi dataset
SET train_path = "../train/images"
SET val_path   = "../valid/images"
SET test_path  = "../test/images"
```

```
# Menentukan jumlah kelas (nc)
SET number_of_classes = 11
```

```
# Menentukan daftar nama kelas
SET class_names = [
    'ayam_goreng_crispy',
    'bakwan_sayur',
    'ikan_mujair',
    'lele_goreng',
    'mendoan',
    'nasi_putih',
    'piring',
    'tahu',
    'telur_bacem',
    'tempe',
    'tumis_kangkung'
]
```

```
STOP
```

4. Training Model

Tahapan ini memuat proses pelatihan model yang berfungsi untuk membuat model memahami pola dari dataset yang telah disiapkan sebelumnya. Pada tahap ini, peneliti memuat model YOLOv8-seg sebagai dasar pelatihan, kemudian menjalankan proses training menggunakan parameter tertentu seperti jumlah epoch, ukuran gambar, batch size, serta optimizer. Pseudocode untuk proses training model ditampilkan pada pseudocode 5 dan 6 berikut.

```
Pseudocode 5 : Training Model
```

```
START
```

```
# Memuat model YOLOv8 untuk tugas segmentasi
LOAD model FROM "yolov8_segmentation/weights/best.pt"
```

Pseudocode 6 : Training Model lanjutan

```
# Menjalankan proses training model
CALL model.train WITH PARAMETERS:
    data = "Makanan-Frasmanan-Domain Gap.v31.yolov8/data.yaml"
    epochs = 50                # Jumlah iterasi pelatihan
    imgsz = 640                # Ukuran input image
    batch = 8                  # Ukuran batch per iterasi
    lr0= 0.001,                # Learning rate
    iou = 0.7,                 # iou
    project = "runs/segment"    # Direktori hasil
    name = "Tune-5"            # Nama folder hasil training
    pretrained = True          # Menggunakan bobot pre-trained
    optimizer = "Adam"         # Jenis optimizer
    patience = 20               # Mencegah Overfitting / 20 epoch
END TRAINING

STOP
```

Penelitian ini melakukan serangkaian eksperimen pelatihan model YOLOv8-seg dengan memvariasikan beberapa parameter secara bertahap sebagaimana ditunjukkan pada Tabel 4.1. Pendekatan ini dirancang berdasarkan prinsip kontrol variabel, dimana setiap tahap eksperimen hanya mengubah satu parameter untuk memastikan bahwa pengaruh masing-masing variabel dapat diidentifikasi secara ilmiah dan terukur. Pengujian parameter dapat dilihat pada tabel 4.1 berikut.

Table 4.1 Parameter Eksperimen Tuning

No	Eksperimen	Dataset	Epoch	LR	Batch	IoU NMS
1	Baseline	Sekunder	50	0.01	16	0.7
2	Tuning 1	Primer	50	0.01	16	0.7
3	Tuning 2	Primer	100	0.01	16	0.7
4	Tuning 3	Primer	100	0.001	16	0.7
5	Tuning 4	Primer	50	0.001	16	0.7
6	Tuning 5	Primer	50	0.001	8	0.7
7	Tuning 6	Primer	50	0.001	8	0.3
8	Tuning 7	Primer	50	0.001	8	0.5

Eksperimen dimulai dari tahap baseline yang menggunakan dataset sekunder dengan seluruh parameter default YOLOv8-seg yaitu epoch 50, learning rate 0.01, batch size 16, dan IoU NMS 0.7. Tahap baseline ini berfungsi sebagai titik acuan awal untuk mengukur performa model sebelum dilakukan optimasi apapun. Selanjutnya pada Tuning 1, dilakukan penggantian dataset dari sekunder ke dataset primer tanpa mengubah parameter lainnya, bertujuan untuk membuktikan pengaruh relevansi dataset terhadap performa model secara terisolasi.

Tuning 2 menambahkan epoch dari 50 menjadi 100 untuk memberikan waktu belajar lebih lama kepada model dalam mengekstraksi fitur yang lebih kompleks dari dataset primer. Tuning 3 kemudian menurunkan learning rate dari 0.01 menjadi 0.001 dengan mempertahankan epoch 100, bertujuan agar pembaruan bobot dilakukan dengan langkah yang lebih kecil dan presisi sehingga konvergensi yang dicapai lebih optimal. Tuning 4 kembali menggunakan epoch 50 namun mempertahankan learning rate 0.001. Tuning 5 mengurangi batch size dari 16 menjadi 8 agar pembaruan bobot menjadi lebih variatif dan model lebih mampu melakukan generalisasi. Terakhir,

Tuning 6 dan Tuning 7 memvariasikan nilai IoU NMS masing-masing menjadi 0.3 dan 0.5 untuk menganalisis pengaruh agresivitas NMS terhadap kemampuan model menangani deteksi objek yang tumpang tindih pada wadah prasmanan.

Tabel 4.2 menyajikan hasil evaluasi model pada data pelatihan dan validasi yang mencerminkan kemampuan model dalam mempelajari pola dari data yang telah dilatihkan. Secara umum, seluruh eksperimen tuning menunjukkan peningkatan performa yang signifikan dibandingkan baseline, dengan tren yang konsisten antara metrik box dan metrik mask pada setiap eksperimen. Hasil akurasi pelatihan dan validation dapat dilihat pada tabel 4.2 berikut.

Table 4.2 Hasil Akurasi Pelatihan dan Validation

Uji	Recall	Precision	MAP@ 50	MAP@ 50-95	Recall	Precision	MAP@ 50	MAP@ 50-95
	Box				Mask			
Baseline	87.6%	87.5%	91.7%	80.1%	87.1%	87%	90.9%	77.2%
Tuning 1	98.1%	98.5%	99.4%	94.6%	98.1%	98.5%	99.4%	94.8%
Tuning 2	98.2%	98.8%	99.3%	93.6%	98.3%	98.7%	99.3%	94.4%
Tuning 3	99.1%	99%	99.4%	95.9%	99%	99%	99.4%	95.2%
Tuning 4	98.2%	98.6%	99.4%	95.5%	98.2%	98.6%	99.4%	94.9%
Tuning 5	98.5%	98%	99.3%	95.3%	98.4%	98.1%	99.3%	94.9%
Tuning 6	98.6%	98%	99.4%	95.2%	98.5%	98.1%	99.3%	94.7%
Tuning 7	98.7%	97.8%	99.2%	95%	98.5%	97.9%	99.2%	94.6%

Model baseline yang menggunakan dataset sekunder mencatatkan mAP@50 box sebesar 91.7% dan mAP@50-95 sebesar 80.1% dengan nilai Recall 87.6% dan Precision 87.5%. Hasil ini merupakan yang terendah di antara seluruh eksperimen akibat ketidaksesuaian karakteristik visual dataset sekunder dengan kondisi nyata prasmanan Haji Sumi. Pada Tuning 1, tuning dataset primer menghasilkan lonjakan performa yang signifikan dengan mAP@50 box meningkat menjadi 99.4% dan mAP@50-95 menjadi 94.6%, membuktikan secara empiris bahwa kualitas dan relevansi dataset merupakan faktor paling dominan dalam menentukan performa model YOLOv8-seg.

Tuning 2 dengan penambahan epoch menjadi 100 menghasilkan sedikit penurunan pada mAP@50-95 menjadi 93.6%, mengindikasikan munculnya gejala overfitting ringan. Tuning 3 dengan kombinasi epoch 100 dan learning rate 0.001 berhasil menghasilkan performa validasi terbaik secara keseluruhan dengan Recall 99.1%, Precision 99%, mAP@50 99.4%, dan mAP@50-95 tertinggi sebesar 95.9% pada metrik box, membuktikan bahwa learning rate yang lebih kecil menghasilkan konvergensi yang lebih optimal.

Tuning 4 hingga Tuning 7 secara konsisten mempertahankan nilai mAP@50 di rentang 99.2% hingga 99.4% pada metrik box, menunjukkan bahwa variasi batch size dan IoU NMS tidak memberikan pengaruh signifikan terhadap performa validasi. Seluruh metrik mask juga menunjukkan tren yang konsisten dan sejalan dengan metrik box pada seluruh eksperimen. Untuk mencerminkan kemampuan generalisasi model pada kondisi yang sesungguhnya, selanjutnya dilakukan pengujian menggunakan data testing sebagaimana dibahas pada Tabel 4.3.

Table 4.3 Hasil Pengujian Pada Data Testing

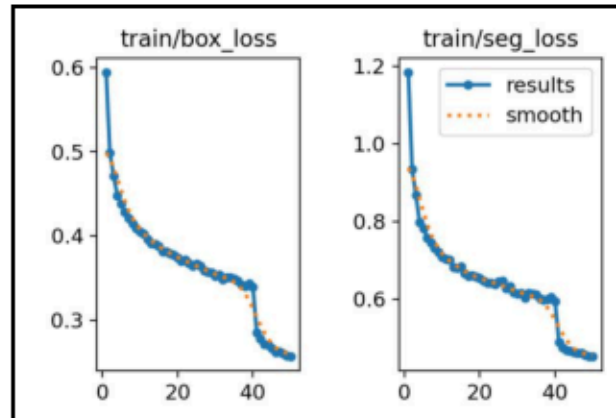
Uji	Recall	Precision	MAP@50	MAP@50-95	Recall	Precision	MAP@50	MAP@50-95
	Box				Mask			
Baseline	51.3%	34%	35.9%	27.5%	46.3%	29.4%	29.5%	18.7%
Tuning 1	81.9%	72.7%	75.7%	54.6%	80.7%	72.6%	74.6%	50.8%
Tuning 2	79%	70.9%	73.1%	52.6%	78.4%	70.3%	72.2%	48.3%
Tuning 3	84.3%	76.2%	79.5%	59.9%	84.4%	75.9%	79.1%	55.8%
Tuning 4	82.8%	79.3%	82.2%	62.8%	83.5%	78.7%	81.8%	58.6%
Tuning 5	83.9%	79.4%	82.6%	62.5%	83.7%	79.6%	82.1%	58.4%
Tuning 6	83.6%	80.7%	82.2%	62%	83.6%	80.7%	82%	58.1%
Tuning 7	83.7%	80.9%	81.3%	61.5%	83.7%	81%	81.4%	57.8%
	83.9%	79.4%	82.6%	62.5%	83.7%	79.6%	82.1%	58.4%

Hasil pengujian Tabel 4.3 secara keseluruhan menunjukkan pola yang berbeda signifikan dibandingkan hasil validasi pada Tabel 4.2. Model baseline mencatatkan mAP@50 box hanya sebesar 35.9% dan mAP@50-95 sebesar 27.5%, jauh lebih rendah dibandingkan hasil validasinya sebesar 91.7% dan 80.1%. Kesenjangan ini membuktikan adanya domain gap yang serius antara dataset sekunder dengan kondisi nyata.

Tuning 1 berhasil meningkatkan mAP@50 box testing menjadi 75.7% hanya dengan penggantian dataset, namun Tuning 2 mengalami penurunan menjadi 73.1% akibat overfitting dari penambahan epoch. Tuning 3 berhasil meningkatkan mAP@50 menjadi 79.5% melalui penurunan learning rate ke 0.001, yang terbukti efektif meningkatkan kemampuan generalisasi model. Tuning 4 dengan pengembalian epoch ke 50 menghasilkan mAP@50 sebesar 82.2%, dan Tuning 5 dengan pengurangan batch size menjadi 8 menghasilkan mAP@50 box testing tertinggi sebesar 82.6% dengan mAP@50-95 sebesar 62.5%, membuktikan bahwa batch size yang lebih kecil meningkatkan kemampuan generalisasi model karena pembaruan bobot dilakukan berdasarkan subset data yang lebih beragam.

Tuning 6 dengan IoU NMS 0.3 dan Tuning 7 dengan IoU NMS 0.5 menunjukkan peningkatan nilai Precision masing-masing menjadi 80.7% dan 80.9%, namun nilai mAP@50 keduanya mengalami penurunan menjadi 82.2% dan 81.3%. Fenomena ini mengindikasikan bahwa nilai IoU NMS yang semakin kecil meningkatkan agresivitas penghapusan bounding box duplikat sehingga Precision meningkat, namun berpotensi menghapus deteksi yang valid pada skenario objek makanan yang berdekatan. Oleh karena itu, Tuning 5 dengan IoU NMS default 0.7 ditetapkan sebagai model terbaik karena menghasilkan keseimbangan optimal antara seluruh metrik evaluasi dengan mAP@50 box 82.6% dan mAP@50-95 62.5%.

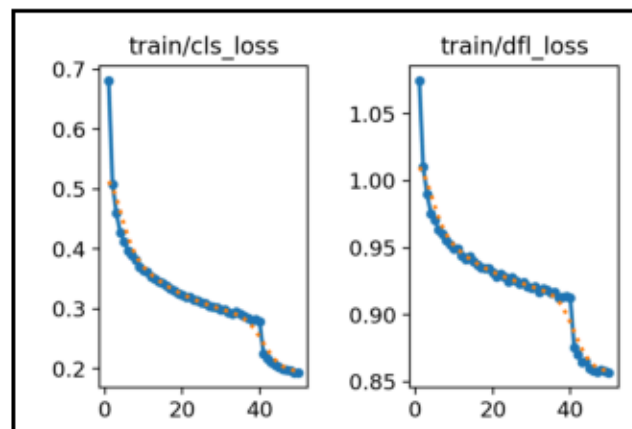
Pada gambar 4.7 dan gambar 4.8 menunjukkan hasil grafik training dari proses fine-tuning model YOLOv8-seg-n. Terdapat empat jenis grafik yang ditampilkan, yaitu train/box_loss, train/seg_loss, train/cls_loss, dan train/df_l_loss. Keempat grafik ini digunakan untuk mengevaluasi performa model selama proses pelatihan dan memastikan bahwa model mengalami peningkatan kemampuan seiring bertambahnya epoch. Grafik training box dan seg loss dapat dilihat pada gambar 4.7.



Gambar 4.7 Grafik Training Box dan Seg Loss

Grafik `train/box_loss` menggambarkan tingkat kesalahan prediksi terkait posisi dan ukuran bounding box. Pada awal pelatihan nilai kesalahan berada di sekitar 0.6, kemudian terus menurun hingga mendekati 0.30 pada epoch ke-50. Penurunan ini menunjukkan bahwa model semakin akurat dalam menentukan bounding box dari objek pada data training.

Grafik `train/seg_loss` menunjukkan tingkat kesalahan pada proses segmentasi mask. Di awal pelatihan, nilai loss berada di sekitar 1.2, kemudian menurun secara konsisten hingga mencapai 0.60. Hal ini menandakan bahwa kemampuan model dalam menghasilkan segmentasi yang sesuai dengan bentuk objek semakin membaik. Sedangkan untuk grafik training class dan dfl loss dapat dilihat pada gambar 4.8 berikut.

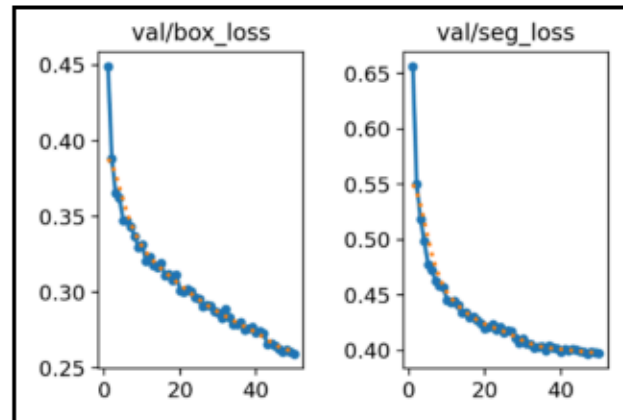


Gambar 4.8 Grafik Training Class dan Dfl Loss

Grafik `train/cls_loss` menggambarkan kesalahan dalam proses klasifikasi objek. Nilai awal loss berada di sekitar 0.70, kemudian menurun hingga mendekati 0.20 pada akhir pelatihan. Penurunan ini menunjukkan bahwa model semakin akurat dalam mengenali dan membedakan kelas objek.

Sedangkan grafik `train/dfl_loss` (Distribution Focal Loss) menggambarkan kesalahan dalam memperkirakan distribusi jarak bounding box pada level yang lebih detail. Nilai awal loss berada pada kisaran 1.05 dan mengalami penurunan hingga sekitar 0.85. Penurunan ini menunjukkan bahwa model semakin baik dalam memprediksi koordinat bounding box dengan lebih presisi.

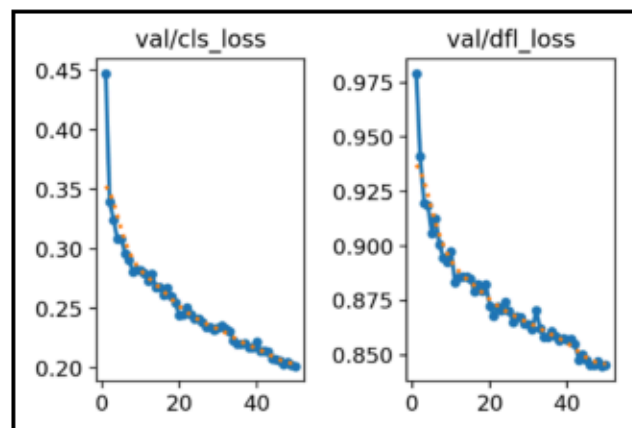
Gambar 4.9 serta Gambar 4.10 menunjukkan hasil grafik validasi dari proses fine-tuning model YOLOv8-seg-n. Grafik-grafik ini digunakan untuk mengevaluasi performa model terhadap data validasi dan memastikan bahwa model tidak hanya belajar dengan baik pada data training, tetapi juga mampu melakukan generalisasi. Terdapat empat grafik yang ditampilkan, yaitu val/box_loss, val/seg_loss, val/cls_loss, dan val/dfl_loss. Grafik validation box dan seg loss dapat dilihat pada gambar 4.9 berikut.



Gambar 4.9 Grafik Validation Box dan Seg Loss

Grafik val/box_loss menggambarkan tingkat kesalahan prediksi posisi dan ukuran bounding box pada data validasi. Pada awal pelatihan, nilai loss berada di sekitar 0.45, kemudian secara konsisten menurun hingga mencapai 0.25 pada epoch ke-50. Penurunan ini menunjukkan bahwa akurasi model dalam memprediksi bounding box pada data validasi semakin meningkat.

Grafik val/seg_loss menunjukkan tingkat kesalahan dalam segmentasi mask pada data validasi. Nilai awalnya berada pada kisaran 0.6, kemudian menurun hingga mendekati 0.4. Hal ini menunjukkan bahwa kemampuan model dalam menghasilkan segmentasi yang sesuai dengan bentuk objek pada data validasi semakin baik dan lebih presisi. Sedangkan untuk grafik validation class dan dfl loss dapat dilihat pada gambar 4.10 berikut.

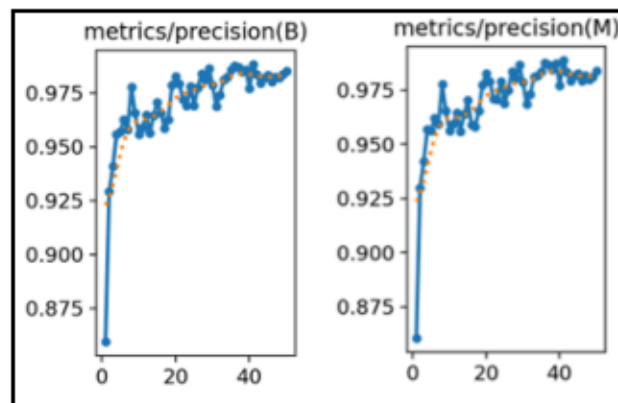


Gambar 4.10 Grafik Validation Class dan Dfl Loss

Grafik `val/cls_loss` menggambarkan kesalahan klasifikasi objek pada data validasi. Loss awal berada di sekitar 0.45 dan menurun hingga mendekati 0,20. Penurunan ini menandakan bahwa model semakin akurat dalam mengidentifikasi kelas objek selama proses validasi.

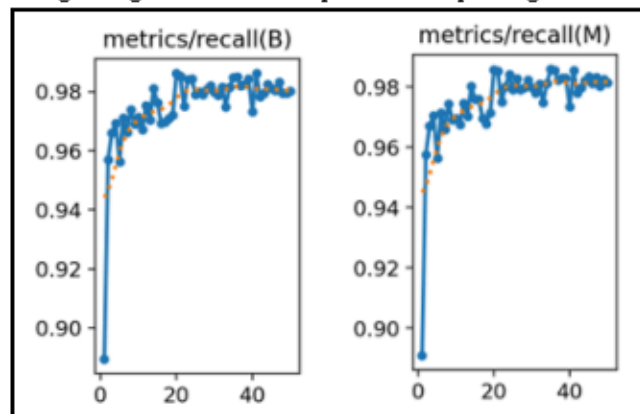
Sementara itu, grafik `val/df_l_loss` (Distribution Focal Loss) menunjukkan tingkat kesalahan dalam memprediksi distribusi koordinat bounding box secara detail. Nilai awal loss berada pada sekitar 0.975 dan menurun secara bertahap hingga mencapai 0.85. Tren penurunan ini menandakan bahwa estimasi koordinat bounding box pada data validasi menjadi semakin presisi.

Gambar 4.11 dan Gambar 4.12 serta menunjukkan hasil pelatihan model fine-tuning YOLOv8-seg-n berupa grafik metrik precision dan recall untuk bounding box (B) dan segmentation mask (M). Grafik ini digunakan untuk mengevaluasi kemampuan model dalam mendeteksi objek dengan benar. Grafik precision dapat dilihat pada gambar 4.11 berikut.



Gambar 4.11 Grafik Precision

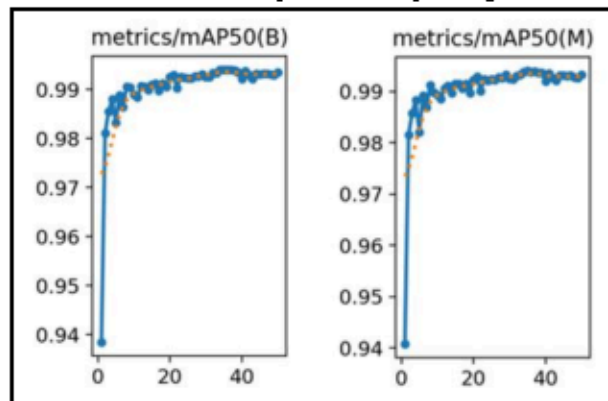
Grafik precision untuk bounding box dan mask menggambarkan proporsi prediksi yang benar dari seluruh objek yang berhasil dideteksi model. Pada awal pelatihan nilai precision berada di kisaran 0.875, kemudian meningkat stabil hingga mendekati 0.975 pada epoch terakhir. Hal ini menunjukkan bahwa model semakin akurat dalam memberikan prediksi yang benar, ditandai dengan menurunnya jumlah false positive. Sedangkan grafik recall dapat dilihat pada gambar 4.12 berikut.



Gambar 4.12 Grafik Recall

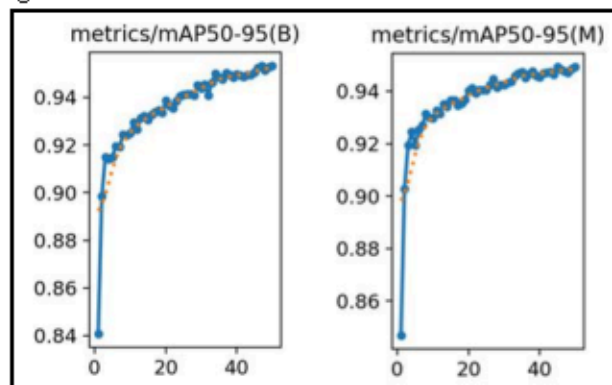
Sementara itu, grafik recall menggambarkan seberapa banyak objek yang berhasil terdeteksi dari seluruh objek yang ada pada ground truth. Nilai awal recall untuk bounding box dan mask berada di sekitar 0.9 dan meningkat hingga mendekati 0.98 pada akhir pelatihan. Hal ini menandakan bahwa model semakin mampu mendeteksi hampir seluruh objek yang seharusnya terdeteksi, dengan menurunnya jumlah false negative.

Gambar 4.13 dan Gambar 4.14 menunjukkan grafik metrik evaluasi performa berupa mean Average Precision (mAP) untuk bounding box (B) dan segmentation mask (M). Grafik ini digunakan untuk menilai tingkat akurasi model dalam mendeteksi objek berdasarkan beberapa ambang batas Intersection over Union (IoU). Grafik mAP 50 untuk box dan mask dapat dilihat pada gambar 4.13 berikut.



Gambar 4.13 Grafik mAP50

Grafik tersebut menunjukkan nilai mAP50, yaitu nilai mAP pada ambang IoU 50%. Pada grafik pertama (mAP50 untuk bounding box), nilai awal berada di sekitar 0.94 dan meningkat stabil hingga mendekati 0.99 pada akhir epoch. Grafik ketiga (mAP50 untuk mask) juga menunjukkan pola peningkatan serupa, dimulai dari sekitar 0.94 dan mencapai mendekati 0.99 pada akhir pelatihan. Hal ini menunjukkan bahwa model mampu mendeteksi objek dengan presisi tinggi ketika menggunakan ambang batas IoU yang relatif longgar (50%). Sedangkan untuk grafik mAP50-95 dapat dilihat pada gambar 4.14 berikut.

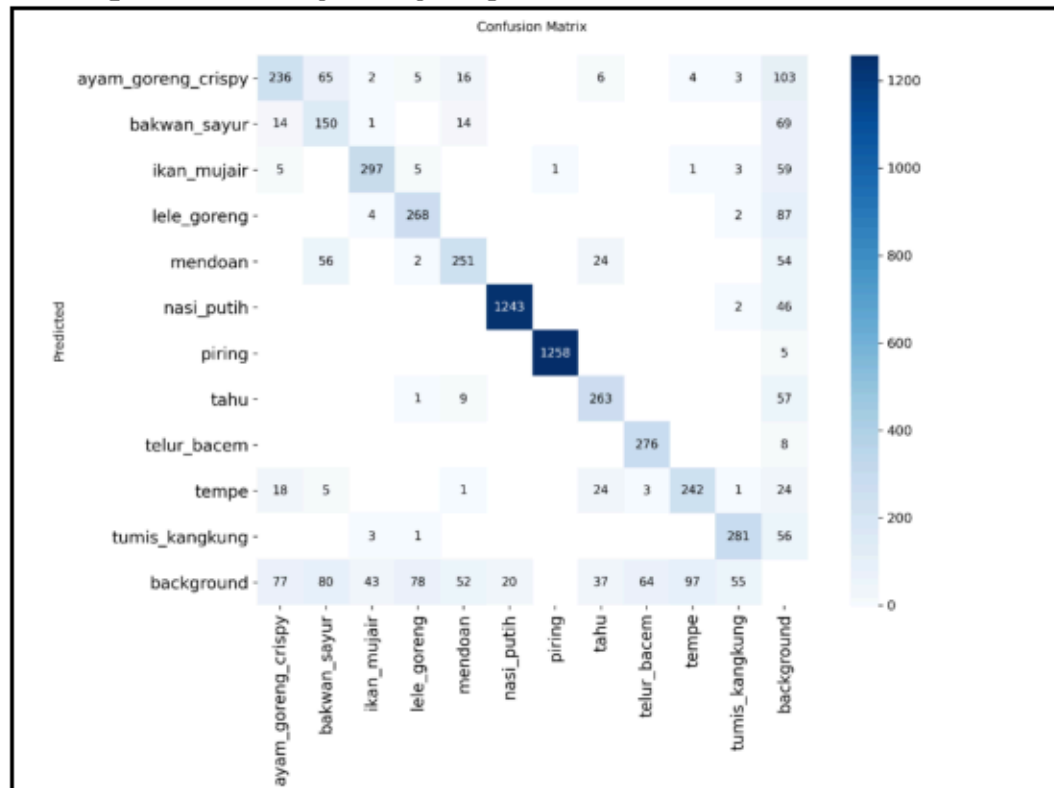


Gambar 4.14 Grafik mAP50-95

Sementara itu, grafik gambar 4.14 menampilkan mAP50-95, yaitu rata-rata mAP pada berbagai ambang IoU dari 50% hingga 95%. Pada grafik kedua, nilai awal berada di sekitar 0.84 dan meningkat hingga mendekati 0.94. Pada grafik keempat,

nilai awal berada pada kisaran 0,86 dan meningkat hingga sekitar 0.94. Peningkatan ini menunjukkan bahwa model tidak hanya akurat pada IoU rendah, tetapi juga tetap konsisten pada ambang IoU yang lebih ketat, sehingga menandakan kemampuan model yang kuat dalam mendeteksi objek secara detail dan presisi.

Tahapan selanjutnya yakni menguji model yang telah dilatih menggunakan data testing. Evaluasi ini menggunakan *confussion matrix*. *confussion matrix* untuk hasil *weight* terbaik ditunjukkan pada gambar 4.15



Gambar 4.15 Confussion Matrix Data Testing

Gambar 4.15 di atas menunjukkan confusion matrix untuk data testing, yang menggambarkan hubungan antara label sebenarnya (true) dan prediksi model (predicted). Secara umum, model menunjukkan performa yang sangat baik, ditunjukkan oleh banyaknya nilai diagonal yang besar pada setiap kelas dibanding nilai vertikal dan horizontalnya, seperti nasi_putih (1243), piring (1258), mendoan (251), ikan_mujair (297), tahu (263), dan telur_bacem (276). Nilai diagonal yang tinggi menandakan bahwa sebagian besar objek berhasil diprediksi dengan benar.

Beberapa kelas seperti ayam_goreng_crispy (236), bakwan_sayur (150), lele_goreng (268), tempe (242), dan tumis_kangkung (281) juga menunjukkan tingkat prediksi benar yang baik, meskipun jumlah data lebih sedikit dibanding class lain.

Terdapat beberapa kesalahan prediksi yang terlihat pada nilai non-diagonal confusion matrix. Sebagai contoh, kelas ayam_goreng_crispy mengalami kesalahan prediksi sebanyak 14 instance yang salah diprediksi sebagai bakwan_sayur. Sebaliknya, kelas bakwan_sayur juga mengalami kesalahan prediksi sebanyak 65 instance yang salah diprediksi sebagai ayam_goreng_crispy. Kelas ikan_mujair salah

diprediksi sebagai lele_goreng sebanyak 4 instance, sedangkan kelas lele_goreng mengalami kesalahan deteksi sebanyak 5 instance yang salah teridentifikasi sebagai ayam_goreng_crispy. Kelas mendoan salah terdeteksi sebagai ayam_goreng_crispy sebanyak 16 instance, dan kelas tahu salah terdeteksi sebagai mendoan sebanyak 24 instance. Selain itu, kelas telur_bacem mengalami kesalahan prediksi sebanyak 3 instance yang salah terdeteksi sebagai tempe, kelas tempe salah terdeteksi sebagai ayam_goreng_crispy sebanyak 4 instance, serta kelas tumis_kangkung salah terdeteksi sebagai ikan_mujair sebanyak 3 instance. Meskipun demikian, jumlah kesalahan tersebut relatif kecil dibandingkan jumlah prediksi benar pada masing-masing kelas, sehingga model secara keseluruhan masih menunjukkan kemampuan klasifikasi yang baik. Untuk performa model tersebut dapat dilihat pada Tabel 4.4 berikut.

Table 4.4 Hasil Pengujian Data Testing

Class	Images	Instances	Precision	Recall	mAP50	mAP50-95
all	360	5669	0.837	0.796	0.821	0.584
ayam_goreng	268	350	0.597	0.723	0.675	0.456
bakwan_sayur	270	356	0.795	0.52	0.651	0.518
ikan_mujair	265	350	0.873	0.911	0.924	0.621
lele_goreng	266	360	0.764	0.744	0.712	0.342
mendoan	265	343	0.699	0.752	0.753	0.581
nasi_putih	355	1263	0.966	0.984	0.978	0.825
piring	360	1259	0.991	0.994	0.993	0.668
tahu	268	354	0.894	0.805	0.881	0.718
telur_bacem	265	343	0.982	0.787	0.897	0.6
tempe	264	344	0.797	0.709	0.783	0.565
tumis_kangkung	263	347	0.849	0.821	0.83	0.529

Hasil tersebut menunjukkan hasil *training* yang diuji dengan data testing menghasilkan nilai mAP50 0.821, nilai mAP50-95 0.584, precision 0.837, dan recall 0.796.

4.1.4 Proses Lokalisasi Makanan & Metode Ray Casting

Model terbaik yang telah dilatih akan disimpan dengan ekstensi ".pt" dan diberi nama "best.pt". Model ini digunakan pada tahap inferensi untuk melakukan proses lokalisasi makanan dan wadah, serta menjadi dasar penerapan metode Ray Casting untuk mengelompokkan makanan di dalam setiap wadah. Pseudocode yang digunakan dalam proses lokalisasi objek dan penerapan metode Ray Casting pada penelitian ini dapat dilihat pada pseudocode 7, 8, dan 9 berikut.

Pseudocode 7 : Proses Lokalisasi Makanan & Metode Ray Casting

START

```
# Memuat model YOLOv8 hasil training
LOAD model FROM "best-fine-tuned.pt"
# Menentukan direktori penyimpanan dan Membuat folder jika belum ada
SET UPLOAD_FOLDER = "uploads/"
SET PREDICT_FOLDER = "predictions/"
CREATE_FOLDER UPLOAD_FOLDER
CREATE_FOLDER PREDICT_FOLDER
```

Pseudocode 8 : Proses Lokalisasi Makanan & Metode Ray Casting Lanjutan

```
# Ketika endpoint /predictplate menerima file gambar
IF REQUEST does NOT contain "file" THEN
    RETURN error "No file uploaded"
END IF

# Menyimpan file input
READ file FROM REQUEST
SET filename = secure(file.name)
SET filepath = UPLOAD_FOLDER + filename
SAVE file TO filepath

# Menjalankan prediksi YOLOv8
CALL model.predict WITH PARAMETERS:
source = filepath
conf = 0.25
STORE OUTPUT IN results

# Inisialisasi list penyimpanan piring
INITIALIZE plates AS EMPTY_LIST

# TAHAP 1: DETEKSI SEMUA PIRING
FOR each result IN results:
    GET boxes FROM result

    FOR each box IN boxes:
        class_id = box.class_id
        class_name = model.names[class_id] ~ LOWERCASE
        conf = box.confidence
        xyxy = box.xyxy_coordinates

        IF class_name IS "piring" THEN

            CREATE plate_polygon FROM xyxy
            # polygon berbentuk persegi dari bbox

            CREATE plate_object WITH:
                id = "Piring " + running_number
                conf = conf
                bbox = xyxy
                polygon = plate_polygon
                foods = EMPTY_LIST

            ADD plate_object TO plates
        END IF
    END FOR
END FOR

# TAHAP 2: DETEKSI SEMUA MAKANAN
FOR each result IN results:
    GET boxes FROM result

    FOR each box IN boxes:
        class_id = box.class_id
        class_name = model.names[class_id] ~ LOWERCASE
        conf = box.confidence
        xyxy = box.xyxy_coordinates
```

Pseudocode 9 : Proses Lokalisasi Makanan & Metode Ray Casting Lanjutan

```
IF class_name IS "piring" THEN
    CONTINUE
END IF

# Hitung titik tengah bounding box makanan
cx = {x1 + x2} / 2
cy = {y1 + y2} / 2

CREATE point FROM {cx, cy}

CREATE food_object WITH:
    name = class_name
    conf = conf
    bbox = xyxy

# Cek makanan masuk ke piring mana
FOR each plate IN plates:
    IF plate.polygon CONTAINS point THEN
        ADD food_object TO plate.foods
        BREAK
    END IF
END FOR

END FOR

# HAPUS POLYGON (TIDAK BISA JSON)
FOR each plate IN plates:
    REMOVE plate.polygon
END FOR

# SIMPAN GAMBAR HASIL PREDIKSI
SET predicted_filename = "predict_" + filename
SET predicted_filepath = PREDICT_FOLDER + predicted_filename

SAVE results[0] TO predicted_filepath

# KIRIM RESPONSE JSON
RETURN:
    image_url = "predictions/" + predicted_filename
    plates = plates

END
```

Pseudocode di atas menjelaskan alur sistem dalam memproses input gambar menggunakan model YOLOv8-seg yang telah dilatih sebelumnya. Proses dimulai dengan memuat model terbaik (best-fine-tuned.pt) serta menyiapkan direktori penyimpanan untuk gambar input dan hasil prediksi. Ketika server menerima permintaan pada endpoint /predictplate, sistem memverifikasi keberadaan file, kemudian menyimpannya ke folder uploads/.

Model YOLOv8-seg kemudian dijalankan untuk melakukan deteksi objek pada gambar tersebut. Hasil prediksi digunakan untuk membentuk tiga struktur data utama, yaitu detected_objects untuk menghitung jumlah objek per kelas, plate_polygons untuk menyimpan representasi area piring berupa polygon, dan plates_content untuk mencatat

jenis serta jumlah makanan pada masing-masing piring. Bounding box yang terklasifikasi sebagai piring diubah menjadi polygon untuk menentukan area wadah.

Setiap objek makanan dihitung titik tengah bounding box-nya, yang kemudian digunakan untuk mengecek apakah objek tersebut berada di dalam salah satu polygon piring. Jika titik tersebut berada dalam polygon tertentu, objek tersebut dicatat sebagai isi piring tersebut. Setelah seluruh objek diproses, sistem menyimpan gambar hasil prediksi ke folder predictions/ dan mengembalikan respons JSON berisi URL gambar hasil prediksi, jumlah objek yang terdeteksi, dan isi masing-masing piring. Pseudocode ini merangkum proses lengkap deteksi, pelokalan objek, dan pengelompokan menggunakan titik tengah objek terhadap polygon piring. Berikut merupakan contoh visualisasi hasil segmentasi mask dari model best-finetuned yang telah dilatih, serta hasil pengelompokan makanan menggunakan metode Ray Casting.

4.1.5 Pengembangan Sistem

Tahap pengembangan sistem bermaksud guna membangun antarmuka yang dapat digunakan pengguna dalam menguji model serta algoritma yang diterapkan. Sistem dikembangkan menggunakan framework react.js. Struktur component dapat dilihat pada pseudocode 10 berikut.

Pseudocode 10 : Struktur Component

```
BEGIN StrukturComponent

  INITIALIZE state plates = null

  DEFINE mapping AS DICTIONARY OF:
    key: "nasi_putih" -> ( name: "Nasi Putih", price: 3000, logo: 🍚 )
    # Seluruh daftar harga makanan
  )

  DISPLAY main container {layout horizontal}

  DISPLAY SummaryPrice COMPONENT WITH:
    INPUT plates
    INPUT mapping

  DISPLAY UploadImage COMPONENT WITH:
    FUNCTION setPlates
    # Mengirim gambar ke API /predictplate
    # Menyimpan response plates ke state

  DISPLAY DetectionResult COMPONENT WITH:
    INPUT plates
    INPUT mapping
    # Menampilkan:
    # - Conf dan bbox piring
    # - Daftar makanan per piring
    # - Conf dan bbox makanan
    # - Jumlah porsi (nasi & tumis max 1 per piring)

END StrukturComponent
```

Komponen *StrukturComponent* berfungsi sebagai pengendali utama yang mengatur alur data pada sistem deteksi makanan. Komponen ini menyimpan dua state utama, yaitu

detectedObjects yang berisi hasil deteksi YOLO dan *plates* yang memuat hasil pengelompokan makanan pada tiap wadah menggunakan metode Ray Casting. Selain itu, komponen ini juga mendefinisikan *mapping* berupa daftar nama makanan, harga, dan ikon yang digunakan untuk menampilkan informasi harga. Komponen ini kemudian menampilkan tiga komponen pendukung, yaitu *SummaryPrice* untuk menampilkan ringkasan harga berdasarkan hasil deteksi, *UploadImage* untuk mengunggah gambar dan menghasilkan output deteksi serta pengelompokan, serta *DetectionResult* untuk menampilkan jumlah piring, jumlah item, dan total harga. Dengan struktur ini, seluruh proses deteksi, pengelompokan, dan perhitungan harga dapat berjalan secara terintegrasi. Untuk bagian *summaryPrice* Component dapat dilihat pada pseudocode 11 berikut.

Pseudocode 11 : *SummaryPrice* Component

```

BEGIN SummaryPrice COMPONENT

    INPUT plates
    INPUT mapping
    DEFINE ONE_PORTION_ONLY = ["nasi_putih", "tumis_kangkung"]

    IF plates IS NULL THEN
        DISPLAY card "Ringkasan Total" kosong
        RETURN
    END IF

    SET totalPiring = LENGTH(plates)
    SET totalItem = 0
    SET totalHarga = 0

    FOR EACH plate IN plates DO

        INITIALIZE grouped AS EMPTY_DICTIONARY
        # Mengelompokkan makanan berdasarkan nama

        FOR EACH food IN plate.foods DO
            IF grouped DOES NOT HAVE KEY food.name THEN
                grouped[food.name] = EMPTY_LIST
            END IF

            ADD food TO grouped[food.name]
        END FOR

        # Hitung jumlah item & harga per jenis makanan
        FOR EACH key IN grouped DO

            SET itemInfo = mapping[key]
            IF itemInfo IS NULL THEN
                CONTINUE
            END IF

            IF key IS IN ONE_PORTION_ONLY THEN
                SET qty = 1
            ELSE
                SET qty = LENGTH(grouped[key])
            END IF

            totalItem = totalItem + qty
        END FOR
    END FOR

```

Pseudocode 12 : *SummaryPrice Component (lanjutan)*

```
        totalHarga = totalHarga + |itemInfo.price * qty|

    END FOR

END FOR

DISPLAY:
    Total Piring = totalPiring
    Total Item = totalItem
    Total Harga = totalHarga |format rupiah|

END SummaryPrice COMPONENT
```

Komponen pendukung *SummaryPrice* menjelaskan proses komponen *SummaryPrice* yang menerima data plates (hasil deteksi makanan per piring) dan mapping (daftar harga makanan), lalu menghitung total piring, total item, dan total harga. Jika plates kosong maka hanya menampilkan kartu ringkasan tanpa perhitungan. Untuk setiap piring, makanan dikelompokkan berdasarkan nama agar bisa dihitung jumlah porsinya, khusus makanan tertentu seperti nasi putih dan tumis kangkung dibatasi maksimal satu porsi per piring meskipun terdeteksi lebih dari satu. Setiap jumlah porsi kemudian dijumlahkan ke totalItem dan dikalikan dengan harga dari mapping untuk memperoleh totalHarga. Hasil akhirnya ditampilkan sebagai total piring, total item, dan total harga dalam format rupiah. Untuk bagian *uploadImage component* dapat dilihat pada pseudocode 13 dan 14 berikut.

Pseudocode 13 : *UploadImage Component*

```
FUNCTION UploadImage|setPlates|:

    SET videoConstraints = {width: 550, height: 300, facingMode: "environment"}

    INITIALIZE image = null
    INITIALIZE prediction = null
    INITIALIZE cameraImage = null
    INITIALIZE cameraOn = false

    CREATE webcamRef
    CREATE streamInterval

    FUNCTION sendFrameToBackend|frame|:
        TRY:
            SEND POST request TO "/predict-stream-image" WITH frame
            RECEIVE data FROM backend

            SET cameraImage = data.image_url
            SET setPlates = data.plates
        CATCH error:
            PRINT "Streaming error"
    END FUNCTION

    FUNCTION startCamera|:
        IF cameraOn = true THEN RETURN

        SET cameraOn = true

        SET streamInterval = INTERVAL every 300ms:
            IF webcamRef has frame THEN
```

Pseudocode 14 : *UploadImage* Component Lanjutan

```
        frame = webcamRef.getScreenshot()
        CALL sendFrameToBackend(frame)
    END FUNCTION

    FUNCTION stopCamera():
        SET cameraOn = false
        CLEAR streamInterval
        SET cameraImage = null
    END FUNCTION

    ON_COMPONENT_UNMOUNT:
        CLEAR streamInterval

    FUNCTION handleFileChange(event):
        SET image = event.selected_file
    END FUNCTION

    FUNCTION handleAnalyze():
        IF image = null THEN SHOW alert|"Pilih gambar terlebih dahulu"

        CREATE formData AND append image

        TRY:
            SEND POST request TO "/predictplate" WITH formData
            RECEIVE data

            SET prediction = data.image_url
            SET setPlates = data.plates

            SHOW alert|"Analisis berhasil"
        CATCH error:
            PRINT "Upload predict error"
    END FUNCTION

    RENDER interface:
        - Tombol Start Camera ~ startCamera()
        - Tombol Stop Camera ~ stopCamera()
        - Webcam {jika cameraOn = true}
        - Menampilkan hasil deteksi streaming {cameraImage}
        - Upload file ~ handleFileChange
        - Tombol analisis ~ handleAnalyze()
        - Menampilkan gambar hasil prediksi upload

    END FUNCTION
```

Komponen *UploadImage* berfungsi untuk menangani dua jenis input, yaitu streaming kamera secara real-time dan unggahan file gambar. Pada saat pengguna menekan tombol *Start Camera*, sistem akan menyalakan webcam dan mengirimkan satu frame setiap 300 ms ke backend untuk dilakukan proses deteksi menggunakan model YOLOv8-seg, kemudian hasilnya segera ditampilkan sebagai umpan balik real-time. Sebaliknya, tombol *Stop Camera* akan menghentikan streaming dan membersihkan interval proses. Selain itu, pengguna juga dapat mengunggah gambar melalui fitur *Upload File*, yang kemudian dikirim ke backend untuk dianalisis menggunakan endpoint khusus. Data hasil deteksi berupa gambar prediksi, objek terdeteksi, serta isi masing-masing piring akan disimpan dalam state

dan ditampilkan ke antarmuka pengguna. Komponen ini memastikan seluruh fungsi input baik kamera maupun file dapat berjalan dengan baik dan memberikan hasil deteksi secara langsung. Untuk bagian `detectionResult` Component dapat dilihat pada pseudocode 15 berikut

Pseudocode 15 : *DetectionResult* Component

```

BEGIN DetectionResult

    SET ONE_PORTION_ONLY = ["nasi_putih", "tumis_kangkung"]

    IF plates IS EMPTY OR NULL THEN
        DISPLAY "Belum ada hasil deteksi"
        STOP
    END IF

    FOR EACH plate IN plates DO

        GROUP foods BY food.name → groupedFoods
        SET totalPrice = 0

        FOR EACH {foodName, foods} IN groupedFoods DO
            SET price = mapping[foodName].price OR 0

            IF foodName IN ONE_PORTION_ONLY THEN
                SET portion = 1
            ELSE
                SET portion = COUNT(foods)
            END IF

            ADD price * portion TO totalPrice
        END FOR

        DISPLAY plate.id, plate.conf, plate.bbox

        FOR EACH {foodName, foods} IN groupedFoods DO
            SET portion USING RULE ONE_PORTION_ONLY

            DISPLAY foodName, portion

            FOR EACH food IN foods DO
                DISPLAY food.conf AND food.bbox
            END FOR
        END FOR

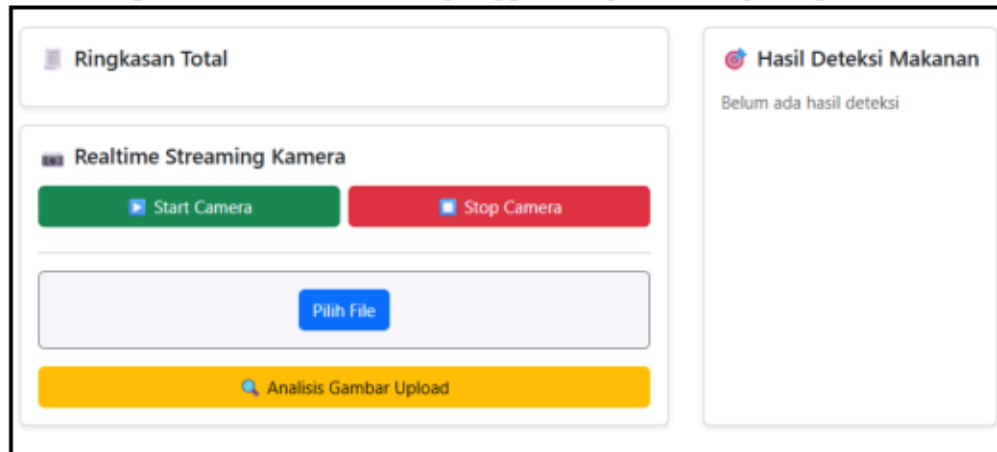
        DISPLAY totalPrice AND COUNT(groupedFoods) AS jumlah jenis makanan
    END FOR

END DetectionResult

```

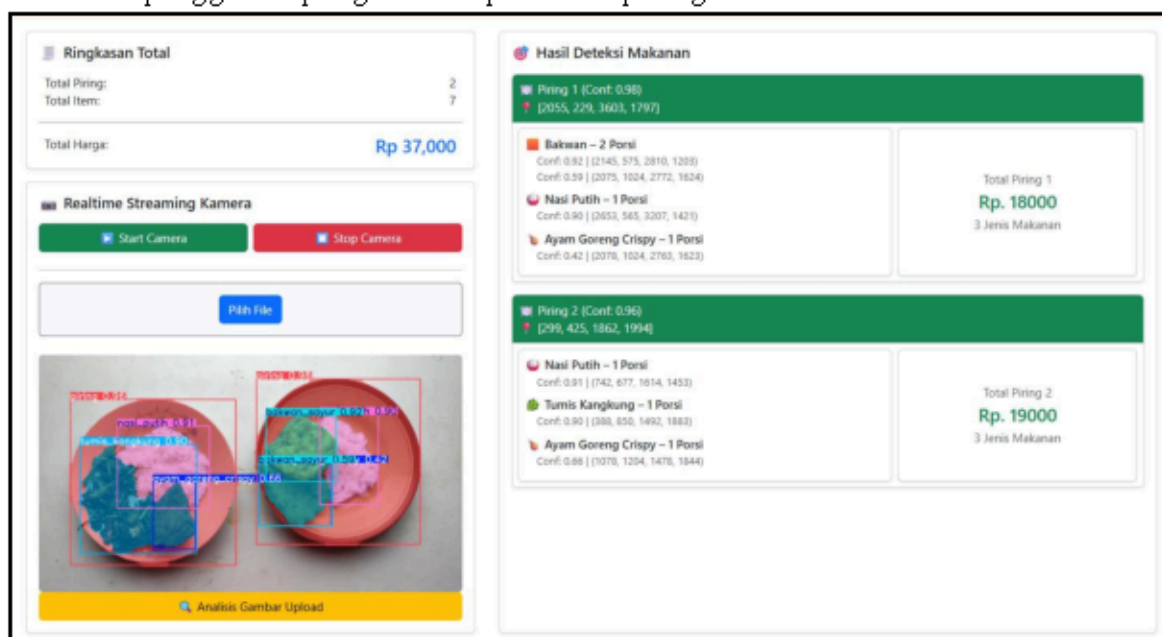
Komponen `DetectionResult` yang menampilkan hasil deteksi makanan per piring berdasarkan data `plates` dan `mapping`. Jika tidak ada data, sistem hanya menampilkan pesan bahwa belum ada hasil deteksi. Untuk setiap piring, makanan dikelompokkan berdasarkan nama agar dapat dihitung jumlah porsinya, aturan khusus diterapkan untuk nasi putih dan tumis kangkung yang maksimal dihitung satu porsi per piring meskipun terdeteksi lebih dari satu. Setiap kelompok makanan menampilkan nama, jumlah porsi, serta seluruh nilai

confidence dan koordinat bounding box dari tiap deteksi. Selanjutnya sistem menghitung total harga per piring dengan mengalikan harga pada mapping dengan jumlah porsi yang sudah mengikuti aturan, lalu menampilkan total harga dan jumlah jenis makanan pada piring tersebut. Untuk gambar antarmuka awal pengguna dapat dilihat pada gambar 4.16 berikut.



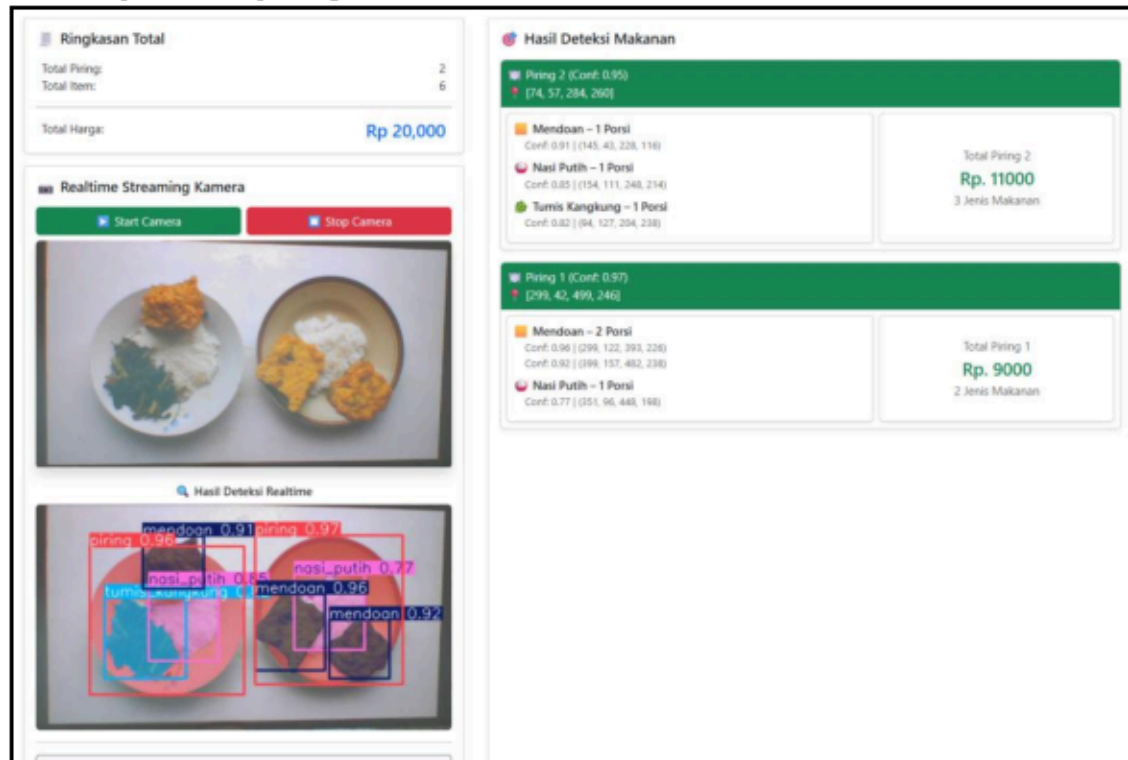
Gambar 4.16 Antarmuka Awal Pengguna

Gambar 4.16 menunjukkan tampilan awal pengguna sebelum memasukkan gambar. Pada sistem ini pengguna akan disajikan beberapa tampilan seperti tombol *start camera* serta *stop camera* yang berguna untuk memulai melakukan deteksi secara real time menggunakan kamera serta melakukan pemberhentian deteksi makanan. Selain itu terdapat tombol pilih file yang berguna untuk melakukan deteksi dengan input gambar. Setelah pengguna melakukan input pengguna dapat melihat hasil deteksi dengan menekan tombol analisis gambar upload. Dalam antarmuka awal pengguna akan ditampilkan teks belum ada deteksi yang menandakan pengguna belum melakukan deteksi sama sekali. Untuk gambar antarmuka pengguna input gambar dapat dilihat pada gambar 4.17 berikut.



Gambar 4.17 Antarmuka Pengguna Input Gambar

Gambar 4.17 menunjukkan tampilan ketika pengguna melakukan input gambar. Setelah pengguna memilih gambar serta menekan tombol analisis gambar upload maka pengguna akan disajikan gambar hasil segmentasi dari model yang telah dibuat. Data hasil deteksi yang diberikan dari *backend* ke *frontend* akan ditampilkan pada hasil deteksi makanan, dibagian tersebut dapat dilihat deteksi makanan yang sudah dikelompokkan masing-masing piring. Data tersebut berupa confidence, jumlah porsi serta lokasi keberaddan objek tersebut. Selanjutnya pada bagian ringkasan total akan menampilkan perhitungan dari data hasil deteksi makanan. Sedangkan untuk antarmuka pengguna input kamera dapat dilihat pada gambar 4.18 berikut.



Gambar 4.18 Antarmuka Pengguna Input Kamera

Gambar 4.18 menunjukkan tampilan ketika pengguna melakukan input berupa kamera *real-time*. Setelah pengguna menekan tombol *start camera* maka sistem otomatis melakukan deteksi secara *real-time* dengan menggunakan kamera yang telah dikonfigurasi yakni EYD PC03. Sistem akan mengirimkan data dari *backend* ke *frontend* secara real-time dan hasil nya akan ditampilkan dalam bagian hasil deteksi makanan serta ringkasan total. Data yang dikirim tersebut berupa hasil pengelompokkan berdasarkan piring, objek makanan, *confidence* objek, serta lokasi objek. Untuk pemberhentian deteksi secara *real-time* pengguna dapat menekan tombol *stop camera*, dengan begitu sistem otomatis memberhentikan akses API menuju endpoint *stream image*.

4.1.6 Pengujian Sitem

Pengujian sistem dilakukan untuk memastikan bahwa sistem berjalan sesuai dengan yang diharapkan. Penggunakan ini menggunakan Black Box Testing. Metode ini berfokus pada keberhasilan dalam mengunggah gambar, mengakses kamera secara *real-time*, serta kemampuan sistem menampilkan data. Tabel 4.5 merupakan hasil dari pengujian sistem.

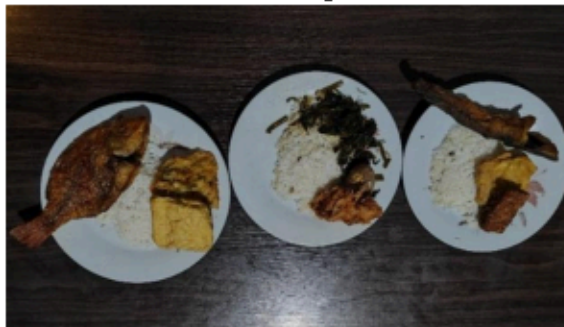
Table 4.5 Pengujian Sistem

No	Test Case	Parameter Uji	Hasil
1	Pengguna dapat melakukan unggah gambar/kamera secara <i>real-time</i>	Pengguna berhasil melakukan unggah gambar	Berhasil
2	Pengguna dapat melihat jumlah dan harga masing-masing makanan pada wadah tertentu	Pengguna berhasil melihat jumlah dan harga masing-masing makanan pada wadah tertentu	Berhasil

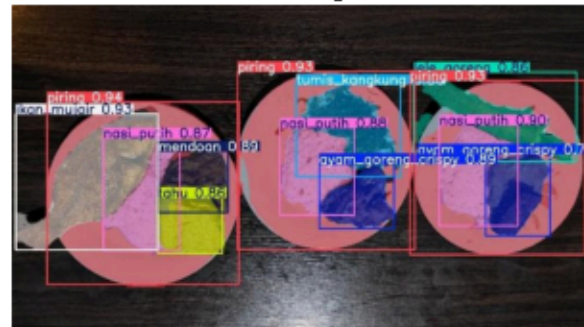
Tabel 4.5 di atas menunjukkan bahwa sistem dengan baik berhasil dalam menguji 2 aspek yang telah ditentukan dalam metode *black box testing*. Pengujian dilakukan pada dua aspek utama, yaitu proses unggah gambar atau penggunaan kamera secara *real-time*, serta kemampuan sistem dalam menampilkan jumlah dan harga masing-masing makanan pada wadah tertentu. Hasil tersebut menunjukkan pengguna dapat melakukan input gambar serta melihat output hasil deteksi makanan pada wadah tertentu dengan baik.

Untuk pengujian pengaruh jumlah wadah, objek serta kondisi tumpang tindih objek akan dilakukan percobaan dengan kriteria jumlah wadah, objek serta kondisi *overlapping* yang berbeda. Pada pengujian ini dilakukan dengan 10 gambar *no-overlapping* dan 10 gambar *overlapping*. Untuk contoh hasil gambar pengujian pada objek *overlapping* dan tidak *overlapping* dapat dilihat pada gambar serta tabel dibawah. Berikut gambar pengujian *overlapping* ke-1

(a) Sebelum Segmentasi



(b) Sesudah Segmentasi



Gambar 4.19 Pengujian Overlapping ke-1

Pada gambar 4.19 menunjukkan hasil pengujian sistem pada kondisi *overlapping* dengan tiga wadah makanan. Pada piring 1, sistem berhasil mendeteksi ikan mujair, tahu, dan nasi putih (3TP). Namun pada *ground truth* bakwan objek tidak terdeteksi (1FN) dan terklasifikasi sebagai objek mendoan (1FP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 1 memperoleh nilai sebesar 75%. Pada piring 2, sistem berhasil mendeteksi ayam goreng, nasi putih, dan tumis kangkung (3TP), tetapi pada *ground truth* telur bacem objek tidak terdeteksi (1FN). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 2 memperoleh nilai sebesar 86%. Sementara itu, pada piring 3 sistem berhasil mendeteksi nasi putih dan lele goreng (2TP). Namun pada *ground truth* mendoan serta tempe objek tidak terdeteksi (2FN) dan terklasifikasi sebagai objek ayam goreng (1FP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada

piring 3 memperoleh nilai sebesar 57%. Hasil dari evaluasi pengujian *overlapping* ke-1 dapat dilihat pada tabel 4.6 berikut.

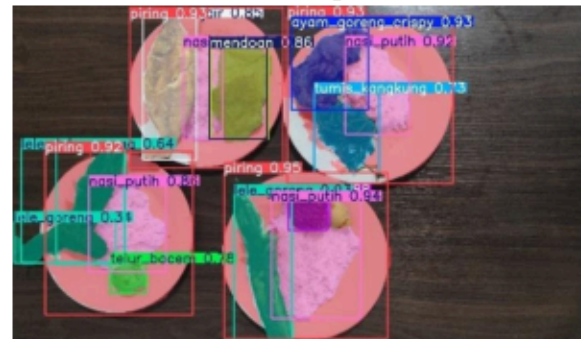
Table 4.6 Hasil Evaluasi Pengujian Overlapping ke-1

Piring	Ground Truth	Hasil Deteksi	RC	Deteksi			
				TP	FP	FN	TF
Piring 1	Ikan Mujair, Tahu, Bakwan, Nasi Putih	Ikan Mujair, Tahu, Mendoan, Nasi Putih	75%	3	1	1	0
Piring 2	Ayam Goreng, Nasi Putih, Tumis Kangkung, Telur Bacem	Ayam Goreng, Nasi Putih, Tumis Kangkung	86%	3	0	1	0
Piring 3	Lele Goreng, Mendoan, Tempe, Nasi Putih	Lele Goreng, Ayam Goreng, Nasi Putih	57%	2	1	2	0

(a) Sebelum Segmentasi



(b) Sesudah Segmentasi



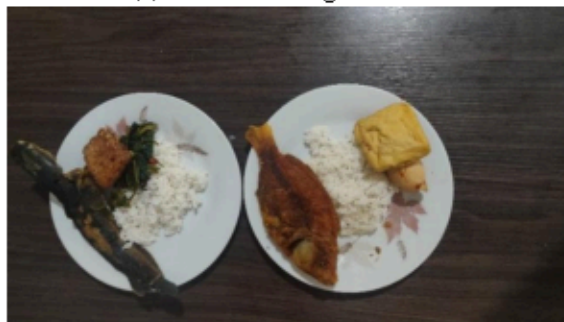
Gambar 4.20 Pengujian Overlapping ke-2

Pada gambar 4.20 menunjukkan hasil pengujian sistem pada kondisi *overlapping* dengan empat wadah makanan. Pada piring 1, sistem berhasil mendeteksi semua objek makanan yakni ikan mujair, tahu, mendoan, dan nasi putih (4TP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 1 memperoleh nilai sebesar 100%. Pada piring 2, sistem berhasil mendeteksi ayam goreng, nasi putih, dan tumis kangkung (3TP), tetapi pada *ground truth* bakwan objek tidak terdeteksi (1FN). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 2 memperoleh nilai sebesar 86%. Sementara itu, pada piring 3 sistem berhasil mendeteksi nasi putih dan lele goreng (2TP). Namun pada *ground truth* tempe dan tumis kangkung objek tidak terdeteksi (2FN) dan terklasifikasi sebagai objek telur bacem dan dua lele goreng (3FP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 3 memperoleh nilai sebesar 60%. Pada piring 4 sistem berhasil mendeteksi lele goreng, tempe, serta nasi putih (3TP). Namun pada *ground truth* telur bacem objek tidak terdeteksi (1FN). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 3 memperoleh nilai sebesar 86%. Hasil dari evaluasi pengujian *overlapping* ke-2 dapat dilihat pada tabel 4.7 berikut.

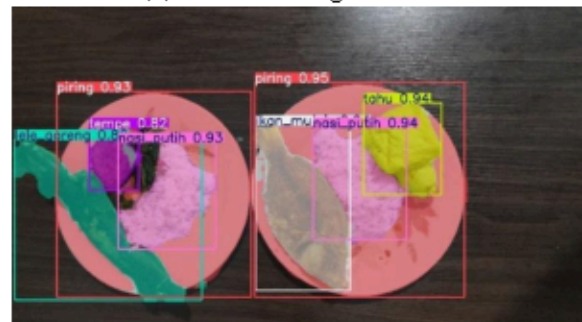
Table 4.7 Hasil Evaluasi Pengujian Overlapping ke-2

Piring	Ground Truth	Hasil Deteksi	RC	Deteksi			
				TP	FP	FN	TF
Piring 1	Ikan Mujair, Tahu, Mendoan, Nasi Putih	Ikan Mujair, Tahu, Mendoan, Nasi Putih	100%	4	0	0	0
Piring 2	Ayam Goreng, Bakwan, Tumis Kangkung, Nasi Putih	Ayam Goreng, Nasi Putih, Tumis Kangkung	86%	3	0	1	0
Piring 3	Lele Goreng, Tempe, Telur Bacem, Nasi Putih	Lele Goreng, Lele Goreng, Lele Goreng, Telur Bacem, Nasi Putih	60%	3	2	1	0
Piring 4	Lele Goreng, Tempe, Telur Bacem, Nasi Putih	Lele Goreng, Tempe, Nasi Putih	86%	3	0	1	0

(a) Sebelum Segmentasi



(b) Sesudah Segmentasi



Gambar 4.21 Pengujian Overlapping ke-3

Pada gambar 4.21 menunjukkan hasil pengujian sistem pada kondisi *overlapping* dengan dua wadah makanan. Pada piring 1, sistem berhasil mendeteksi semua objek makanan yakni lele goren, tempe, dan nasi putih (3TP). Namun *ground truth* tumis kangkung objek tidak terdeteksi (1FN). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 1 memperoleh nilai sebesar 86%. Pada piring 2, sistem berhasil mendeteksi ikan mujair, tahu, dan nasi putih (3TP), tetapi pada *ground truth* telur bacem objek tidak terdeteksi (1FN). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 2 memperoleh nilai sebesar 86%. Hasil dari evaluasi pengujian *overlapping* ke-3 dapat dilihat pada tabel 4.8 berikut.

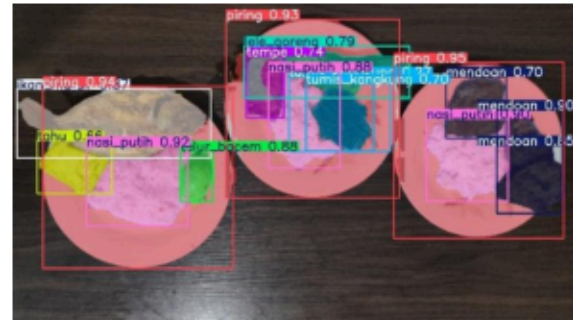
Table 4.8 Hasil Evaluasi Pengujian Overlapping ke-3

Piring	Ground Truth	Hasil Deteksi	RC	Deteksi			
				TP	FP	FN	TF
Piring 1	Lele Goreng, Tempe, Tumis Kangkung, Nasi Putih	Lele Goreng, Tempe, Nasi Putih	86%	3	0	1	0
Piring 2	Ikan Mujair, Tahu, Telur Bacem, Nasi Putih	Ikan Mujair, Tahu, Nasi Putih	86%	3	0	1	0

(a) Sebelum Segmentasi



(b) Sesudah Segmentasi



Gambar 4.22 Pengujian Overlapping ke-4

Pada gambar 4.22 menunjukkan hasil pengujian sistem pada kondisi *overlapping* dengan tiga wadah makanan. Pada piring 1, sistem berhasil mendeteksi semua objek makanan yakni ikan mujair, tahu, telur bacem, dan nasi putih (4TP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 1 memperoleh nilai sebesar 100%. Pada piring 2, sistem berhasil mendeteksi lele goreng, tempe, tumis kangkung, dan nasi putih (4TP), tetapi terdapat satu objek yang terklasifikasi sebagai objek tumis kangkung secara berlebihan (1FP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 2 memperoleh nilai sebesar 80%. Sementara itu, pada piring 3 sistem berhasil mendeteksi mendoan dan nasi putih (2TP). Namun pada *ground truth* ayam goreng dan bakwan objek tidak terdeteksi (2FN) dan terklasifikasi sebagai objek dua mendoan (2FP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 3 memperoleh nilai sebesar 50%. Hasil dari evaluasi pengujian *overlapping* ke-4 dapat dilihat pada tabel 4.9 dan tabel 4.10 berikut.

Table 4.9 Hasil Evaluasi Pengujian Overlapping ke-4

Piring	Ground Truth	Hasil Deteksi	RC	Deteksi			
				TP	FP	FN	TF
Piring 1	Ikan Mujair, Tahu, Telur Bacem, Nasi Putih	Ikan Mujair, Tahu, Telur Bacem, Nasi Putih	100%	4	0	0	0

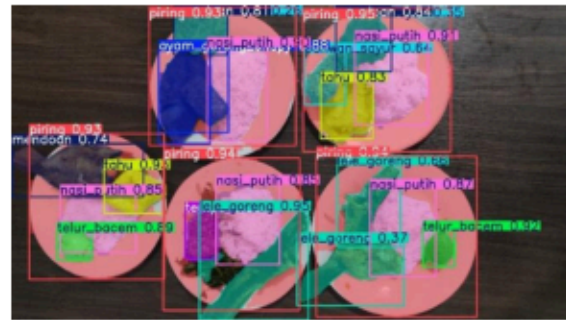
Table 4.10 Hasil Evaluasi Pengujian Overlapping ke-4 (lanjutan)

Piring	Ground Truth	Hasil Deteksi	RC	Deteksi			
				TP	FP	FN	TF
Piring 2	Lele Goreng, Tempe, Tumis Kangkung, Nasi Putih	Lele Goreng, Tempe, Tumis Kangkung, Nasi Putih	80%	4	1	0	0
Piring 3	Mendoan, Ayam Goreng, Bakwan Nasi Putih	Mendoan, Mendoan, Mendoan, Nasi Putih	50%	2	2	2	0

(a) Sebelum Segmentasi



(b) Sesudah Segmentasi



Gambar 4.23 Pengujian Overlapping ke-5

Pada gambar 4.23 menunjukkan hasil pengujian sistem pada kondisi *overlapping* dengan lima wadah makanan. Pada piring 1, sistem berhasil mendeteksi objek bakwan serta nasi putih (2TP). Namun pada *ground truth* ikan mujair dan tempe objek tidak terdeteksi (2FN) dan terklasifikasi sebagai objek mendoan dan dua ayam goreng (3FP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 1 memperoleh nilai sebesar 40%. Pada piring 2, sistem berhasil mendeteksi ayam goreng, bakwan, mendoan, dan nasi putih (4TP), tetapi terdapat satu objek yang terklasifikasi sebagai objek tahu secara berlebihan (1FP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 2 memperoleh nilai sebesar 60%. Sementara itu, pada piring 3 sistem berhasil mendeteksi telur bacem, nasi putih dan tahu (3TP). Namun pada *ground truth* ikan mujair objek tidak terdeteksi (1FN) dan terklasifikasi sebagai objek mendoan (1FP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 3 memperoleh nilai sebesar 75%. Pada piring 4 sistem berhasil mendeteksi lele goreng, tempe, serta nasi putih (3TP). Namun pada *ground truth* tumis kangkung objek tidak terdeteksi (1FN). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 4 memperoleh nilai sebesar 86%. Sedangkan pada piring 5 sistem berhasil mendeteksi lele goreng, telur bacem, serta nasi putih (3TP). Namun pada *ground truth* mendoan objek tidak terdeteksi (1FN) dan terklasifikasi sebagai objek lele goreng (1FP). Berdasarkan perhitungan TP, FP, dan FN,

algoritma Ray Casting pada piring 5 memperoleh nilai sebesar 75%. Hasil dari evaluasi pengujian *overlapping* ke-5 dapat dilihat pada tabel 4.11 berikut.

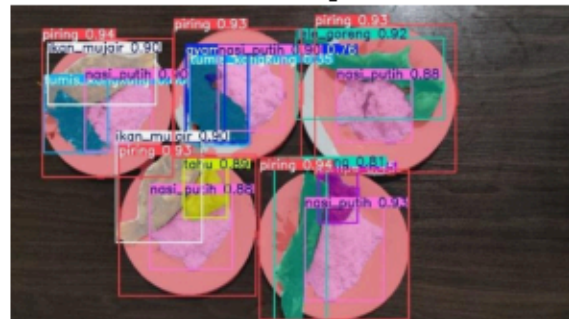
Table 4.11 Hasil Evaluasi Pengujian *Overlapping* ke-5

Piring	Ground Truth	Hasil Deteksi	RC	Deteksi			
				TP	FP	FN	TF
Piring 1	Ikan Mujair, Tempe, Bakwan, Nasi Putih	Ayam Goreng, Ayam Goreng, Mendoan, Bakwan, Nasi Putih	40%	2	3	2	0
Piring 2	Ayam Goreng, Bakwan, Mendoan, Nasi Putih	Bakwan, Bakwan, Tahu, Mendoan, Nasi Putih	60%	3	2	1	0
Piring 3	Ikan Mujair, Tahu, Telur Bacem, Nasi Putih	Mendoan, Tahu, Telur Bacem, Nasi Putih	75%	3	1	1	0
Piring 4	Lele Goreng, Tempe, Tumis Kangkung, Nasi Putih	Lele Goreng, Tempe, Nasi Putih	86%	3	0	1	0
Piring 5	Lele Goreng, Mendoan, Telur Bacem, Nasi Putih	Lele Goreng, Lele Goreng, Telur Bacem, Nasi Putih	75%	3	1	1	0

(a) Sebelum Segmentasi



(b) Sesudah Segmentasi



Gambar 4.24 Pengujian *Overlapping* ke-6

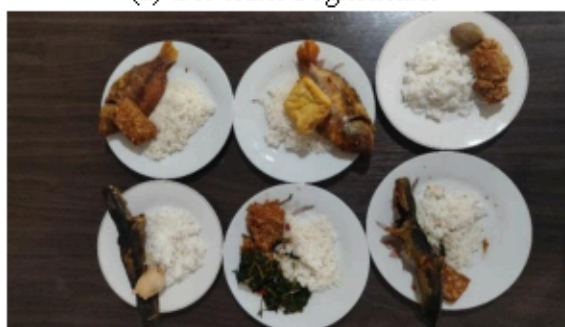
Pada gambar 4.24 menunjukkan hasil pengujian sistem pada kondisi *overlapping* dengan lima wadah makanan. Pada piring 1, sistem berhasil mendeteksi semua objek makanan yakni ikan mujair, tumis kangkung, serta nasi putih (3TP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 1 memperoleh nilai sebesar 100%. Pada piring 2, sistem berhasil mendeteksi ayam goreng, tumis kangkung, dan nasi putih (3TP), tetapi terdapat satu objek yang terklasifikasi sebagai objek ayam goreng secara

berlebihan (1FP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 2 memperoleh nilai sebesar 86%. Sementara itu, pada piring 3 sistem berhasil mendeteksi lele goreng, dan nasi putih (2TP). Namun pada *ground truth* telur bacem objek tidak terdeteksi (1FN). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 3 memperoleh nilai sebesar 80%. Pada piring 4 sistem berhasil mendeteksi semua objek makanan yakni ikan mujair, tahu, serta nasi putih (3TP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 4 memperoleh nilai sebesar 100%. Sedangkan pada piring 5 sistem berhasil mendeteksi lele goreng, serta nasi putih (2TP). Namun pada *ground truth* bakwan objek tidak terdeteksi (1FN) dan terklasifikasi sebagai objek tempe (1FP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 5 memperoleh nilai sebesar 60%. Hasil dari evaluasi pengujian *overlapping* ke-6 dapat dilihat pada tabel 4.12 berikut.

Table 4.12 Hasil Evaluasi Pengujian Overlapping ke-6

Piring	Ground Truth	Hasil Deteksi	RC	Deteksi			
				TP	FP	FN	TF
Piring 1	Ikan Mujair, Tumis Kangkung, Nasi Putih	Ikan Mujair, Tumis Kangkung, Nasi Putih	100%	3	0	0	0
Piring 2	Ayam Goreng, Tumis Kangkung, Nasi Putih	Ayam Goreng, Ayam Goreng, Tumis Kangkung, Nasi Putih	86%	3	1	0	0
Piring 3	Lele Goreng, Telur Bacem, Nasi Putih	Lele Goreng, Nasi Putih	80%	2	0	1	0
Piring 4	Ikan Mujair, Tahu, Nasi Putih	Ikan Mujair, Tahu, Nasi Putih	100%	3	0	0	0
Piring 5	Lele Goreng, Bakwan, Nasi Putih	Lele Goreng, Tempe, Nasi Putih	60%	2	1	1	0

(a) Sebelum Segmentasi



(b) Sesudah Segmentasi



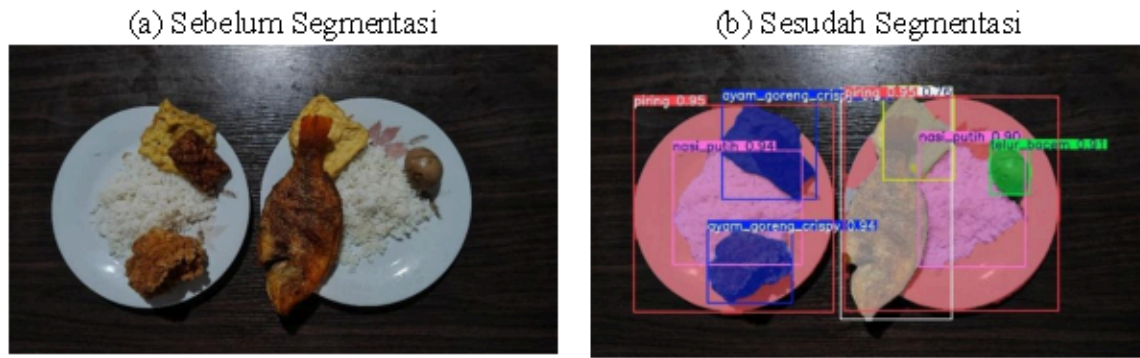
Gambar 4.25 Pengujian Overlapping ke-7

Pada gambar 4.25 menunjukkan hasil pengujian sistem pada kondisi *overlapping* dengan enam wadah makanan. Pada piring 1, sistem berhasil mendeteksi objek makanan

ikan mujair, serta nasi putih (2TP). Namun pada *ground truth* tempe objek tidak terdeteksi (1FN). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 1 memperoleh nilai sebesar 80%. Pada piring 2, sistem berhasil mendeteksi ikan mujair, dan nasi putih (2TP). Namun pada *ground truth* tahu objek tidak terdeteksi (1FN) dan terklasifikasi sebagai objek ayam goreng (1FP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 2 memperoleh nilai sebesar 60%. Sementara itu, pada piring 3 sistem berhasil mendeteksi semua objek makanan yakni ayam goreng, telur bacem, dan nasi putih (3TP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 3 memperoleh nilai sebesar 100%. Pada piring 4 sistem berhasil mendeteksi semua objek makanan yakni lele goreng, telur bacem, serta nasi putih (3TP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 4 memperoleh nilai sebesar 100%. Sedangkan pada piring 5 sistem berhasil mendeteksi ayam goreng, tumus kangkung serta nasi putih (3TP), tetapi terdapat objek yang terklasifikasi sebagai objek tempe dan nasi putih secara berlebihan (2FP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 5 memperoleh nilai sebesar 75%. Untuk piring 6 sistem berhasil mendeteksi semua objek makanan yakni lele goreng, tempe, serta nasi putih. Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 4 memperoleh nilai sebesar 100%. Hasil dari evaluasi pengujian *overlapping* ke-8 dapat dilihat pada tabel 4.13 berikut.

Table 4.13 Hasil Evaluasi Pengujian Overlapping ke-7

Piring	Ground Truth	Hasil Deteksi	RC	Deteksi			
				TP	FP	FN	TF
Piring 1	Ikan Mujair, Tempe, Nasi Putih	Ikan Mujair, Nasi Putih	80%	2	0	1	0
Piring 2	Ikan Mujair, Tahu, Nasi Putih	Ikan Mujair, Ayam Goreng, Nasi Putih	60%	2	1	1	0
Piring 3	Ayam Goreng, Telur Bacem, Nasi Putih	Ayam Goreng, Telur Bacem, Nasi Putih	100%	3	0	0	0
Piring 4	Lele Goreng, Telur Bacem, Nasi Putih	Lele Goreng, Telur Bacem, Nasi Putih	100%	3	0	0	0
Piring 5	Ayam Goreng, Tumis Kangkung, Nasi Putih	Ayam Goreng, Tempe, Tumis Kangkung, Nasi Putih, Nasi Putih	75%	3	2	0	0
Piring 6	Lele Goreng, Tempe, Nasi Putih	Lele Goreng, Tempe, Nasi Putih	100%	3	0	0	0



Gambar 4.26 Pengujian Overlapping ke-8

Pada gambar 4.26 menunjukkan hasil pengujian sistem pada kondisi *overlapping* dengan dua wadah makanan. Pada piring 1, sistem berhasil mendeteksi objek makanan mendoa, ayam goreng, serta nasi putih (3TP). Namun pada *ground truth* tempe objek tidak terdeteksi (1FN) dan terklasifikasi sebagai ayam goreng (1FN). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 1 memperoleh nilai sebesar 67%. Pada piring 2, sistem berhasil mendeteksi semua objek makanan yakni ikan mujair, tahu, telur bacem, dan nasi putih (4TP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 2 memperoleh nilai sebesar 100%. Hasil dari evaluasi pengujian *overlapping* ke-8 dapat dilihat pada tabel 4.14 berikut.

Table 4.14 Hasil Evaluasi Pengujian Overlapping ke-8

Piring	Ground Truth	Hasil Deteksi	RC	Deteksi			
				TP	FP	FN	TF
Piring 1	Mendoan, Tempe, Ayam Goreng, Nasi Putih	Mendoan, Ayam Goreng, Ayam Goreng, Nasi Putih	75%	3	1	1	0
Piring 2	Ikan Mujair, Tahu, Telur Bacem, Nasi Putih	Ikan Mujair, Tahu, Telur Bacem, Nasi Putih	100%	4	0	0	0



Gambar 4.27 Pengujian Overlapping ke-9

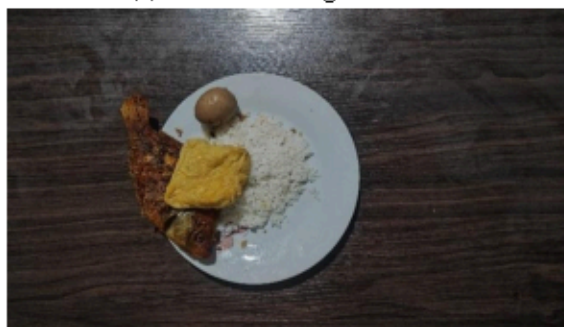
Pada gambar 4.27 menunjukkan hasil pengujian sistem pada kondisi *overlapping* dengan empat wadah makanan. Pada piring 1, sistem berhasil mendeteksi objek makanan

ayam goreng, tempe, serta nasi putih (3TP). Namun pada *ground truth* mendoan objek tidak terdeteksi (1FN). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 1 memperoleh nilai sebesar 86%. Pada piring 2, sistem berhasil mendeteksi lele goreng, tahu, bakwan, dan nasi putih (4TP), tetapi terdapat objek yang terklasifikasi sebagai nasi putih secara berlebihan (1FP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 2 memperoleh nilai sebesar 80%. Sementara itu, pada piring 3 sistem berhasil mendeteksi lele goreng, dan nasi putih (2TP). Namun pada *ground truth* tempe, dan nasi putih objek tidak terdeteksi (2FN). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 3 memperoleh nilai sebesar 60%. Pada piring 4 sistem berhasil mendeteksi semua objek makanan yakni ikan mujair, telur bacem, tumis kangkung, serta nasi putih (4TP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 4 memperoleh nilai sebesar 100%. Hasil dari evaluasi pengujian *overlapping* ke-9 dapat dilihat pada tabel 4.15 berikut.

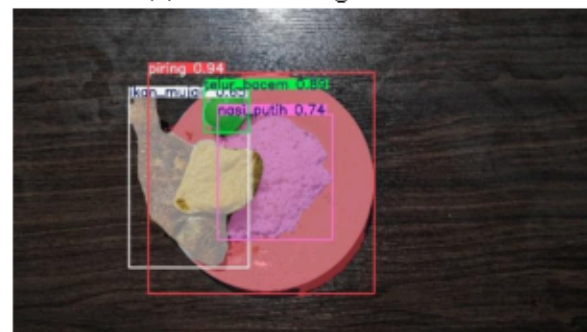
Table 4.15 Hasil Evaluasi Pengujian Overlapping ke-9

Piring	Ground Truth	Hasil Deteksi	RC	Deteksi			
				TP	FP	FN	TF
Piring 1	Ayam Goreng, Mendoan, Tempe, Nasi Putih	Ayam Goreng, , Tempe, Nasi Putih	86%	3	0	1	0
Piring 2	Lele Goreng, Tahu, Bakwan, Nasi Putih	Lele Goreng, Tahu, Bakwan, Nasi Putih, Nasi Putih	80%	4	1	0	0
Piring 3	Lele Goreng, Tempe, Tumis Kangkung, Nasi Putih	Lele Goreng, Nasi Putih	60%	2	0	2	0
Piring 4	Ikan Mujair, Telur Bacem, Tumis Kangkung, Nasi Putih	Ikan Mujair, Telur Bacem, Tumis Kangkung, Nasi Putih	100%	4	0	0	0

(a) Sebelum Segmentasi



(b) Sesudah Segmentasi



Gambar 4.28 Pengujian Overlapping ke-10

Pada gambar 4.28 menunjukkan hasil pengujian sistem pada kondisi *overlapping* dengan satu wadah makanan. Pada piring 1, sistem berhasil mendeteksi objek makanan ikan mujair, serta nasi putih (2TP). Namun pada *ground truth* tahu objek tidak terdeteksi (1FN). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 1 memperoleh nilai sebesar 80%. Hasil dari evaluasi pengujian *overlapping* ke-10 dapat dilihat pada tabel 4.16 berikut.

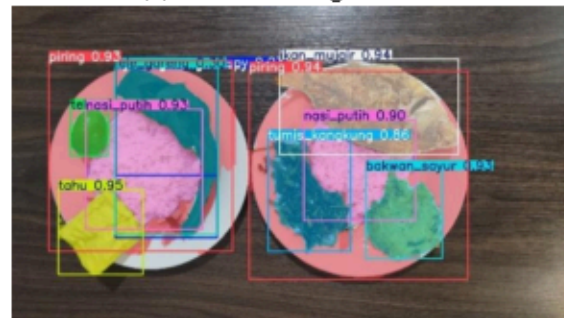
Table 4.16 Hasil Evaluasi Pengujian Overlapping ke-10

Piring	Ground Truth	Hasil Deteksi	RC	Deteksi			
				TP	FP	FN	TF
Piring 1	Ikan Mujair, Telur Bacem, Tahu, Nasi Putih	Ikan Mujair, Telur Bacem, Nasi Putih	80%	2	0	1	0

(a) Sebelum Segmentasi



(b) Sesudah Segmentasi



Gambar 4.29 Pengujian No Overlapping ke-1

Pada gambar 4.29 menunjukkan hasil pengujian sistem pada kondisi *no overlapping* dengan dua wadah makanan. Pada piring 1, sistem berhasil mendeteksi objek makanan lele goreng, tahu, telur bacem, serta nasi putih (4TP), tetapi terdapat objek yang terklasifikasi secara berlebihan yakni dua ayam goreng (2FP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 1 memperoleh nilai sebesar 80%. Pada piring 2, sistem berhasil mendeteksi semua objek makanan yakni tumis kakung, ikan mujair, bakwan, dan nasi putih (4TP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 2 memperoleh nilai sebesar 100%. Hasil dari evaluasi pengujian *no overlapping* ke-1 dapat dilihat pada tabel 4.17 dan tabel 4.18 berikut.

Table 4.17 Hasil Evaluasi Pengujian No Overlapping ke-1

Piring	Ground Truth	Hasil Deteksi	RC	Deteksi			
				TP	FP	FN	TF
Piring 1	Lele Goreng, Tahu, Telur Bacem, Nasi Putih	Ayam Goreng, Ayam Goreng, Lele Goreng, Tahu, Telur Bacem, Nasi Putih	80%	4	2	0	0

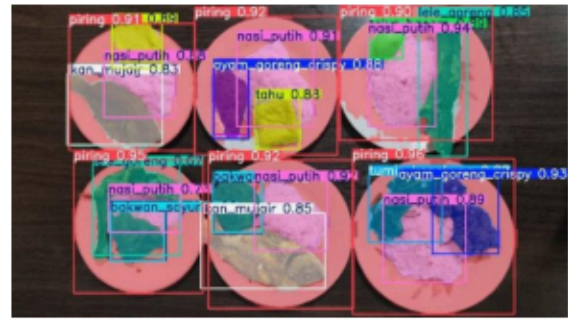
Table 4.18 Hasil Evaluasi Pengujian No Overlapping ke-1 (Lanjutan)

Piring	Ground Truth	Hasil Deteksi	RC	Deteksi			
				TP	FP	FN	TF
Piring 2	Tumis Kangkung, Ikan Mujair, Bakwan, Nasi Putih	Tumis Kangkung, Ikan Mujair, Bakwan, Nasi Putih	100%	4	0	0	0

(a) Sebelum Segmentasi



(b) Sesudah Segmentasi



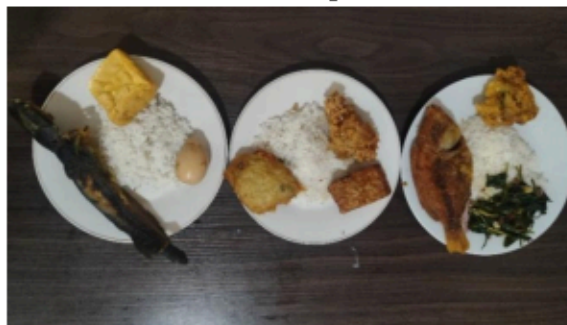
Gambar 4.30 Pengujian No Overlapping ke-2

Pada gambar 4.30 menunjukkan hasil pengujian sistem pada kondisi *no overlapping* dengan enam wadah makanan. Pada piring 1, sistem berhasil mendeteksi objek makanan ikan mujair, tahu, serta nasi putih (3TP), tetapi terdapat objek yang terklasifikasikan secara berlebihan yakni lele goreng (1FP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 1 memperoleh nilai sebesar 86%. Pada piring 2, sistem berhasil mendeteksi ayam goreng, tahu, dan nasi putih (3TP), tetapi terdapat objek yang terklasifikasi secara berlebihan yakni tempe secara berlebihan (1FP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 2 memperoleh nilai sebesar 86%. Sementara itu, pada piring 3 sistem berhasil mendeteksi semua objek makanan yakni lele goreng, telur bacem, dan nasi putih (3TP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 3 memperoleh nilai sebesar 100%. Pada piring 4 sistem berhasil mendeteksi lele goreng, bakwan, serta nasi putih (3TP), tetapi terdapat objek yang terklasifikasikan secara berlebihan yakni mendoan (1FP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 4 memperoleh nilai sebesar 86%. Sedangkan pada piring 5, sistem berhasil mendeteksi ikan mujair, bakwan, serta nasi putih (3TP), tetapi terdapat objek yang terklasifikasikan secara berlebihan yakni mendoan (1FP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 5 memperoleh nilai sebesar 86%. Untuk piring 6 sistem berhasil mendeteksi semua objek makanan yakni ayam goreng, tumis kangkung, serta nasi putih (3TP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 6 memperoleh nilai sebesar 100%. Hasil dari evaluasi pengujian *no overlapping* ke-2 dapat dilihat pada tabel 4.19 berikut.

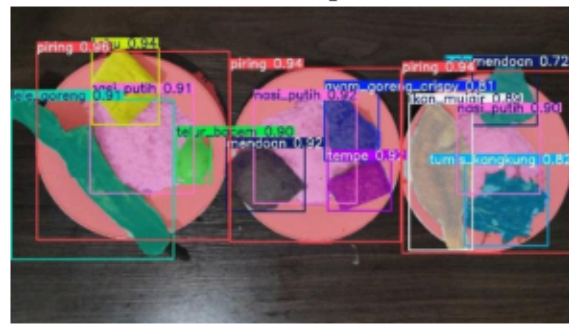
Table 4.19 Hasil Evaluasi Pengujian No Overlapping ke-2

Piring	Ground Truth	Hasil Deteksi	RC	Deteksi			
				TP	FP	FN	TF
Piring 1	Ikan Mujair, Tahu, Nasi Putih	Ikan Mujair, Tahu, Lele Goreng, Nasi Putih	86%	3	1	0	0
Piring 2	Ayam Goreng, Tahu, Nasi Putih	Ayam Goreng, Tahu, Tempe, Nasi Putih	86%	3	1	0	0
Piring 3	Lele Goreng, Telur Bacem, Nasi Putih	Lele Goreng, Telur Bacem, Nasi Putih	100%	3	0	0	0
Piring 4	Lele Goreng, Bakwan, Nasi Putih	Lele Goreng, Bakwan, Mendoan, Nasi Putih	86%	3	1	0	0
Piring 5	Ikan Mujair, Bakwan, Nasi Putih	Ikan Mujair, Bakwan, Mendoan, Nasi Putih	86%	3	1	0	0
Piring 6	Ayam Goreng, Tumisan Kangkung, Nasi Putih	Ayam Goreng, Tumisan Kangkung, Nasi Putih	100%	3	0	0	0

(a) Sebelum Segmentasi



(b) Sesudah Segmentasi



Gambar 4.31 Pengujian No Overlapping ke-3

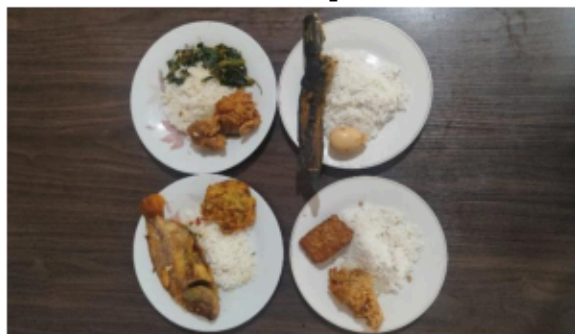
Pada gambar 4.31 menunjukkan hasil pengujian sistem pada kondisi *no overlapping* dengan tiga wadah makanan. Pada piring 1, sistem berhasil mendeteksi semua objek makanan lele goreng, telur bacem, tahu, serta nasi putih (4TP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 1 memperoleh nilai sebesar 100%. Pada piring 2, sistem berhasil mendeteksi semua objek makanan yakni ayam goreng, mendoan, tempe, dan nasi putih (4TP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 2 memperoleh nilai sebesar 100%. Sementara itu, pada piring 3 sistem berhasil mendeteksi objek makanan ikan mujair, bakwan, tumis kangkung, dan nasi putih

(4TP), tetapi terdapat objek yang terklasifikasikan secara berlebihan yakni mendoan (1FP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 3 memperoleh nilai sebesar 80%. Hasil dari evaluasi pengujian *no overlapping* ke-3 dapat dilihat pada tabel 4.20 berikut.

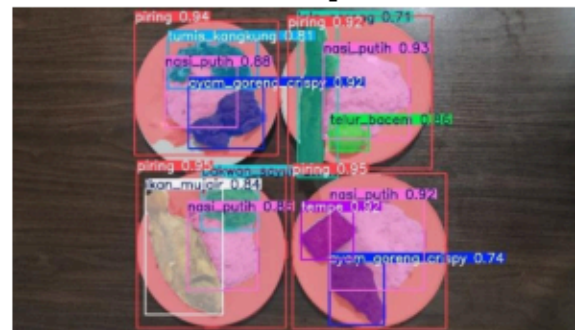
Table 4.20 Hasil Evaluasi Pengujian No Overlapping ke-3

Piring	Ground Truth	Hasil Deteksi	RC	Deteksi			
				TP	FP	FN	TF
Piring 1	Lele Goreng, Telur Bacem, Tahu, Nasi Putih	Lele Goreng, Telur Bacem, Tahu, Nasi Putih	100%	4	0	0	0
Piring 2	Ayam Goreng, Mendoan, Tempe, Nasi Putih	Ayam Goreng, Mendoan, Tempe, Nasi Putih	100%	4	0	0	0
Piring 3	Ikan Mujair, Bakwan, Tumis Kangkung, Nasi Putih	Ikan Mujair, Bakwan, Mendoan Tumis Kangkung, Nasi Putih	80%	4	1	0	0

(a) Sebelum Segmentasi



(b) Sesudah Segmentasi



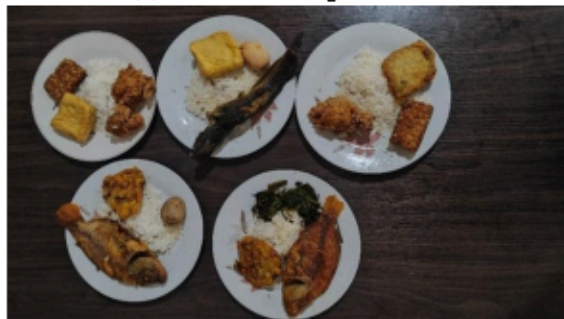
Gambar 4.32 Pengujian No Overlapping ke-4

Pada gambar 4.32 menunjukkan hasil pengujian sistem pada kondisi *no overlapping* dengan empat wadah makanan. Pada piring 1, sistem berhasil mendeteksi semua objek makanan yakni ayam goreng, tumis kangkung, serta nasi putih (3TP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 1 memperoleh nilai sebesar 100%. Pada piring 2, sistem berhasil mendeteksi semua objek makanan yakni lele goreng, telur bacem, dan nasi putih (3TP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 2 memperoleh nilai sebesar 100%. Sementara itu, pada piring 3 sistem berhasil mendeteksi semua objek makanan yakni ikan mujair, bakwan, dan nasi putih (3TP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 3 memperoleh nilai sebesar 100%. Pada piring 4 sistem berhasil mendeteksi ayam goreng, tempe, serta nasi putih (3TP), tetapi terdapat objek yang terklasifikasikan secara berlebihan yakni tempe (1FP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 4 memperoleh nilai sebesar 86%. Hasil dari evaluasi pengujian *no overlapping* ke-4 dapat dilihat pada tabel 4.21 berikut

Table 4.21 Hasil Evaluasi Pengujian No Overlapping ke-4

Piring	Ground Truth	Hasil Deteksi	RC	Deteksi			
				TP	FP	FN	TF
Piring 1	Ayam Goreng, Turnis Kangkung, Nasi Putih	Ayam Goreng, Turnis Kangkung, Nasi Putih	100%	3	0	0	0
Piring 2	Lele Goreng, Telur Bacem, Nasi Putih	Lele Goreng, Telur Bacem, Nasi Putih	100%	3	0	0	0
Piring 3	Ikan Mujair, Bakwan, Nasi Putih	Ikan Mujair, Bakwan, Nasi Putih	100%	3	0	0	0
Piring 4	Ayam Goreng, Tempe, Nasi Putih	Ayam Goreng, Tempe, Tempe, Nasi Putih	86%	3	1	0	0

(a) Sebelum Segmentasi



(b) Sesudah Segmentasi



Gambar 4.33 Pengujian No Overlapping ke-5

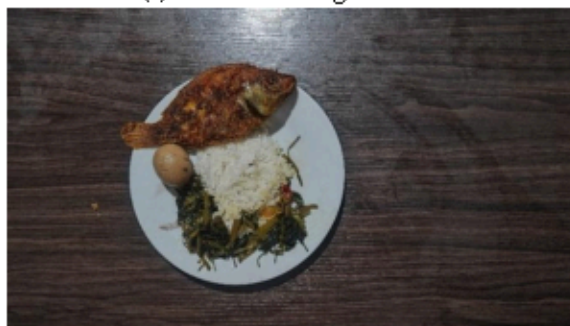
Pada gambar 4.33 menunjukkan hasil pengujian sistem pada kondisi *no overlapping* dengan lima wadah makanan. Pada piring 1, sistem berhasil mendeteksi objek makanan ayam goreng, tahu, tempe, serta nasi putih (4TP), tetapi terdapat objek yang terklasifikasikan secara berlebihan yakni ayam goreng (1FP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 1 memperoleh nilai sebesar 80%. Pada piring 2, sistem berhasil mendeteksi semua objek makanan yakni lele goreng, tahu, telur bacem, dan nasi putih (4TP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 2 memperoleh nilai sebesar 100%. Sementara itu, pada piring 3 sistem berhasil mendeteksi semua objek makanan yakni ayam goreng, mendoan, tempe, dan nasi putih (4TP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 3 memperoleh nilai sebesar 100%. Pada piring 4 sistem berhasil mendeteksi ikan mujair, telur bacem, serta nasi putih (3TP). Namun pada *ground truth* bakwan objek tidak terdeteksi (1FN) dan terklasifikasikan sebagai objek mendoan (1FP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 4 memperoleh nilai sebesar 75%. Sedangkan pada piring 5, sistem berhasil mendeteksi ikan mujair, bakwan, tumis kangkung, serta nasi putih (4TP), tetapi terdapat objek yang terklasifikasikan secara berlebihan yakni ayam goreng dan tumis kangkung (2FP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada

piring 5 memperoleh nilai sebesar 80%. Hasil dari evaluasi pengujian *no overlapping* ke-5 dapat dilihat pada tabel 4.22 berikut.

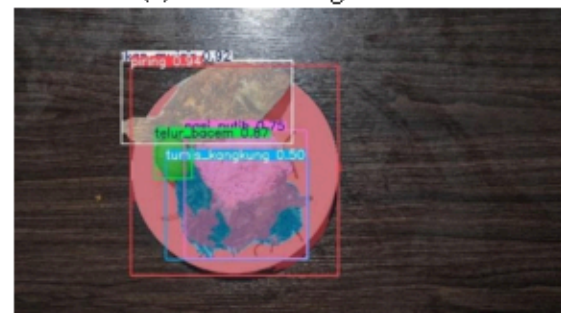
Table 4.22 Hasil Evaluasi Pengujian No Overlapping ke-5

Piring	Ground Truth	Hasil Deteksi	RC	Deteksi			
				TP	FP	FN	TF
Piring 1	Ayam Goreng, Tahu, Tempe, Nasi Putih	Ayam Goreng, Ayam Goreng, Tahu, Tempe, Nasi Putih	80%	4	1	0	0
Piring 2	Lele Goreng, Telur Bacem, Tahu, Nasi Putih	Lele Goreng, Telur Bacem, Tahu, Nasi Putih	100%	4	0	0	0
Piring 3	Ayam Goreng, Mendoan, Tempe, Nasi Putih	Ayam Goreng, Mendoan, Tempe, Nasi Putih	100%	4	0	0	0
Piring 4	Ikan Mujair, Telur Bacem, Bakwan, Nasi Putih	Ikan Mujair, Telur Bacem, Mendoan, Nasi Putih	75%	3	1	1	0
Piring 5	Ikan Mujair, Bakwan, Tumis Kangkung, Nasi Putih	Ikan Mujair, Bakwan, Ayam Goreng, Tumis Kangkung, Tumis Kangkung, Nasi Putih	80%	4	2	0	0

(a) Sebelum Segmentasi



(b) Sesudah Segmentasi



Gambar 4.34 Pengujian No Overlapping ke-6

Pada gambar 4.34 meunjukkan hasil pengujian sistem pada kondisi *no overlapping* dengan satu wadah makanan. Pada piring 1, sistem berhasil mendeteksi semua objek makanan yakni ikan mujair, telur bacem, tumis kangkung, serta nasi putih (4TP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 1 memperoleh nilai sebesar 100%. Hasil dari evaluasi pengujian *no overlapping* ke-6 dapat dilihat pada tabel 4.23 berikut.

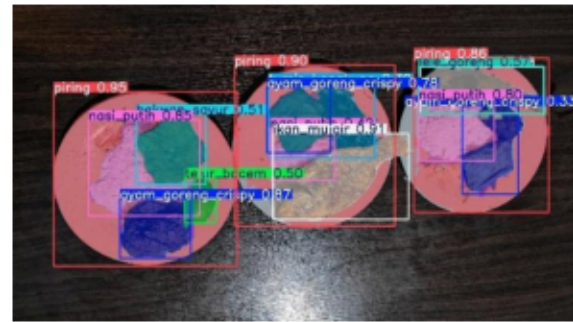
Table 4.23 Hasil Evaluasi Pengujian No Overlapping ke-6

Piring	Ground Truth	Hasil Deteksi	RC	Deteksi			
				TP	FP	FN	TF
Piring 1	Ikan Mujair, Telur Bacem, Tumis Kangkung, Nasi Putih	Ikan Mujair, Telur Bacem, Tumis Kangkung, Nasi Putih	100%	4	0	0	0

(a) Sebelum Segmentasi



(b) Sesudah Segmentasi



Gambar 4.35 Pengujian No Overlapping ke-7

Pada gambar 4.35 meunjukkan hasil pengujian sistem pada kondisi *no overlapping* dengan tiga wadah makanan. Pada piring 1, sistem berhasil mendeteksi objek makanan ayam goreng, bakwan, telur bacem, serta nasi putih (4TP), tetapi terdapat objek yang terklasifikasi secara berlebihan yakni mendoan (1FP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 1 memperoleh nilai sebesar 80%. Pada piring 2, sistem berhasil mendeteksi ikan mujair, tumis kangkung, dan nasi putih (3TP). Namun pada *ground truth* mendoan objek tidak terdeteksi (1FN) dan terklasifikasi sebagai ayam goreng (1FP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 2 memperoleh nilai sebesar 75%. Sementara itu, pada piring 3 sistem berhasil mendeteksi lele goreng, dan nasi putih (2TP). Namun pada *ground truth* tahu dan tempe objek tidak terdeteksi (2FN) dan terklasifikasi sebagai objek ikan mujair dan ayam goreng (2FP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 3 memperoleh nilai sebesar 50%. Hasil dari evaluasi pengujian *no overlapping* ke-7 dapat dilihat pada tabel 4.24 dan tavel 4.25 berikut.

Table 4.24 Hasil Evaluasi Pengujian No Overlapping ke-7

Piring	Ground Truth	Hasil Deteksi	RC	Deteksi			
				TP	FP	FN	TF
Piring 1	Ayam Goreng, Bakwan, Telur Bacem, Nasi Putih	Ayam Goreng, Bakwan, Telur Bacem, Mendoan, Nasi Putih	80%	4	1	0	0
Piring 2	Ikan Mujair, Mendoan, Tumis Kangkung, Nasi Putih	Ikan Mujair, Ayam Goreng, Tumis Kangkung, Nasi Putih	75%	3	1	1	0

Table 4.25 Hasil Evaluasi Pengujian No Overlapping ke-7 (Lanjutan)

Piring	Ground Truth	Hasil Deteksi	RC	Deteksi			
				TP	FP	FN	TF
Piring 3	Lele Goreng, Tahu, Tempe, Nasi Putih	Lele Goreng, Ikan Mujair, Ayam Goreng, Nasi Putih	50%	2	2	2	0

(a) Sebelum Segmentasi



(b) Sesudah Segmentasi



Gambar 4.36 Pengujian No Overlapping ke-8

Pada gambar 4.36 menunjukkan hasil pengujian sistem pada kondisi *no overlapping* dengan empat wadah makanan. Pada piring 1, sistem berhasil mendeteksi objek makanan ikan mujair, mendoan, serta nasi putih (3TP). Namun pada *ground truth* bakwan objek tidak terdeteksi (1FN) dan terklasifikasi sebagai objek mendoan (1FP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 1 memperoleh nilai sebesar 75%. Pada piring 2, sistem berhasil mendeteksi semua objek makanan yakni lele goreng, telur bacem, tempe, dan nasi putih (4TP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 2 memperoleh nilai sebesar 100%. Sementara itu, pada piring 3 sistem berhasil mendeteksi ikan mujair, bakwan, tahu, dan nasi putih (4TP), tetapi terdapat objek yang terklasifikasikan secara berlebihan yakni objek mendoan (1FP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 3 memperoleh nilai sebesar 80%. Pada piring 4 sistem berhasil mendeteksi semua objek makanan yakni ayam goreng, mendoan, turnis kangkung, serta nasi putih (4TP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 4 memperoleh nilai sebesar 100%. Hasil dari evaluasi pengujian *no overlapping* ke-8 dapat dilihat pada tabel 4.26 dan tabel 4.27 berikut.

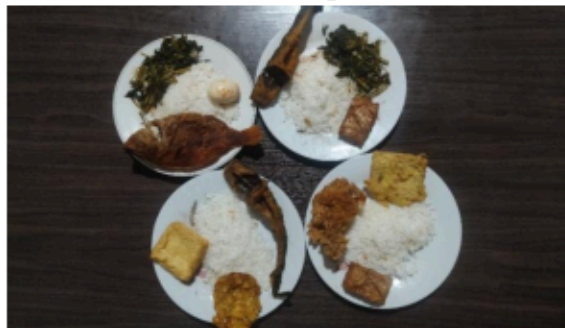
Table 4.26 Hasil Evaluasi Pengujian No Overlapping ke-8

Piring	Ground Truth	Hasil Deteksi	RC	Deteksi			
				TP	FP	FN	TF
Piring 1	Ikan Mujair, Bakwan, Mendoan, Nasi Putih	Ikan Mujair, Mendoan, Mendoan, Nasi Putih	75%	3	1	1	0

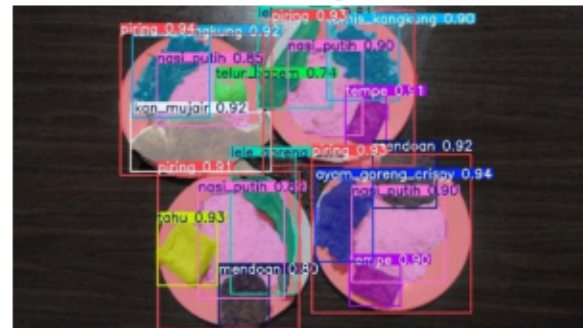
Table 4.27 Hasil Evaluasi Pengujian No Overlapping ke-8 (Lanjutan)

Piring	Ground Truth	Hasil Deteksi	RC	Deteksi			
				TP	FP	FN	TF
Piring 2	Lele Goreng, Telur Bacem, Tempe, Nasi Putih	Lele Goreng, Telur Bacem, Tempe, Nasi Putih	100%	4	0	0	0
Piring 3	Ikan Mujair, Bakwan, Tahu, Nasi Putih	Ikan Mujair, Bakwan, Mendoan Tahu, Nasi Putih	80%	4	1	0	0
Piring 4	Ayam Goreng, Mendoan, Tumris Kangkung, Nasi Putih	Ayam Goreng, Mendoan, Tumris Kangkung, Nasi Putih	100%	4	0	0	0

(a) Sebelum Segmentasi



(b) Sesudah Segmentasi



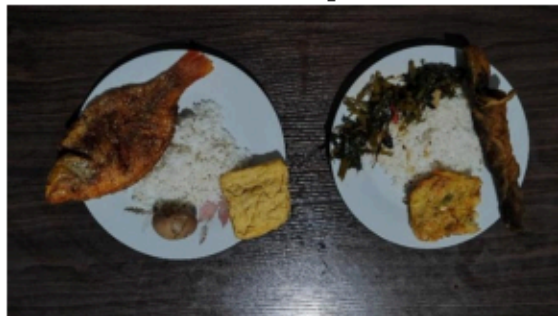
Gambar 4.37 Pengujian No Overlapping ke-9

Pada gambar 4.37 menunjukkan hasil pengujian sistem pada kondisi *no overlapping* dengan empat wadah makanan. Pada piring 1, sistem berhasil mendeteksi semua objek makanan yakni ikan mujair, tumis kangkung, telur bacem, serta nasi putih (4TP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 1 memperoleh nilai sebesar 100%. Pada piring 2, sistem berhasil mendeteksi semua objek makanan yakni lele goreng, tempe, tumis kangkung, dan nasi putih (4TP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 2 memperoleh nilai sebesar 100%. Sementara itu, pada piring 3 sistem berhasil mendeteksi ikan mujair, tahu, dan nasi putih (3TP). Namun *ground truth* bakwan (1FN) tidak terdeteksi dan terklasifikasikan sebagai objek mendoan (1FP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 3 memperoleh nilai sebesar 75%. Pada piring 4 sistem berhasil mendeteksi semua objek makanan yakni ayam goreng, mendoan, tempe, serta nasi putih (4TP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 4 memperoleh nilai sebesar 100%. Hasil dari evaluasi pengujian *no overlapping* ke-9 dapat dilihat pada tabel 4.28 berikut.

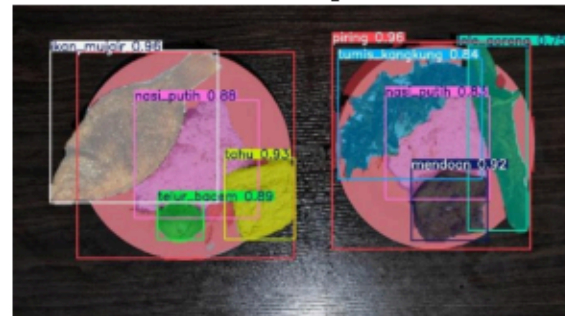
Table 4.28 Hasil Evaluasi Pengujian No Overlapping ke-9

Piring	Ground Truth	Hasil Deteksi	RC	Deteksi			
				TP	FP	FN	TF
Piring 1	Ikan Mujair, Tumis Kangkung, Telur Bacem, Nasi Putih	Ikan Mujair, Tumis Kangkung, Telur Bacem, Nasi Putih	100%	4	0	0	0
Piring 2	Lele Goreng, Tempe, Tumis Kangkung, Nasi Putih	Lele Goreng, Tempe, Tumis Kangkung, Nasi Putih	100%	4	0	0	0
Piring 3	Lele Goreng, Bakwan, Tahu, Nasi Putih	Lele Goreng, Mendoan, Tahu, Nasi Putih	75%	3	1	1	0
Piring 4	Ayam Goreng, Tempe, Mendoan, Nasi Putih	Ayam Goreng, Mendoan, Tempe, Nasi Putih	100%	4	0	0	0

(a) Sebelum Segmentasi



(b) Sesudah Segmentasi



Gambar 4.38 Pengujian No Overlapping ke-10

Pada gambar 4.38 menunjukkan hasil pengujian sistem pada kondisi *no overlapping* dengan dua wadah makanan. Pada piring 1, sistem berhasil mendeteksi semua objek makanan yakni ikan mujair, telur bacem, tahu serta nasi putih (4TP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 1 memperoleh nilai sebesar 100%. Pada piring 2, sistem berhasil mendeteksi lele goreng, tumis kangkung, dan nasi putih (3TP). Namun pada *ground truth* bakwan objek tidak terdeteksi (1FN) dan terklasifikasikan sebagai bakwan (1FP). Berdasarkan perhitungan TP, FP, dan FN, algoritma Ray Casting pada piring 2 memperoleh nilai sebesar 75%. Hasil dari evaluasi pengujian *no overlapping* ke-10 dapat dilihat pada tabel 4.29 dan tabel 4.30 berikut.

Table 4.29 Hasil Evaluasi Pengujian No Overlapping ke-10

Piring	Ground Truth	Hasil Deteksi	RC	Deteksi			
				TP	FP	FN	TN
Piring 1	Ikan Mujair, Telur Bacem, Tahu, Nasi Putih	Ikan Mujair, Telur Bacem, Tahu, Nasi Putih	100%	4	0	0	0

Table 4.30 Hasil Evaluasi Pengujian No Overlapping ke-10 (Lanjutan)

Piring	Ground Truth	Hasil Deteksi	RC	Deteksi			
				TP	FP	FN	TN
Piring 1	Ikan Mujiar, Telur Bacem, Tahu, Nasi Putih	Ikan Mujiar, Telur Bacem, Tahu, Nasi Putih	100%	4	0	0	0
Piring 2	Lele Goreng, Bakwan, Tumis Kangkung, Nasi Putih	Lele Goreng, Mendoan, Tumis Kangkung, Nasi Putih	75%	3	1	1	0

Pengujian dilakukan terhadap 20 gambar yang terdiri dari 10 gambar kondisi *non-overlapping* dan 10 gambar kondisi *overlapping*. Setiap gambar diuji menggunakan model YOLOv8-seg untuk mendeteksi objek makanan serta algoritma Ray Casting untuk mengelompokkan objek ke dalam wadah yang sesuai. Evaluasi dilakukan menggunakan parameter True Positive (TP), False Positive (FP), False Negative (FN), True Negative (TN) serta nilai Ray Casting (RC) yang dihitung berdasarkan hasil deteksi objek pada setiap wadah. Hasil evaluasi pengujian *overlapping* dan *no overlapping* dapat dilihat pada tabel 4.31 berikut.

Table 4.31 Evaluasi Pengujian Overlapping dan No Overlapping

Kondisi	Deteksi			Akurasi Ray Casting		
	TP	FP	FN	Recall	Precisiuin	F1-Score
Overlapping	102	21	26	79.7%	82.9%	81.3%
No Overlapping	119	18	7	94%	86.8%	90.2%
Rata-Rata	221	39	33	87%	85%	86%

Berdasarkan hasil pengujian pada kondisi *overlapping*, performa sistem mengalami penurunan dibandingkan kondisi *non-overlapping*. Hal ini disebabkan oleh adanya objek makanan yang saling bertumpang tindih sehingga sebagian fitur objek tertutupi dan sulit dikenali. Hasil confusion matrix menunjukkan peningkatan nilai FP dan FN pada beberapa pengujian *overlapping*. Kesalahan yang sering terjadi meliputi objek yang tidak terdeteksi serta kesalahan klasifikasi antar makanan yang memiliki bentuk dan warna serupa. Meskipun demikian, sistem masih mampu mendeteksi sebagian besar objek makanan dengan baik dan algoritma Ray Casting tetap dapat mengelompokkan objek hasil deteksi ke dalam wadah yang sesuai dengan total TP 102, FP 21, serta FN 26. Untuk nilai akurasi ray casting untuk recall 79.7%, precision 82.9%, dan F1-score sebesar 81.3%

Sementara itu hasil dari pengujian *no overlapping*, sistem menunjukkan performa deteksi yang baik dengan nilai TP yang lebih dominan dibandingkan FP dan FN. Kondisi *non-overlapping* memungkinkan setiap objek makanan terlihat lebih jelas tanpa adanya objek yang saling menutupi, sehingga model YOLOv8-seg dapat melakukan segmentasi secara lebih optimal. Kesalahan deteksi yang terjadi umumnya disebabkan oleh kemiripan bentuk dan tekstur antar objek makanan, seperti ayam goreng dan lele goreng. Namun, jumlah kesalahan tersebut relatif kecil dibandingkan jumlah objek yang berhasil dideteksi dengan benar. Selain itu, algoritma Ray Casting mampu mengelompokkan sebagian besar

objek hasil deteksi ke dalam wadah yang sesuai dengan total TP 119, FP 18, FN 7. Untuk nilai akurasi ray casting untuk recall 94%, precision 86.6%, dan F1-score sebesar 90.2%

Berdasarkan pengujian yang telah dilakukan, kondisi *no overlapping* menghasilkan performa yang lebih baik dibandingkan kondisi *overlapping*. Hal ini ditunjukkan oleh nilai True Positive (TP) yang lebih tinggi serta nilai False Positive (FP) dan False Negative (FN) yang lebih rendah pada kondisi *no overlapping*. Kondisi tersebut menunjukkan bahwa model mampu mengenali objek makanan dengan lebih baik ketika batas visual antar objek terlihat jelas dan tidak saling menutupi. Sebaliknya, pada kondisi *overlapping* terjadi peningkatan kesalahan deteksi akibat objek makanan saling bertumpang tindih sehingga sebagian fitur objek tidak dapat dikenali secara optimal oleh model. Akibatnya, beberapa objek mengalami kesalahan klasifikasi maupun tidak terdeteksi sama sekali. Kondisi *overlapping* juga menyebabkan proses segmentasi objek menjadi lebih kompleks karena area antar objek saling beririsan. Hal tersebut berdampak pada penurunan nilai RC pada pengujian *overlapping* dibandingkan kondisi *no overlapping*. Berdasarkan hasil evaluasi secara keseluruhan, nilai akurasi Ray Casting dari pengujian *overlapping* dan *no overlapping* memperoleh recall sebesar 87%, precision sebesar 85%, dan F1-score sebesar 86%, sehingga menunjukkan bahwa algoritma Ray Casting cukup efektif dalam mendukung proses pengelompokan objek makanan berdasarkan wadahnya.

Pada pengujian sistem yang telah dilakukan, sebagian besar objek makanan berhasil terdeteksi dengan baik yang ditunjukkan oleh nilai TP yang dominan pada setiap kelas, seperti nasi putih yang memiliki jumlah deteksi benar paling tinggi. Namun, masih terdapat beberapa kesalahan klasifikasi pada objek makanan yang memiliki bentuk dan tekstur serupa, seperti bakwan sayur dengan mendoan, bakwan sayur dengan ayam goreng, serta ikan mujair dengan ayam goreng. Selain itu, kondisi *overlapping* menyebabkan beberapa objek tidak terdeteksi dan diklasifikasikan sebagai background, terutama pada kelas tempe, telur bacem, tumis kangkung, dan tahu. Hal ini menunjukkan bahwa kondisi objek yang saling bertumpang tindih memengaruhi performa deteksi model karena sebagian fitur objek tertutupi. Meskipun demikian, secara keseluruhan model masih mampu mendeteksi sebagian besar objek makanan dengan baik pada berbagai variasi jumlah wadah, jumlah objek, dan kondisi *overlapping* maupun *non-overlapping*.

4.2 Pembahasan

Pada penelitian ini, sistem deteksi makanan dan pengelompokan objek berdasarkan wadah menggunakan model YOLOv8-seg dan algoritma Ray Casting diuji dengan beberapa parameter tuning. Pengujian parameter dilakukan dengan memvariasikan epoch, learning rate, batch size, serta IoU NMS untuk memperoleh konfigurasi model yang optimal. Selain itu pengujian dilakukan menggunakan 20 gambar, yang mencakup kombinasi jumlah wadah, jumlah objek, serta kondisi objek *no-overlapping* dan *overlapping*. Evaluasi performa sistem dilakukan menggunakan confusion matrix untuk mengetahui kemampuan model dalam mendeteksi objek secara benar maupun kesalahan deteksi yang terjadi pada setiap kelas makanan.

Berdasarkan hasil pengujian parameter pada Tabel 4.1 dan Tabel 4.3, menunjukkan perubahan kombinasi dataset, epoch, learning rate, batch size, dan IoU NMS memberikan

pengaruh yang signifikan terhadap performa model. Penggunaan dataset primer terbukti meningkatkan hasil secara drastis dibanding baseline yang menggunakan dataset sekunder. Saat model baseline diuji dengan data uji, model ini hanya memiliki nilai box recall, precision, mAP@50, mAP50-95 berturut turut sebesar 51.3%, 34%, 35.9%, dan 27.5%. Selain itu, penurunan learning rate dari 0.01 menjadi 0.001 serta penyesuaian batch size dari 16 menjadi 8 mampu menghasilkan performa yang lebih optimal pada data testing. Hasil terbaik diperoleh pada Tuning 5 dengan konfigurasi dataset primer, epoch 50, learning rate 0,001, batch size 8, dan IoU NMS 0.7 yang menghasilkan nilai Box Recall 83.9%, Precision 79.4%, mAP@50 82.6%, mAP@50-95 62.5%, serta Mask Recall 83.7%, Precision 79.6%, mAP@50 82.1%, dan mAP@50-95 58.4%. Hasil tersebut menunjukkan bahwa proses tuning parameter dan penambahan dataset primer mampu meningkatkan kemampuan model dalam mendeteksi objek makanan dan wadah secara lebih akurat. Sementara itu, perubahan nilai IoU NMS pada Tuning 6 dan Tuning 7 tidak memberikan peningkatan performa yang lebih baik dibandingkan Tuning 5 sehingga konfigurasi pada Tuning 5 dipilih sebagai model terbaik pada penelitian ini.

Hasil penelitian juga menunjukkan bahwa pada kondisi *no-overlapping*, algoritma ray casting mampu mendeteksi seluruh objek makanan dengan baik pada berbagai jumlah wadah dan objek. Hal ini menandakan bahwa model memiliki kemampuan yang baik dalam mengenali objek ketika batas visual antar objek terlihat jelas. Namun, pada kondisi *overlapping*, performa deteksi cenderung menurun, yang ditandai dengan terjadinya kegagalan deteksi objek (False Negative) serta kesalahan klasifikasi antar kelas yang memiliki kemiripan visual, seperti bakwan yang terdeteksi sebagai mendoan ataupun ayam goreng (False Positive dan False Negative).

Hasil evaluasi pada tabel 4.31 menunjukkan bahwa pengujian *no-overlapping* memiliki nilai F1-Score Ray Casting sebesar 90.2%. Sedangkan pada pengujian *overlapping* nilai F1-Score Ray Casting sebesar 81.3%. Nilai rata-rata Ray Casting dari kedua kondisi tersebut menghasilkan recall 87%, precision 85%, dan F1-Score 86%. Nilai Ray Casting *no-overlapping* yang lebih tinggi dibandingkan *overlapping* menunjukkan bahwa pengujian *no-overlapping* cenderung akurat dan lebih mudah dideteksi daripada pengujian *overlapping*. Hal ini menunjukkan bahwa algoritma Ray Casting efektif dalam menentukan keterkaitan antara objek dan wadah berdasarkan perhitungan posisi geometris.

Implementasi sistem secara real-time juga berhasil dilakukan menggunakan kamera sebagai input utama untuk menangkap citra makanan secara langsung pada lingkungan rumah makan prasmanan. Sistem mampu menampilkan hasil deteksi, segmentasi, serta pengelompokan objek makanan secara otomatis pada antarmuka aplikasi. Meskipun sistem telah berjalan dengan baik, penelitian ini masih memiliki beberapa keterbatasan, seperti kesulitan mendeteksi objek pada kondisi overlapping yang tinggi, variasi pencahayaan, serta kemiripan visual antar objek makanan. Oleh karena itu, pengembangan lebih lanjut dapat dilakukan dengan menambahkan metode relationship detection, spatial reasoning, ataupun optimisasi proses anotasi dan konfigurasi model untuk meningkatkan kemampuan sistem dalam memahami hubungan antar objek pada kondisi kompleks.

BAB V PENUTUP

5.1 Kesimpulan

Berdasarkan hasil dari penelitian sistem deteksi dan mengelompokkan makanan dengan wadah dapat ditarik kesimpulan berupa:

1. Model YOLOv8-seg mampu mendeteksi objek makanan dan wadah dengan baik pada sistem self-service prasmanan. Hal tersebut dibuktikan melalui hasil pengujian data testing dengan parameter penambahan dataset primer, epoch 50, learning rate 0.001, batch size 8, serta IoU 0.7. Hasil pelatihan menunjukkan nilai precision sebesar 83.7%, recall sebesar 79.6%, mAP@50 sebesar 82.1%, dan mAP@50-95 sebesar 58.4%. Nilai tersebut mengalami peningkatan sebesar 39.7% pada mAP@50-95 dibandingkan model baseline, sehingga menunjukkan bahwa proses tuning parameter dan penambahan dataset primer mampu meningkatkan performa model secara signifikan. Selain itu, hasil evaluasi pengujian menunjukkan bahwa sebagian besar objek makanan berhasil terdeteksi dengan benar, terutama pada kondisi *no overlapping* dengan F1-Score 91.3%. Namun, pada kondisi *overlapping* masih ditemukan beberapa kesalahan klasifikasi dan objek yang tidak terdeteksi akibat objek makanan saling bertumpang tindih dengan F1-Score 90.2%. Penerapan algoritma Ray Casting juga menunjukkan bahwa objek makanan yang berhasil dideteksi dapat dikelompokkan ke dalam wadah yang sesuai dengan tingkat akurasi recall sebesar 87%, precision 85%, dan F1-Score 86%. Hasil tersebut menunjukkan bahwa kombinasi YOLOv8-seg dan algoritma Ray Casting cukup efektif digunakan untuk mendukung proses identifikasi dan pengelompokan makanan secara otomatis.
2. Implementasi sistem secara real-time pada lingkungan rumah makan prasmanan berhasil dilakukan menggunakan kamera sebagai input utama. Sistem mampu menampilkan hasil deteksi, segmentasi, serta pengelompokan objek makanan secara langsung pada antarmuka aplikasi. Hasil penelitian menunjukkan bahwa kombinasi YOLOv8-seg dan algoritma Ray Casting efektif digunakan untuk mendukung otomatisasi sistem identifikasi dan pengelompokan makanan pada layanan self-service prasmanan secara real-time.

5.2 Saran

Dalam pengembangan penelitian ini, beberapa saran untuk pengembangan lebih lanjut dalam mendeteksi makanan dan mengelompokkannya berdasarkan wadah yakni:

1. Pengembangan lebih lanjut berupa model yang mampu mendeteksi sekaligus mengelompokkan hasil deteksi ke dalam strata yang telah dideteksi sebelumnya sesuai dengan kondisi spasial atau hubungan lainnya.
2. Menelusuri lebih lanjut pengaruh jumlah objek, wadah, dan kondisi *overlapping* mengenai anotasi data, perbandingan hasil anotasi antara pengelompokan objek berdasarkan hubungan fungsional atau hanya berdasarkan hubungan spasial warna objek.

DAFTAR PUSTAKA

- Agarwal, R., Choudhury, T., Ahuja, N. J., & Sarkar, T. (2023). International Journal of Computational Methods and Experimental Measurements IndianFoodNet : Detecting Indian Food Items Using Deep Learning. *International Journal of Computational Methods and Experimental Measurements (IETA)*, 11(4), 221–232. <https://doi.org/10.18280/ijcmem.110403>
- Alzubaidi, L., Zhang, J., Humaidi, A. J., Dujaili, A. Al, Duan, Y., Shamma, O. Al, Santamaría, J., Fadhel, M. A., Amidie, M. Al, & Farhan, L. (2021). Review of deep learning : concepts , CNN architectures , challenges , applications , future directions. *Journal of Big Data*, 8, 1–74. <https://doi.org/10.1186/s40537-021-00444-8>
- Ayu, S. S., Hasibuan, M., Saragih, T. H. P. S., & Nurbaiti, N. (2023). Analisis Strategi Rumah Makan Padang Sinar Dalam Memenuhi Customers Expectations. *Jurnal Ekonomi, Manajemen Pariwisata Dan Perhotelan*, 3(1), 39–49. <https://doi.org/10.55606/jempper.v3i1.2379>
- Bingol, E. C., & Al-Raweshidy, H. (2025). From Benchmarking to Optimisation: A Comprehensive Study of Aircraft Component Segmentation for Apron Safety Using YOLOv8-Seg. *Applied Sciences*, 15(21), 11582. <https://doi.org/10.3390/app152111582>
- Bolya, D., Zhou., C., Xiao., F., & Lee, Y. J. (2019). YOLACT Real-time Instance Segmentation. *IEEE/CVF International Conference on Computer Vision (ICCV 2019)*, 9157–9166. <https://doi.org/10.1109/ICCV.2019.00925>
- Chan, C. Y. T., & Petrikat, D. (2022). Teknologi Layanan Mandiri: Manfaat dan Tantangan. *Jurnal Ilmu Komputer Dan Studi Teknologi Jurnal*, 4(2), 118–127. <https://doi.org/10.32996/jcsts>
- Chopra., M., & Purwar, A. (2022). Food Image Recognition Using CNN, Faster R- CNN and YOLO. *Applications of Artificial Intelligence, Big Data and Internet of Things in Sustainable Development*, 81–89. <https://doi.org/10.1201/9781003245469>
- Dani., & Anwar, A. N. (2024). *Pengolahan citra digital dalam klasifikasi gender menggunakan OpenCV* (P. Rosyani, Ed.). Eureka Media Aksara. <https://repository.penerbiteitureka.com/id/publications/568738/pengolahan-citra-digital-dalam-klasifikasi-gender-menggunakan-opencv#cite>
- Fahturaman, R., Hariyani, Y. S., & Hadiyoso, S. (2023). Klasifikasi Jajanan Tradisional Indonesia berbasis Deep Learning dan Metode Transfer Learning. *ELKOMIKA: Jurnal Teknik Energi Elektrik, Teknik Telekomunikasi, & Teknik Elektronika*, 11(4), 945. <https://doi.org/10.26760/elkomika.v11i4.945>
- Feng, X., Jiang, Y., Yang, X., Du, M., & Li, X. (2019). Computer vision algorithms and hardware implementations : A survey. *Integration, the VLSI Journal*, 69, 309–320.

<https://doi.org/10.1016/j.vlsi.2019.07.005>

- Galetzka, M., & Glauner, P. (2017). A Simple and Correct Even-Odd Algorithm for the Point-in-Polygon Problem for Complex Polygons. *Proceedings of the 12th International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications (VISIGRAPP 2017)*, 175–178. <https://doi.org/10.5220/0006040801750178>
- Girshick, R. (2015). Fast R-CNN. *Proceedings of the IEEE International Conference on Computer Vision (ICCV 2015)*, 1440–1448. <https://doi.org/10.1109/ICCV.2015.169>
- Hariharan, B., Arbel, P., Girshick, R., & Malik, J. (2014). Simultaneous Detection and Segmentation. In *European Conference on Computer Vision*, 297–312. https://doi.org/10.1007/978-3-319-10584-0_20
- He, K., Gkioxari, G., Dollar, P., & Girshick, R. (2017). Mask R-CNN. *IEEE International Conference on Computer Vision Mask*, 2961–2969. <https://doi.org/10.1109/ICCV.2017.322>
- Heriyanti, F., & Ishak, A. (2020). Design of logistics information system in the finished product warehouse with the waterfall method : review literature Design of logistics information system in the finished product warehouse with the waterfall method : review literature. *IOP Conference Series: Materials Science and Engineering*. <https://doi.org/10.1088/1757-899X/801/1/012100>
- Hussain, M. (2023). YOLO-v1 to YOLO-v8 , the Rise of YOLO and Its Complementary Nature toward Digital Manufacturing and Industrial Defect Detection. *11(7)*, 677. <https://doi.org/10.3390/machines11070677>
- Ioffe, S., & Szegedy, C. (2015). Batch Normalization : Accelerating Deep Network Training by Reducing Internal Covariate Shift. *Proceedings of Machine Learning Research*, 37, 448–456. <https://doi.org/https://proceedings.mlr.press/v37/ioffe15.html>
- Jain, A., Rathi, K., Ganguly, Y., Kumar, A., & Bhale, Y. (2025). A Comparative Analysis of DBSCAN , K-Means and Agglomerative Clustering Algorithms for Geospatial Data. *Proceedings of the International Conference on Recent Advancement and Modernization in Sustainable Intelligent Technologies & Applications (RAMSITA-2025)*, 212–221. <https://doi.org/10.2991/978-94-6463-716-8>
- Jamil, J. B., Afiz, N. I. B. A. A., Jalil, P. N. R. B. A., & Sumar, M. B. (2019). CUSTOMER ACCEPTANCE TOWARDS SELF-SERVICE TECHNOLOGY AT MCDONALD ' S. *Tugas Akhir, Politeknik Malaysia Sultan Salahuddin Abdul Aziz Shah, December*, 1–39.
- Khan, A., Sohail, A., Zahoor, U., & Qureshi, A. S. (2020). A Survey of the Recent Architectures of Deep Convolutional Neural Networks. *Artificial Intelligence Review*, 53(8), 5455–5516. <https://doi.org/10.1007/s10462-020-09825-6>

- Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet Classification with Deep Convolutional Neural Networks. *Advances in Neural Information Processing Systems*, 25, 1097–1105. <https://doi.org/10.1145/3065386>
- Kularathne, D., & Jayarathne, L. (2018). Point in Polygon Determination Algorithm for 2-D Vector Graphics Applications. *2018 National Information Technology Conference (NITC)*, 1–5. <https://doi.org/10.1109/NITC.2018.8550057>
- Lecun, Y., Bengio, Y., & Hinton, G. (2015). *Deep learning*. 521(7553), 436–444. <https://doi.org/10.1038/nature14539>
- Lecun, Y., Bottou, L., Bengio, Y., & Ha, P. (1998). Gradient-Based Learning Applied to Document Recognition. *Proceedings of the IEEE*, 86(11), 2278–2324. <https://doi.org/10.1109/5.726791>
- Liu, S., Qi, L., Qin, H., Shi, J., & Jia, J. (2018). Path Aggregation Network for Instance Segmentation. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 8759–8768. <https://doi.org/10.1109/CVPR.2018.00913>
- Meuter, M. L., Ostrom, A. L., Roundtree, R. I., & Bitner, M. J. (2000). Self-Service Technologies : Satisfaction with Technology-Based. *Journal of Marketing*, 64(3), 50–64. <https://doi.org/10.1509/jmkg.64.3.50.18024>
- Nurulhikmah, F., & Abdi, D. N. E. (2024). Classification of Foods Based on Nutritional Content Using K- Means and DBSCAN Clustering Methods. *Teknika - Jurnal Ilmiah Teknologi Dan Rekayasa*, 13(3), 481–486. <https://doi.org/10.34148/teknika.v13i3.1067>
- Padilla, R., Netto, S. L., & Silva, E. A. B. (2020). A Survey on Performance Metrics for Object-Detection Algorithms. *2020 International Conference on Systems, Signals and Image Processing (IWSSIP)*, 237–242. <https://doi.org/10.1109/IWSSIP48289.2020.9145130>
- Purnomo, N., Gata, W., Kusuma, M. R., Dinata, R. M., & Husna, M. B. (2025). Implementasi you only look once. *JATI (Jurnal Mahasiswa Teknik Informatika)*, 9(2), 3476–3483.
- Putri, M., Dewi, C., Siam, E. P., Wijayanti, G. A., Aulia, N., & Nooraeni, R. (2021). Comparison of DBSCAN and K-Means Clustering for Grouping the Village Status in Central Java 2020. *Matematika, Statistika & Komputasi*, 17(3), 394–404. <https://doi.org/10.20956/j.v17i3.11704>
- Ramachandran, P., Barret Zoph., & Le, Q. V. (2017). *SEARCHING FOR ACTIVATION FUNCTIONS*. 1–13. <https://arxiv.org/abs/1710.05941>
- Ren, S., He, K., Girshick, R., & Sun, J. (2016). Faster R-CNN : Towards Real-Time Object Detection with Region Proposal Networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 39(6), 1137–1149. <https://doi.org/10.1109/TPAMI.2016.2577031>

- Ronneberger, O., Fischer, P., & Brox, T. (2015). *U-Net : Convolutional Networks for Biomedical. 9351*, 234–241. https://doi.org/10.1007/978-3-319-24574-4_28
- Sapkota, R., Ahmed, D., & Karkee, M. (2024). Comparing YOLOv8 and Mask R-CNN for instance segmentation in complex orchard environments. *Artificial Intelligence in Agriculture, 13*, 84–99. <https://doi.org/10.1016/j.aiaa.2024.07.001>
- Shelhamer, E., Long, J., & Darrell, T. (2017). Fully Convolutional Networks for Semantic Segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence (IEEE TPAMI)*, 39(4), 640–651. <https://doi.org/10.1109/TPAMI.2016.2572683>
- Stanley, B., Pratama, Y., & Subakti, A. G. (2023). The Impact of Self-Order Kiosk and Service Quality on Customer Experience in McDonald ' s Citra Garden 6 Jakarta. *E3S Web of Conferences*, 426. <https://doi.org/10.1051/e3sconf/202342602073>
- Subedi, A. (2024). Improving Generalization Performance of YOLOv8 for Camera Trap Object Detection Master of Science Department of Computer Science. *Department of Computer Science Of, March*. <https://arxiv.org/pdf/2412.14211>
- Szegedy, C., Liu, W., Jia, Y., Reed, S., Sermanet, P., Anguelov, D., Erhan, D., Vanhoucke, V., & Rabinovich, A. (2015). Going deeper with convolutions. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2015)*, 1–12. <https://doi.org/10.1109/CVPR.2015.7298594>
- Wijaya, A. (2020). (SST) Pada Model Bisnis Restoran. *Restoran. Prosiding Seminar Nasional Dinamika Informatika*, 4(1), 63–66.
- Yamashita, R., Nishio, M., Kin, R., Do, G., & Togashi, K. (2018). Convolutional neural networks : an overview and application in radiology. *Insights into Imaging*, 9(4), 611–629. <https://doi.org/10.1007/s13244-018-0639-9>
- Zhao, L., & Zhang, Z. (2024). A improved pooling method for convolutional neural networks. *Scientific Reports*, 14(Article 1589), 1–22. <https://doi.org/10.1038/s41598-024-51258-6>